
Modern C++ for Python Programmers

From Absolute Beginner to Systems, Graphics, and Game Development

2026

Contents

0.1	Modern C++ for Python Programmers	7
0.1.1	From Absolute Beginner to Systems, Graphics, and Game Development	7
0.1.2	How to Use This Book	7
0.1.3	Table of Contents	7
0.2	Part I – Foundations	8
0.2.1	What Does “Running a Program” Actually Mean?	8
0.2.2	How Python Runs Your Code	8
0.2.3	How C++ Runs Your Code	10
0.2.4	Your First C++ Program	12
0.2.5	Compiling: What the Flags Mean	13
0.2.6	What Is in the Compiled Binary?	14
0.2.7	Multi-File Programs	15
0.2.8	Reading Compiler Errors	15
0.2.9	Common Mistakes in This Chapter	17
0.2.10	Exercises	18
0.2.11	What IS a Variable?	19
0.2.12	How Integers Are Stored: Binary and Two’s Complement	20
0.2.13	The Fundamental Types	22
0.2.14	Fixed-Width Integer Types	24
0.2.15	Declaring and Initializing Variables	24
0.2.16	Memory Layout of Variables	26
0.2.17	auto – Type Deduction	26
0.2.18	const and constexpr	27
0.2.19	Type Conversion	28
0.2.20	Checking Sizes with sizeof	28
0.2.21	Common Mistakes in This Chapter	29
0.2.22	Exercises	30
0.2.23	Arithmetic Operators	32
0.2.24	Increment and Decrement Operators	33
0.2.25	Compound Assignment	34
0.2.26	Comparison Operators	34
0.2.27	Logical Operators	35
0.2.28	Bitwise Operators	36
0.2.29	Operator Precedence	38
0.2.30	The Ternary Operator	38
0.2.31	Common Mistakes in This Chapter	39
0.2.32	Exercises	40
0.2.33	if/else if/else	41
0.2.34	switch	42

0.2.35	while Loop	43
0.2.36	do-while Loop	44
0.2.37	for Loop: C-Style	44
0.2.38	Range-Based for Loop (C++11)	46
0.2.39	break and continue	47
0.2.40	Loop Execution Visualization	47
0.2.41	Common Mistakes in This Chapter	48
0.2.42	Exercises	50
0.2.43	What Happens When You Call a Function	52
0.2.44	Function Syntax	53
0.2.45	Declarations vs Definitions	53
0.2.46	Pass by Value: What Is Copied and Why It Matters	55
0.2.47	Default Parameters	56
0.2.48	Function Overloading	57
0.2.49	Compiler Messages for Function Errors	57
0.2.50	Common Mistakes in This Chapter	59
0.2.51	Exercises	60
0.3	Part II – Core C++ (the part that is not like Python)	62
0.3.1	The Problem References Solve	62
0.3.2	What Is a Reference?	63
0.3.3	Pass by Reference	64
0.3.4	Pass by const Reference	64
0.3.5	References Cannot Be Null	65
0.3.6	Returning References	66
0.3.7	Common Mistakes in This Chapter	66
0.3.8	Exercises	67
0.3.9	The Address of a Variable	68
0.3.10	Modifying Through a Pointer	69
0.3.11	nullptr: The Null Pointer	70
0.3.12	Pointer Arithmetic	70
0.3.13	Pointers vs Python References	71
0.3.14	The -> Operator	72
0.3.15	const and Pointers	72
0.3.16	Common Mistakes in This Chapter	73
0.3.17	Exercises	74
0.3.18	Two Regions of Memory	75
0.3.19	The Stack in Detail	75
0.3.20	The Heap in Detail	76
0.3.21	Why Python Uses the Heap for Everything	77
0.3.22	Memory Leaks	78
0.3.23	Stack vs Heap: Which to Use	79
0.3.24	Common Mistakes in This Chapter	79
0.3.25	Exercises	80
0.3.26	Why const Matters	81
0.3.27	const Variables	81
0.3.28	const Function Parameters	82
0.3.29	const Member Functions	82

0.3.30	<code>constexpr</code> – Compile-Time Constants	83
0.3.31	Cascading <code>const</code>	83
0.3.32	Common Mistakes in This Chapter	84
0.3.33	Exercises	85
0.3.34	C-Style Arrays (Understand, then Avoid)	85
0.3.35	<code>std::vector</code> – The Right Way to Do Dynamic Arrays	87
0.3.36	<code>std::string</code> – The Right Way to Handle Text	88
0.3.37	<code>std::array</code> – Fixed-Size Array With Safety	90
0.3.38	Common Mistakes in This Chapter	90
0.3.39	Exercises	91
0.3.40	Scope	93
0.3.41	Lifetime	94
0.3.42	Organizing Code: The Header/Source Split	94
0.3.43	Namespaces	96
0.3.44	Static Local Variables	98
0.3.45	Common Mistakes in This Chapter	98
0.3.46	Exercises	99
0.4	Part III – Ownership and Memory Management	102
0.4.1	The Problem: Resources Need Cleanup	102
0.4.2	What RAII Is	103
0.4.3	The Destructor	103
0.4.4	RAII for File Handling	104
0.4.5	RAII for Mutex Locks	105
0.4.6	Python’s Equivalent: Context Managers	105
0.4.7	The Lifetime Guarantee Visualized	106
0.4.8	Common Mistakes in This Chapter	106
0.4.9	Exercises	107
0.4.10	<code>new</code> and <code>delete</code> Revisited	108
0.4.11	The Problems With Raw <code>new/delete</code>	109
0.4.12	When Raw <code>new/delete</code> Is Acceptable	110
0.4.13	<code>std::bad_alloc</code> – Out of Memory	110
0.4.14	Common Mistakes in This Chapter	111
0.4.15	Exercises	111
0.4.16	The Core Idea	112
0.4.17	<code>std::unique_ptr</code> – Exclusive Ownership	112
0.4.18	<code>std::shared_ptr</code> – Shared Ownership	114
0.4.19	<code>std::weak_ptr</code> – Non-Owning Observer	115
0.4.20	Choosing the Right Smart Pointer	116
0.4.21	Common Mistakes in This Chapter	116
0.4.22	Exercises	117
0.4.23	The Performance Problem With Copies	119
0.4.24	lvalues and rvalues	119
0.4.25	The Move Operation	120
0.4.26	<code>std::move</code> – Casting to rvalue	120
0.4.27	Move Semantics and Returned Values	120
0.4.28	Move Constructor and Move Assignment	121
0.4.29	<code>std::string</code> and <code>std::vector</code> Move in Action	122

0.4.30	When C++ Moves Automatically	122
0.4.31	Common Mistakes in This Chapter	123
0.4.32	Exercises	123
0.4.33	The Problem: Special Member Functions	124
0.4.34	The Rule of Zero	125
0.4.35	The Rule of Three	125
0.4.36	The Rule of Five	126
0.4.37	<code>= delete</code> and <code>= default</code>	127
0.4.38	Compiler-Generated Functions Are Suppressed When You Define Others	128
0.4.39	The Practical Takeaway	128
0.4.40	Common Mistakes in This Chapter	128
0.4.41	Exercises	129
0.5	Part IV – Object-Oriented Programming	131
0.5.1	Python Classes vs C++ Classes	131
0.5.2	Defining a Class: Full Example	132
0.5.3	<code>struct</code> vs <code>class</code>	133
0.5.4	Memory Layout of a Class	133
0.5.5	<code>this</code> Pointer	134
0.5.6	Static Members	135
0.5.7	Common Mistakes in This Chapter	135
0.5.8	Exercises	136
0.5.9	The Member Initializer List	137
0.5.10	Delegating Constructors (C++11)	138
0.5.11	Constructor Kinds	139
0.5.12	Destructor Revisited: The Full Rules	139
0.5.13	In-Class Member Initializers (C++11)	140
0.5.14	Common Mistakes in This Chapter	140
0.5.15	Exercises	141
0.5.16	Two Kinds of Reuse	142
0.5.17	Inheritance Syntax	142
0.5.18	Constructors in Derived Classes	143
0.5.19	Memory Layout With Inheritance	144
0.5.20	Slicing: The Object-Oriented Trap	144
0.5.21	Composition vs Inheritance	145
0.5.22	Common Mistakes in This Chapter	146
0.5.23	Exercises	146
0.5.24	The Problem Without Virtual	147
0.5.25	<code>virtual</code> Enables Dynamic Dispatch	147
0.5.26	How Virtual Dispatch Works: The <code>vtable</code>	148
0.5.27	<code>override</code> and <code>final</code> (C++11)	149
0.5.28	Virtual Destructor: Why It Is Required	149
0.5.29	Polymorphism With <code>unique_ptr</code>	150
0.5.30	Python Polymorphism vs C++ Polymorphism	150
0.5.31	Common Mistakes in This Chapter	151
0.5.32	Exercises	152
0.5.33	Pure Virtual Functions	153
0.5.34	Concrete Derived Classes	153

0.5.35	Interfaces in C++	154
0.5.36	Partial Implementations (Partially Abstract)	155
0.5.37	Checking the Type at Runtime: <code>dynamic_cast</code>	155
0.5.38	Common Mistakes in This Chapter	156
0.5.39	Exercises	156
0.5.40	What Operator Overloading Is	157
0.5.41	Which Operators to Overload	158
0.5.42	Member vs Non-Member Operators	159
0.5.43	The Spaceship Operator (C++20)	159
0.5.44	<code>operator()</code> – Making Objects Callable	160
0.5.45	Rules and Guidelines for Operator Overloading	160
0.5.46	Common Mistakes in This Chapter	161
0.5.47	Exercises	161
0.6	Part V – Generic Programming	162
0.6.1	The Problem Without Templates	163
0.6.2	Function Templates	163
0.6.3	How Templates Are Compiled	164
0.6.4	Class Templates	165
0.6.5	Non-Type Template Parameters	166
0.6.6	Template Functions in the Standard Library	166
0.6.7	Common Mistakes in This Chapter	167
0.6.8	Exercises	168
0.6.9	Template Specialization	169
0.6.10	<code>if constexpr</code> – Compile-Time Branching	170
0.6.11	Type Traits	171
0.6.12	Variadic Templates	172
0.6.13	Common Mistakes in This Chapter	173
0.6.14	Exercises	174
0.6.15	The Problem With Unconstrained Templates	175
0.6.16	Defining and Using Concepts	175
0.6.17	Standard Library Concepts (<code><concepts></code>)	176
0.6.18	Writing Your Own Concepts	177
0.6.19	Abbreviated Function Templates (C++20)	177
0.6.20	Concepts vs Runtime Type Checks	178
0.6.21	Common Mistakes in This Chapter	178
0.6.22	Exercises	179
0.6.23	What Is Template Metaprogramming?	180
0.6.24	Compile-Time Recursion	180
0.6.25	<code>std::enable_if</code> – Conditional Compilation (Old Style)	181
0.6.26	Type Lists – Compile-Time Lists of Types	181
0.6.27	Compile-Time Lookup Tables	182
0.6.28	<code>std::tuple</code> – Compile-Time Heterogeneous Collections	182
0.6.29	<code>std::conditional</code> – Type Selection at Compile Time	183
0.6.30	The Difference Between TMP and <code>constexpr</code>	183
0.6.31	Common Mistakes in This Chapter	183
0.6.32	Exercises	184

0.7	Part VI – The Standard Library	186
0.7.1	The Container Taxonomy	186
0.7.2	<code>std::vector<T></code> – Your Default Container	186
0.7.3	<code>std::deque<T></code> – Fast at Both Ends	187
0.7.4	<code>std::map<K, V></code> – Sorted Key-Value Store	188
0.7.5	<code>std::unordered_map<K, V></code> – Hash Map	190
0.7.6	<code>std::set<K></code> – Sorted Unique Keys	190
0.7.7	Choosing the Right Container	191
0.7.8	Common Mistakes in This Chapter	191
0.7.9	Exercises	192
0.7.10	What Is an Iterator?	193
0.7.11	Iterator Categories	194
0.7.12	The Iterator-Pair Pattern	194
0.7.13	<code>begin / end</code> Free Functions	195
0.7.14	Reverse Iterators	195
0.7.15	Insert Iterators	195
0.7.16	Stream Iterators	196
0.7.17	Common Mistakes in This Chapter	196
0.7.18	Exercises	197
0.7.19	Why Algorithms	197
0.7.20	The Most Useful Algorithms	198
0.7.21	Common Mistakes in This Chapter	200
0.7.22	Exercises	201
0.7.23	What Is a Lambda?	201
0.7.24	Lambda Syntax	202
0.7.25	Captures: Accessing the Enclosing Scope	202
0.7.26	Lambda Memory Model	203
0.7.27	Generic Lambdas (C++14)	203
0.7.28	Storing Lambdas: <code>std::function</code>	204
0.7.29	Immediately Invoked Lambdas	204
0.7.30	Common Mistakes in This Chapter	205
0.7.31	Exercises	205
0.7.32	The Problem With Iterator Pairs	206
0.7.33	Ranges: The C++20 Solution	207
0.7.34	Views: Lazy, Composable Transformations	207
0.7.35	Common Views	208
0.7.36	Materializing a View	208
0.7.37	Common Mistakes in This Chapter	208
0.7.38	Exercises	209
0.7.39	<code>std::optional<T></code> – A Value That May Not Exist	209
0.7.40	<code>std::variant<Types...></code> – Type-Safe Union	210
0.7.41	<code>std::any</code> – Type-Erased Storage	211
0.7.42	<code>std::tuple</code> Revisited: Structured Bindings and Helpers	212
0.7.43	<code>std::pair<A, B></code> – Two Values	212
0.7.44	Choosing the Right Utility Type	212
0.7.45	Common Mistakes in This Chapter	213
0.7.46	Exercises	213

0.1 Modern C++ for Python Programmers

0.1.1 From Absolute Beginner to Systems, Graphics, and Game Development

Target Standard: C++23 | **Prerequisites:** Comfortable Python, zero C++ assumed

0.1.2 How to Use This Book

Each chapter teaches one concept from scratch. Every concept is first shown in Python (which you know), then explained in C++, then built up with multiple examples, memory diagrams where helpful, common mistakes, and exercises with answers.

Work through Part I completely before moving on. The foundations – types, memory, functions – underpin everything else.

0.1.3 Table of Contents

Part I – Foundations 1. [The Compilation Model and Your First Program](#) 2. [Variables, Types, and the Static Type System](#) 3. [Operators and Expressions](#) 4. [Control Flow: Branching and Loops](#) 5. [Functions, Overloading, and Declarations vs Definitions](#)

Part II – Core C++ (the part that is not like Python) 6. [References – Aliases for Variables](#) 7. [Pointers and Memory Addresses](#) 8. [The Stack and the Heap](#) 9. [const Correctness](#) 10. [Arrays, `std::vector`, and `std::string`](#) 11. [Scope, Lifetime, and Organizing Code into Files](#)

Part III – Ownership and Memory Management 12. [RAII – The Core Idea That Replaces Garbage Collection](#) 13. [Dynamic Allocation: `new`, `delete`, and Why You Avoid Them](#) 14. [Smart Pointers: `unique\_ptr`, `shared\_ptr`, `weak\_ptr`](#) 15. [Move Semantics, lvalues and rvalues](#) 16. [The Rule of 0, 3, and 5](#)

Part IV – Object-Oriented Programming 17. [Classes, Objects, and Encapsulation](#) 18. [Constructors, Destructors, and Initialization](#) 19. [Inheritance and Composition](#) 20. [Virtual Functions and Polymorphism](#) 21. [Abstract Classes and Interfaces](#) 22. [Operator Overloading](#)

Part V – Generic Programming 23. [Function and Class Templates](#) 24. [Template Specialization and Variadic Templates](#) 25. [Concepts \(C++20\) – Compile-Time Duck Typing](#) 26. [An Introduction to Template Metaprogramming](#)

Part VI – The Standard Library 27. [Containers: `vector`, `map`, `set`, `array`, and `friends`](#) 28. [Iterators](#) 29. [Algorithms: `sort`, `find`, `transform`, and the rest](#) 30. [Lambdas and Function Objects](#) 31. [Ranges and Views \(C++20\)](#) 32. [Utility Types: `optional`, `variant`, `any`, `tuple`](#)

Part VII – Modern C++ (C++11 to C++23) 33. [`auto`, type deduction, and structured bindings](#) 34. [`constexpr` and compile-time computation](#) 35. [`std::format` and modern string handling](#) 36. [Coroutines and Generators](#) 37. [Modules \(C++20\)](#)

Part VIII – Performance 38. [Value vs Reference Semantics](#) 39. [How Memory Layout Affects Speed](#) 40. [Data-Oriented Design](#) 41. [Profiling and Flamegraphs](#)

Part IX – Concurrency 42. [Threads and `std::jthread`](#) 43. [Mutexes, Locks, and Race Conditions](#) 44. [Atomics and the C++ Memory Model](#) 45. [Async, Futures, and Tasks](#)

Part X – Graphics and Game Development 46. [The Math: vectors, matrices, quaternions](#) 47. [How the GPU Works](#) 48. [OpenGL Fundamentals](#) 49. [Moving to Vulkan](#) 50. [Game Loop, ECS Architecture, and Engine Design](#)

Part XI – Systems Programming 51. [The Machine: registers, memory, syscalls](#) 52. [Working with the OS and Linux APIs](#) 53. [Networking from the Ground Up](#) 54. [Where C++ Meets C, eBPF, and Go](#)

Appendices - A. [Setting Up Your Toolchain](#) - B. [Compiler Flags Reference](#) - C. [Python to C++ Cheat Sheet](#) - D. [Common Mistakes and How to Debug Them](#)

0.2 Part I – Foundations

Chapter 1: The Compilation Model and Your First Program

0.2.1 What Does “Running a Program” Actually Mean?

Before you write a single line of C++, you need to understand something Python completely hides: what a computer actually does to run your code.

When you type `python hello.py`, a lot of invisible machinery kicks in. When you compile and run a C++ program, that machinery becomes visible – and you control it. That visibility is both the source of C++’s power and the main source of beginner confusion.

0.2.2 How Python Runs Your Code

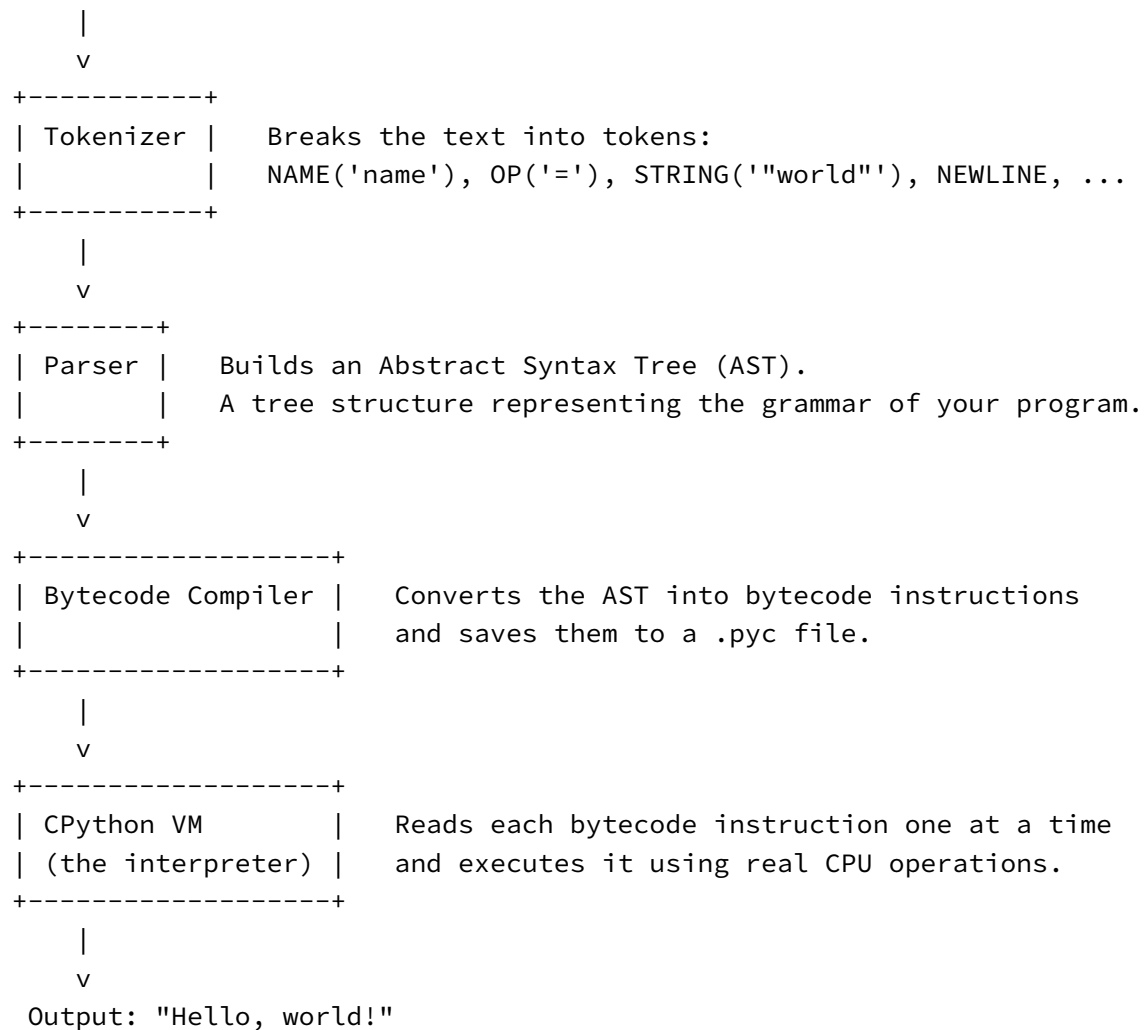
Let’s trace exactly what happens when you run a Python script.

```
# hello.py
name = "world"
print(f"Hello, {name}!")
```

```
$ python hello.py
Hello, world!
```

CPython (the standard Python interpreter) runs your code through five stages:

hello.py (plain text file)



The .pyc file contains instructions for a **virtual machine** – a software CPU that Python invented. These are NOT instructions your real CPU understands. The CPython VM translates each bytecode instruction into real CPU instructions as the program runs.

You can actually see the bytecode:

```

import dis

def greet(name):
    return f"Hello, {name}!"

dis.dis(greet)

```

Output:

```

2          0 RESUME                0

3          2 LOAD_CONST            1 ('Hello, ')
          4 LOAD_FAST              0 (name)

```

6	FORMAT_VALUE	0
8	BUILD_STRING	2
10	RETURN_VALUE	

LOAD_FAST, FORMAT_VALUE, BUILD_STRING – your CPU has never heard of these. The CPython VM reads each one and does the corresponding real work.

The consequence: There is always a middleman between your code and the CPU. Every Python operation passes through the interpreter loop. This is why Python is typically 10 to 100 times slower than C++ for CPU-bound work.

0.2.3 How C++ Runs Your Code

C++ uses a completely different model. Your source code is **translated directly into CPU instructions** before the program runs. There is no interpreter, no virtual machine, no middleman. When you run the program, your CPU executes your instructions directly.

This is called **ahead-of-time compilation**.

The translation is done by a chain of three programs:

```
hello.cpp (plain text, C++ source)
  |
  v
+-----+
| Preprocessor | Handles lines starting with #.
|             | Pastes #include files, substitutes #define macros.
|             | Pure text manipulation -- does NOT understand C++.
+-----+
  |
  | (preprocessed .cpp -- still text, but with all #includes pasted in)
  v
+-----+
| Compiler     | Parses C++, checks all types, optimizes, generates
|             | machine code for your CPU architecture.
+-----+
  |
  | (object file: .o -- real machine code, but references unresolved)
  v
+-----+
| Linker       | Combines all .o files plus library code into one
|             | self-contained executable.
+-----+
  |
  v
./hello (executable -- native machine code, runs directly on CPU)
```

0.2.3.1 Stage 1: The Preprocessor in Detail

The preprocessor is a dumb text tool. It knows only three things:

- `#include <file>` or `#include "file"` – paste the entire text of that file here
- `#define NAME text` – replace every occurrence of `NAME` with `text`
- `#ifdef` / `#ifndef` / `#else` / `#endif` – include or exclude blocks of text

It does not parse C++. It does not understand types. It just manipulates text.

When you write `#include <iostream>`, the preprocessor pastes thousands of lines into your file before the compiler sees anything. You can see how much:

```
$ g++ -E hello.cpp | wc -l
45231
```

One line expands to 45,231 lines. That is why large projects are slow to compile – every `.cpp` file re-expands every header it includes.

0.2.3.2 Stage 2: The Compiler in Detail

The compiler reads the preprocessed text and does the real work:

1. **Parsing:** builds an AST (same concept as Python's parser)
2. **Semantic analysis:** checks types, resolves names, catches errors
3. **Optimization** (with `-O2`): rewrites your code to run faster
4. **Code generation:** emits machine code for your target CPU

The output is an **object file** (`.o`). It contains real machine code, but it has gaps – references to functions defined in other files (like `std::cout`) that haven't been connected yet.

0.2.3.3 Stage 3: The Linker in Detail

The linker takes all object files and the standard library and combines them into one executable. It resolves every function call to an actual memory address. When `main.cpp` calls `std::cout`, the linker finds `cout`'s implementation in the standard library and wires them together.

0.2.3.4 The Core Difference

Python:

source --> bytecode (at first run) --> CPU (via interpreter, every run)

C++:

source --> machine code (once, at compile time) --> CPU (directly, every run)

After you compile once, running the program has zero translation overhead. The CPU just executes instructions.

0.2.4 Your First C++ Program

Create a file called `hello.cpp`:

```
#include <iostream>

int main() {
    std::cout << "Hello, world!\n";
    return 0;
}
```

Compile and run:

```
$ g++ -std=c++23 -o hello hello.cpp
$ ./hello
Hello, world!
```

Now let's understand every token in this program.

0.2.4.1 `#include <iostream>`

A preprocessor directive. Pastes the `iostream` header (thousands of lines) into your file.

`iostream` declares `std::cout`, the standard output stream. Without it:

```
error: 'cout' is not a member of 'std'
  4 |     std::cout << "Hello, world!\n";
    |     ^~~~~~
note: 'std::cout' is defined in header '<iostream>';
     did you forget to '#include <iostream>'?
```

Angle brackets `<iostream>` mean “find this in the compiler’s system include path.” Double quotes `"my-file.h"` mean “look in my project directory first.”

0.2.4.2 `int main()`

The **entry point** of every C++ program. When the OS launches your executable, it calls `main()`. Everything begins here.

`int` is the **return type**. The function returns an integer to the OS as an exit code: `-0` = success (by convention) - Non-zero = some kind of failure

Unlike Python where top-level code runs directly, C++ requires exactly one function called `main` as the entry point. You can’t rename it or have two of them.

The `{ }` braces delimit the function body, playing the same role as `:` and indentation in Python.

0.2.4.3 `std::cout << "Hello, world!\n";`

This is how you print to the terminal.

std is a **namespace** – a named container for related identifiers. The entire C++ standard library lives inside `std`. The `::` operator means “look inside.” So `std::cout` means “the `cout` that lives inside the `std` namespace.”

cout stands for “character output.” It is an object representing your terminal’s standard output.

`<<` is the **stream insertion operator**. It sends the value on the right into the stream on the left. You can chain it:

```
std::cout << "Hello" << ", " << "world" << "!\n";
```

Each `<<` sends one more thing into the stream.

`"Hello, world!\n"` is a string literal. The `\n` is a newline character. Unlike Python’s `print()`, which adds a newline automatically, C++ streams do not add newlines unless you include them.

`;` terminates the statement. Every statement ends with a semicolon. This is the single most common beginner error. Forgetting it produces an error on the *next* line because the compiler doesn’t notice until it sees an unexpected token.

0.2.4.4 `return 0;`

Returns the exit code 0 (success) to the OS. In `main` specifically, you can omit this and the compiler inserts it, but writing it is explicit and clear.

0.2.5 Compiling: What the Flags Mean

```
$ g++ -std=c++23 -Wall -Wextra -o hello hello.cpp
```

Flag	What it does
<code>g++</code>	The GNU C++ compiler. Also try <code>clang++</code> (often better error messages).
<code>-std=c++23</code>	Use the C++23 standard. Without this you get an older, less capable standard.
<code>-Wall</code>	Enable all common warnings. The W stands for Warning.
<code>-Wextra</code>	Enable even more warnings.
<code>-o hello</code>	Name the output executable <code>hello</code> . Default without <code>-o</code> is <code>a.out</code> .

Flag	What it does
hello.cpp	The source file to compile.

0.2.5.1 Two Build Modes You Will Use

```
# Development (use this while writing code):
$ g++ -std=c++23 -Wall -Wextra -g -fsanitize=address,undefined -o hello hello.cpp
#                                     |  |
#                                     |  +-- catches memory bugs and UB at runtime
#                                     +----- adds debug symbols for stack traces

# Release (use this for final builds):
$ g++ -std=c++23 -O2 -o hello hello.cpp
#                                     |
#                                     +-- enables optimization (2x-10x faster)
```

The sanitizers (`-fsanitize=address,undefined`) catch entire classes of bugs that would otherwise produce silent wrong answers or random crashes. Always use them during development.

0.2.6 What Is in the Compiled Binary?

The executable file is organized into sections:

`./hello` (ELF executable on Linux, Mach-O on macOS, PE on Windows)

Section	Contents
-----	-----
.text	Your compiled machine code instructions
.rodata	Read-only data: string literals like "Hello, world!\n"
.data	Initialized global variables
.bss	Uninitialized global variables (zeroed at program start)

You can disassemble the executable to see what your code became:

```
$ objdump -d -C hello | grep -A 15 "<main>"

0000000000001149 <main>:
   1149: push    rbp
   114a: mov     rbp, rsp
   114d: lea     rdi, [rip+0xeb0]    <- loads address of "Hello, world!\n"
   1154: call    1060 <puts@plt>    <- calls puts() (compiler optimized cout to this)
   1159: mov     eax, 0x0
   115e: pop     rbp
   115f: ret
```

Seven CPU instructions. No interpreter, no type checking, no reference counting. This is what runs when you execute `./hello`.

0.2.7 Multi-File Programs

Once programs grow beyond one file, you split them:

```
main.cpp  --compile-->  main.o  --+
                                     +--> linker --> program
greet.cpp --compile--> greet.o  --+
```

```
// greet.h -- declaration (the interface, what other files need to know)
#pragma once
#include <string>
void greet(const std::string& name);

// greet.cpp -- definition (the actual implementation)
#include <iostream>
#include "greet.h"
void greet(const std::string& name) {
    std::cout << "Hello, " << name << "!\n";
}

// main.cpp -- uses the function
#include "greet.h"
int main() {
    greet("world");
    return 0;
}
```

```
$ g++ -std=c++23 -o program main.cpp greet.cpp
```

Chapter 11 covers the full rationale for this split.

0.2.8 Reading Compiler Errors

The compiler is your most important tool. It catches errors before your program runs. Learning to read its output quickly is a core skill.

0.2.8.1 Missing Semicolon


```
int main() {
    std::cout << "Hello"    // missing semicolon here
    return 0;
}
```

```
hello.cpp:4:5: error: expected ';' before 'return'
   4 |     std::cout << "Hello"
     |                                     ^
     |                                     ;
   5 |     return 0;
```

The caret ^ marks where the problem was detected – the end of line 4. The message says “before ‘return’” because the compiler only noticed something was wrong when it hit `return`.

0.2.8.2 Missing #include

```
int main() {
    std::cout << "Hello\n";
}
```

```
hello.cpp:2:5: error: 'cout' is not a member of 'std'
   2 |     std::cout << "Hello\n";
     |     ^~~~~~
note: 'std::cout' is defined in header '<iostream>';
    did you forget to '#include <iostream>'?
```

Read every note: line. Modern compilers often tell you exactly what to do.

0.2.8.3 Undefined Reference (Linker Error)

```
// main.cpp
void greet(const std::string& name); // declared but never defined anywhere

int main() { greet("world"); }
```

```
/usr/bin/ld: main.o: undefined reference to `greet(std::string const&)'
collect2: error: ld returned 1 exit status
```

“Undefined reference” means the compiler accepted the code (it saw the declaration), but the linker couldn’t find the function’s body. Either you forgot to define it, or forgot to include its `.cpp` file in the build command.

0.2.8.4 Type Mismatch

```
int x = "hello";
```

error: invalid conversion from 'const char*' to 'int'

```
1 | int x = "hello";
  |           ^~~~~~
```

The static type system caught this before the program ran. Python would allow `x = 5; x = "hello"` – C++ refuses.

0.2.9 Common Mistakes in This Chapter

0.2.9.1 Mistake 1: Forgetting the semicolon

The bug: `std::cout << "Hello"` with no `;`

The symptom: Error message appears on the *next* line, not the line where you forgot it. The compiler doesn't notice until it encounters an unexpected token.

The fix: Every statement ends with `;`. After `return 0`. After every `cout`. After every variable declaration.

0.2.9.2 Mistake 2: Writing `cout` without `std::`

The bug:

```
cout << "Hello\n"; // 'cout' -- where is it?
```

The symptom:

error: 'cout' was not declared in this scope

The fix: Write `std::cout`. You can also add `using std::cout;` at the top of a function to avoid typing `std::` everytime. Do not use `using namespace std;` – it pulls in hundreds of names that may conflict with your own.

0.2.9.3 Mistake 3: Confusing `#include <...>` with `#include "..."`

The bug: Writing `#include "iostream"` instead of `#include <iostream>`

What happens: The preprocessor looks for `iostream` in the current directory first. It may find nothing and fail, or find the wrong file.

The fix: Use `<...>` for system and library headers. Use `"..."` for your own headers.

0.2.10 Exercises

Exercise 1.1 – Predict the output

What does this print? Work it out before running it.

```
#include <iostream>
int main() {
    std::cout << "Line 1\n";
    std::cout << "Line " << 2 << "\n";
    std::cout << "Line " << 1 + 2 << "\n";
    return 0;
}
```

Answer:

Line 1
Line 2
Line 3

1 + 2 is evaluated to 3 before being inserted into the stream.

Exercise 1.2 – Find all three bugs

This program has exactly three syntax errors. Find them all.

```
#include <iostream>
int main() {
    std::cout << "Hello, world!"
    std::cout << "How are you?"
    return 0
}
```

Answer: 1. Missing ; after "Hello, world!" 2. Missing ; after "How are you?" 3. Missing ; after return 0

Exercise 1.3 – Write from scratch

Write a program that prints this output exactly (copy the spacing):

```
=== My C++ Program ===
Author: Your Name
Version: 1.0
```

Answer:

```
#include <iostream>
int main() {
    std::cout << "=== My C++ Program ===\n";
    std::cout << "Author: Your Name\n";
    std::cout << "Version: 1.0\n";
    return 0;
}
```

Exercise 1.4 – Explain the difference

In your own words: what is the difference between compiling with `-O0` and `-O2`? Why would you use each?

Answer: `-O0` (no optimization) translates your code nearly literally into machine code. The machine code closely mirrors your source, making it easier to debug – variables are in the places you’d expect, and the program steps through code the way you wrote it. Use this during development.

`-O2` enables aggressive optimization: the compiler may inline functions, eliminate redundant computations, reorder instructions, and more. The program runs significantly faster. The machine code may look very different from your source. Use this for release builds.

Chapter 2: Variables, Types, and the Static Type System

0.2.11 What IS a Variable?

In Python, a variable is a **name tag** attached to an object. The object lives on the heap; the name is just a reference to it.

```
x = 42          # 'x' is a label pointing to an int object on the heap
y = x          # 'y' is another label pointing to the SAME object
x = "hello"    # 'x' now points to a different object; y is unchanged
print(y)       # 42
```

The object carries its own type:

Python memory:

Variable	Points to
x	[type: int refcount: 2 value: 42]
y	[same object]

After `x = "hello"`:

x	[type: str refcount: 1 value: "hello"]
y	[type: int refcount: 1 value: 42]

Every Python variable is a pointer. The object it points to carries its type, a reference count, and metadata.

In C++, a variable is a **named region of memory with a fixed type**. The variable IS the storage. There is no separate heap object (unless you explicitly create one), no reference count, no type tag at runtime.

```
int x = 42; // 'x' is 4 bytes of memory containing the bit pattern for 42
           // The type 'int' exists only in the compiler's mind
```

C++ memory:

Address	Variable	Bytes stored
0x7fff10	x (int)	[00][00][00][2A] <- that is 42 in hex

No type tag. No reference count. No metadata. Four bytes. That is it.

The type `int` tells the **compiler** (not the CPU, not the runtime): - Reserve 4 bytes - Treat those bytes as a signed two's complement integer - Allow operations like `+`, `-`, `*`, `/` - Reject operations like string concatenation

At runtime, there are only bytes. Types are a compile-time concept.

This is the most fundamental difference between Python and C++. Almost every other difference you will encounter flows from this one.

0.2.12 How Integers Are Stored: Binary and Two's Complement

To understand C++ types, you need to understand how integers are stored in memory. If you already know two's complement, skim this section.

A computer stores everything as bits: 0 or 1. Eight bits make one byte.

The number 42 in binary:

$$42 = 32 + 8 + 2 = 2^5 + 2^3 + 2^1$$

Bit positions (0 = rightmost):

7	6	5	4	3	2	1	0
0	0	1	0	1	0	1	0
		32		8		2	
	0						

$$0 \times 128 + 0 \times 64 + 1 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 = 42$$

An `int` uses 4 bytes = 32 bits. So 42 stored as a 32-bit int:

Byte 3	Byte 2	Byte 1	Byte 0
00000000	00000000	00000000	00101010
= 42			

No type metadata anywhere. The CPU interprets these bytes according to which instruction operates on them.

0.2.12.1 How Negative Numbers Work: Two's Complement

For signed integers, the highest bit is the sign bit: 0 = positive, 1 = negative.

For a 32-bit signed int: - Maximum positive: 0111 1111 ... 1111 = 2,147,483,647 - Maximum negative: 1000 0000 ... 0000 = -2,147,483,648 - -1: 1111 1111 ... 1111 (all bits set)

To negate a number: flip all bits, then add 1.

42 in binary:	0000 0000 0000 0000 0000 0000 0010 1010
Flip all bits:	1111 1111 1111 1111 1111 1111 1101 0101
Add 1:	1111 1111 1111 1111 1111 1111 1101 0110 = -42

The critical implication: **signed integer overflow is undefined behavior in C++.**

```
int max = 2147483647; // the largest possible int
int bad = max + 1;    // UNDEFINED BEHAVIOR
                     // The standard does not define what happens.
                     // In practice: wraps to -2147483648 on most systems.
                     // But the compiler is allowed to assume overflow
                     // never happens and optimize in ways that break you.
```

Python integers are arbitrary precision – they never overflow. C++ `int` has fixed size and hard limits. Always think about whether your values can exceed the range.

0.2.13 The Fundamental Types

0.2.13.1 bool

```
bool yes = true;
bool no  = false;

std::cout << yes << "\n"; // prints 1
std::cout << no  << "\n"; // prints 0
```

Stores true/false. Takes 1 byte (even though 1 bit would suffice – byte is the smallest addressable unit on most hardware). `true` is stored as 1, `false` as 0.

Any non-zero integer converts to `true`, zero converts to `false`:

```
bool b1 = 42; // true  (42 != 0)
bool b2 = 0;  // false
bool b3 = -1; // true  (-1 != 0)
```

0.2.13.2 char

```
char letter = 'A'; // single character, single quotes
char newline = '\n'; // escape sequences work like Python
char tab = '\t';
char null_c = '\0'; // null character (value 0)
```

One byte. Holds one ASCII character. The character is stored as its ASCII code: `'A'` = 65, `'Z'` = 90, `'0'` = 48, `'\n'` = 10.

You can do arithmetic on chars because they are just numbers:

```
char c = 'A';
std::cout << c + 1; // 66 (int arithmetic)
std::cout << static_cast<char>(c + 1); // 'B' (cast back to char)
std::cout << (char)('a' + 2); // 'c' (C-style cast, avoid)
```

0.2.13.3 Integer Types

Type	Typical size	Signed range	Notes
<code>short</code>	2 bytes	-32,768 to 32,767	Rarely used
<code>int</code>	4 bytes	-2.1B to 2.1B	Default integer type
<code>long</code>	4 or 8 bytes	platform-dependent	Avoid – use long long
<code>long long</code>	8 bytes	-9.2e18 to 9.2e18	Use when int overflows

To make a type unsigned (no negatives, larger maximum):

```
unsigned int u = 4000000000u; // 'u' suffix = unsigned literal
unsigned long long big = 18446744073709551615ull; // 'ull' suffix
```

An unsigned type with N bits holds 0 to $2^N - 1$. A signed type with N bits holds $-2^{(N-1)}$ to $2^{(N-1)} - 1$.

0.2.13.4 float and double

```
float f = 3.14f; // 32-bit floating point, 'f' suffix is required
double d = 3.14; // 64-bit floating point -- use this by default
```

double is what Python calls float. Use double unless you have a specific reason (memory, GPU/SIMD code).

Why prefer double? Precision: - float: approximately 7 significant decimal digits - double: approximately 15 significant decimal digits

```
float f = 1.23456789f;
double d = 1.23456789;
std::cout << std::setprecision(10) << f << "\n"; // 1.234567881 (wrong at 7th digit)
std::cout << std::setprecision(10) << d << "\n"; // 1.23456789 (correct)
```

Floating point is not exact – same in Python and C++:

```
# Python
print(0.1 + 0.2) # 0.30000000000000004
```

```
// C++
#include <iomanip>
double x = 0.1 + 0.2;
std::cout << x << "\n"; // 0.3 (default rounds nicely)
std::cout << std::setprecision(17) << x << "\n"; // 0.30000000000000004
std::cout << std::setprecision(17) << 0.3 << "\n"; // 0.29999999999999999
```

0.1, 0.2, and 0.3 cannot be represented exactly in binary floating point – the same way $1/3$ cannot be represented exactly in decimal. This is not a C++ bug; it is IEEE 754 floating point. The implication: **never compare floating-point numbers with ==.**

0.2.13.5 std::string

```
#include <string>
std::string name = "Alice";
std::string greeting = "Hello, " + name + "!"; // concatenation with +
std::cout << greeting.size() << "\n"; // 13
```


Not a primitive type. `std::string` is a class from the standard library that manages heap-allocated character data. We cover it in depth in Chapter 10.

0.2.14 Fixed-Width Integer Types

The sizes of `short`, `int`, `long` depend on the platform and compiler. For code that must work correctly regardless of platform – networking, file formats, hardware interfaces – use exact-size types from `<cstdint>`:

```
#include <cstdint>

int8_t  a = -128;           // exactly 8 bits, signed (-128 to 127)
uint8_t b = 255;           // exactly 8 bits, unsigned (0 to 255)
int16_t c = 32767;          // exactly 16 bits, signed
uint16_t d = 65535;         // exactly 16 bits, unsigned
int32_t e = -1;            // exactly 32 bits, signed
uint32_t f = 4294967295u;   // exactly 32 bits, unsigned
int64_t g = -1LL;          // exactly 64 bits, signed
uint64_t h = 0xFFFFFFFFFFFFFFFFull; // exactly 64 bits, unsigned, all bits set
```

Use these whenever the exact bit width matters. Use `int` when it does not.

0.2.15 Declaring and Initializing Variables

0.2.15.1 Never Leave a Variable Uninitialized

This is the most important rule in this chapter:

```
int sum;                                // BAD: sum contains whatever bytes were
for (int i = 0; i < 10; ++i)           // in that memory location before.
    sum += i;                          // Adding to garbage = garbage.
std::cout << sum << "\n";              // Prints something unpredictable.

int sum{0};                             // GOOD: explicitly initialized to zero.
for (int i = 0; i < 10; ++i)
    sum += i;
std::cout << sum << "\n";              // 45, always.
```

On some runs, the garbage value might happen to be 0, making the bug invisible. On other runs, it will be -858993460 or similar. Undefined behavior is unpredictable.

0.2.15.2 The Four Initialization Forms

C++ has four ways to initialize a variable. They have subtle differences.

Form 1: Default initialization (leaves value unset – dangerous)

```
int x;           // x has garbage value
double d;        // d has garbage value
```

Do not do this for primitive types.

Form 2: Copy initialization (C-style, from C)

```
int x = 5;
double d = 3.14;
std::string s = "hello";
```

Looks like assignment but is initialization (happens at construction, not after). Fine for simple cases.

Form 3: Direct initialization

```
int x(5);
double d(3.14);
std::string s("hello");
```

More verbose for primitives but is the natural form for class objects with constructors.

Form 4: Brace (uniform) initialization – prefer this

```
int x{5};
double d{3.14};
std::string s{"hello"};
```

Why prefer brace initialization? It prevents **narrowing conversions** – type truncations that silently discard data:

```
// With = initialization, narrowing is SILENT:
int a = 3.7;           // truncates to 3. No error. Potential bug hiding.
int b = 300000;         // fits in int, fine
int c = 300000LL;       // long long to int, may truncate on 16-bit systems, no warning

// With {} initialization, narrowing is a COMPILE ERROR:
int a{3.7};            // error: narrowing conversion of '3.7e+0' from 'double' to 'int'
int b{300000};          // fine
int c{300000LL};        // fine (value fits in int)
```

The compiler tells you about the truncation at compile time instead of letting it silently corrupt your data at runtime.

You can value-initialize (set to zero/false/null) with empty braces:

```
int    n{};    // 0
double d{};    // 0.0
bool   b{};    // false
char   c{};    // '\0'
```

0.2.16 Memory Layout of Variables

```
bool   a{true};    // 1 byte
char   b{'X'};     // 1 byte
int    c{42};      // 4 bytes
double d{3.14};    // 8 bytes
```

Stack memory (addresses are illustrative):

Addr	Var	Size	Raw bytes (hex)	
0x1000	a (bool)	1 byte	[01]	
0x1001	b (char)	1 byte	[58]	'X' = ASCII 88 = 0x58
0x1004	c (int)	4 bytes	[00][00][00][2A]	42 = 0x2A
0x1008	d (double)	8 bytes	[40][09][1E][B8][51][EB][85][1F]	

(The actual layout may differ due to alignment, but the sizes are as shown.)

There is no type information stored at runtime. The CPU knows to treat `c` as a signed integer because the compiled instructions operating on it are integer instructions. The type system is entirely a compile-time construct.

0.2.17 auto – Type Deduction

When the type is obvious from the right-hand side, `auto` lets the compiler deduce it:

```
auto x = 42;           // int    (integer literals default to int)
auto y = 42L;          // long   (L suffix)
auto z = 42LL;         // long long (LL suffix)
auto d = 3.14;         // double (decimal literals default to double)
auto f = 3.14f;        // float  (f suffix)
auto b = true;         // bool
auto s = std::string{"hi"}; // std::string
```

`auto` does NOT make C++ dynamically typed. The type is deduced once at compile time and then fixed permanently:

```

auto x = 42;           // x is int, fixed forever
x = 3.14;              // fine: double 3.14 truncates to int 3 (with = init)
x = "hello";           // COMPILE ERROR: cannot assign const char* to int

```

`auto` strips top-level `const` and references from the deduced type. Add them explicitly if you need them:

```

int n = 5;
auto a = n;           // int (copy)
auto& b = n;          // int& (reference to n)
const auto& c = n;    // const int& (const reference, no copy, no modify)

```

Where `auto` really shines is avoiding verbose type names:

```

// Without auto:
std::vector<std::pair<std::string, int>>::iterator it = scores.begin();

// With auto (same type, more readable):
auto it = scores.begin();

```

0.2.18 `const` and `constexpr`

0.2.18.1 `const` – Constant After Initialization

```

const int MAX = 8;
MAX = 10;           // COMPILE ERROR: assignment of read-only variable

const int n = get_input(); // value known only at runtime -- still valid const

```

Use `const` for values that are initialized once and never change. The compiler enforces this – it's not just a convention.

0.2.18.2 `constexpr` – Must Be Computed at Compile Time

```

constexpr int BOARD = 8;
constexpr double PI = 3.14159265358979;

int grid[BOARD][BOARD]; // OK -- array size must be a compile-time constant

constexpr int square(int x) { return x * x; }
constexpr int area = square(BOARD); // 64, computed at compile time, zero runtime cost

```

`constexpr` is stronger than `const`. The value must be computable without running the program. If it isn't, you get a compile error – which is what you want.

Rule: use `constexpr` for mathematical constants, array sizes, and configuration values. Use `const` for values set at runtime that shouldn't change afterward.

0.2.19 Type Conversion

0.2.19.1 Safe Widening (Automatic)

```
int    i = 42;
double d = i;    // int to double: no data loss, happens automatically
long long l = i; // int to long long: always safe
```

0.2.19.2 Narrowing (Dangerous Without Braces)

```
double d = 3.99;
int    i = d;    // silently truncates to 3 (with = initialization)
int    j{d};     // COMPILE ERROR: narrowing (with {} initialization)
```

0.2.19.3 Explicit Conversion with `static_cast`

When you intend to convert, be explicit about it:

```
double d = 3.99;
int i = static_cast<int>(d);    // 3, explicit truncation, intent is documented

int a = 5, b = 2;
// Bug: division happens as int first, then converts to double
double wrong = a / b;          // 2.0

// Fix: cast one operand to double before dividing
double correct = static_cast<double>(a) / b; // 2.5
```

`static_cast<NewType>(expr)` is the C++ way to convert. It is checked at compile time. Do not use C-style casts like `(int)x` – they bypass some safety checks and are harder to search for in code.

0.2.20 Checking Sizes with `sizeof`

```
#include <iostream>
int main() {
    std::cout << "bool:      " << sizeof(bool) << " byte(s)\n"; // 1
    std::cout << "char:      " << sizeof(char) << " byte(s)\n"; // 1
    std::cout << "int:       " << sizeof(int) << " byte(s)\n"; // 4
    std::cout << "long long: " << sizeof(long long) << " byte(s)\n"; // 8
    std::cout << "float:     " << sizeof(float) << " byte(s)\n"; // 4
    std::cout << "double:    " << sizeof(double) << " byte(s)\n"; // 8
    return 0;
}
```

`sizeof` is evaluated at compile time – zero runtime overhead. It works on both types (`sizeof(int)`) and variables (`sizeof(x)`).

0.2.21 Common Mistakes in This Chapter

0.2.21.1 Mistake 1: Uninitialized Variable

The bug:

```
int total;
for (int i = 1; i <= 5; ++i)
    total += i; // total starts with garbage, not 0
std::cout << total;
```

The symptom: Wrong or unpredictable output. Sometimes appears correct (when garbage happens to be 0).

The fix: `int total{0};`

How to detect: Compile with `-Wall` (warns about “total may be used uninitialized”). Run with `-fsanitize=undefined` for runtime detection.

0.2.21.2 Mistake 2: Signed Integer Overflow

The bug:

```
int seconds_per_year = 365 * 24 * 60 * 60; // 31,536,000 -- fits in int
int seconds_per_century = seconds_per_year * 100; // 3,153,600,000 -- OVERFLOW!
// int max is ~2.1 billion; 3.15 billion overflows
```

The symptom: Wrong answer (often a negative number). No compile error.

The fix: `long long seconds_per_century = (long long)seconds_per_year * 100;`

How to detect: `-fsanitize=undefined` reports “signed integer overflow.”

0.2.21.3 Mistake 3: Integer Division When Expecting Decimal

The bug:

```
double average = (100 + 75 + 90) / 3;    // 88 -- not 88.333...
```

All literals are `int`. The division is integer division before the result is stored in `double`.

The fix: `double average = (100 + 75 + 90) / 3.0;` or use `static_cast<double>`.

0.2.21.4 Mistake 4: Comparing Floats with ==

The bug:

```
double x = 0.1 + 0.2;
if (x == 0.3) {
    std::cout << "equal\n";    // never prints
}
```

The fix:

```
#include <cmath>
if (std::abs(x - 0.3) < 1e-9) {
    std::cout << "equal\n";    // works
}
```

0.2.22 Exercises

Exercise 2.1 – Type sizes

Without looking anything up, predict the output of this program on a 64-bit Linux system, then verify:

```
#include <iostream>
int main() {
    std::cout << sizeof(bool) << "\n";
    std::cout << sizeof(char) << "\n";
    std::cout << sizeof(short) << "\n";
    std::cout << sizeof(int) << "\n";
    std::cout << sizeof(long) << "\n";
    std::cout << sizeof(long long) << "\n";
    std::cout << sizeof(float) << "\n";
    std::cout << sizeof(double) << "\n";
}
```

Answer: 1 1 2 4 8 8 4 8 on 64-bit Linux. Note that `long` is 8 bytes on 64-bit Linux but 4 bytes on 64-bit Windows – another reason to use `long long` when you need 64 bits.

Exercise 2.2 – Brace initialization rejection

Which of these lines would brace initialization reject with a compile error?

```
int a{3.0}; // (a)
int b{3}; // (b)
int c{3LL}; // (c) LL = long long literal
double d{3}; // (d)
float e{3.14}; // (e)
```

Answer: - (a): double -> int narrows (loses the decimal). **ERROR.** - (b): int -> int, exact. Fine. - (c): long long -> int may narrow if value doesn't fit. **ERROR** (even though 3 fits, the types differ). - (d): int -> double widens (no data loss). Fine. - (e): double -> float narrows (loses precision). **ERROR.**

Exercise 2.3 – Fix the division

Fix this code so it prints 2.5 instead of 2:

```
#include <iostream>
int main() {
    int a = 5, b = 2;
    std::cout << a / b << "\n";
}
```

Answer:

```
std::cout << static_cast<double>(a) / b << "\n";
// or:
std::cout << a / static_cast<double>(b) << "\n";
// or:
std::cout << a / 2.0 << "\n";
```


Exercise 2.4 – Max value calculation

What is the maximum value of `uint32_t`? Show the calculation. Do the same for `int32_t`.

Answer: - `uint32_t`: 32 bits, unsigned. Range = 0 to $2^{32} - 1 = \mathbf{4,294,967,295}$. - `int32_t`: 32 bits, signed. Range = -2^{31} to $2^{31} - 1 = \mathbf{-2,147,483,648}$ to $\mathbf{2,147,483,647}$. One bit is used for the sign.

Chapter 3: Operators and Expressions

0.2.23 Arithmetic Operators

Most arithmetic operators work the same as Python. Division is the exception that trips everyone up.

```
int a = 17, b = 5;

std::cout << a + b << "\n"; // 22 -- addition
std::cout << a - b << "\n"; // 12 -- subtraction
std::cout << a * b << "\n"; // 85 -- multiplication
std::cout << a / b << "\n"; // 3 -- WATCH OUT: integer division
std::cout << a % b << "\n"; // 2 -- remainder (modulo)
```

The division rule: **when both operands are integers, / discards the remainder.**

```
# Python: / always returns a float
print(17 / 5) # 3.4
print(17 // 5) # 3 (floor division, rounds toward negative infinity)
print(-17 // 5) # -4 (floor: -3.4 rounds down to -4)
```

```
// C++: / on integers truncates toward zero (not floor)
std::cout << 17 / 5 << "\n"; // 3 (truncated toward zero)
std::cout << -17 / 5 << "\n"; // -3 (truncated toward zero, NOT -4)
std::cout << 17.0 / 5 << "\n"; // 3.4 (float division: 17.0 is a double)
```

The key distinction: C++ truncates toward zero. Python's `//` floors toward negative infinity. For positive numbers they agree. For negative numbers they differ.

0.2.23.1 The Classic Division Trap

```
int numerator = 7;
int denominator = 2;

double result = numerator / denominator; // 3.0, NOT 3.5
```

What happened? The division `numerator / denominator` is evaluated first – both are `int`, so integer division gives 3. Then 3 is converted to 3.0 for the `double` assignment.

The fix: make at least one operand a `double` before the division:

```
double result = static_cast<double>(numerator) / denominator; // 3.5
// or:
double result = numerator / static_cast<double>(denominator); // 3.5
// or (if one is a literal):
double result = numerator / 2.0; // 3.5
```

0.2.23.2 Modulo With Negative Numbers

```
std::cout << 7 % 3 << "\n"; // 1 (7 = 2*3 + 1)
std::cout << -7 % 3 << "\n"; // -1 (sign follows dividend in C++)
std::cout << 7 % -3 << "\n"; // 1 (sign follows dividend)
```

Python's `%` always returns non-negative when the divisor is positive (e.g., `-7 % 3 = 2` in Python). C++'s `%` sign follows the dividend.

This matters for “wrap around” patterns. To get Python-like non-negative modulo:

```
int python_mod(int a, int b) {
    return ((a % b) + b) % b;
}
// python_mod(-7, 3) = ((-1) + 3) % 3 = 2 % 3 = 2 -- same as Python
```

0.2.24 Increment and Decrement Operators

C++ has `++` (add 1) and `--` (subtract 1) as standalone operators. Python does not.

```
int x = 5;
++x; // pre-increment: x becomes 6
x++; // post-increment: x becomes 7
--x; // pre-decrement: x becomes 6
x--; // post-decrement: x becomes 5
```

Used as standalone statements, `++x` and `x++` are identical in effect. The difference appears when used inside a larger expression:

```
int x = 5;
int a = ++x; // pre: x becomes 6 FIRST, then a = 6 (new value)
int b = x++; // post: b = 6 (old value), then x becomes 7
```

Pre-increment (++x):

Step 1: increment x (x = 7)

Step 2: expression evaluates to new x (7)

Post-increment (x++):

Step 1: save current x (6)

Step 2: increment x (x = 7)

Step 3: expression evaluates to saved old x (6)

Always prefer ++x (pre-increment) as a habit. It is never slower than x++, and for iterator objects (Chapter 28) it can be significantly faster because post-increment must save a copy of the old value.

0.2.25 Compound Assignment

These modify a variable in place:

```
int x = 20;
x += 5;    // x = 25    (same as x = x + 5)
x -= 3;    // x = 22
x *= 2;    // x = 44
x /= 4;    // x = 11    (integer division if x is int)
x %= 3;    // x = 2      (remainder)
x <<= 2;   // x = 8      (left shift by 2: same as x * 4)
x >>= 1;   // x = 4      (right shift by 1: same as x / 2 for non-negative)
```

0.2.26 Comparison Operators

All return bool:

```
int a = 5, b = 10;
bool r1 = (a == b);    // false -- equal
bool r2 = (a != b);    // true  -- not equal
bool r3 = (a < b);     // true  -- less than
bool r4 = (a > b);     // false -- greater than
bool r5 = (a <= b);    // true  -- less than or equal
bool r6 = (a >= b);    // false -- greater than or equal
```

0.2.26.1 The Floating-Point Comparison Problem

```
double x = 0.1 + 0.2;
double y = 0.3;
std::cout << (x == y) << "\n";    // 0 (false!)
```

0.1 cannot be represented exactly in binary. Neither can 0.2 or 0.3. The sum 0.1 + 0.2 comes out as 0.30000000000000004... Stored 0.3 is 0.29999999999999998... They are not equal, even though mathematically they should be.

This is not a C++ problem – the same thing happens in Python and every other language that uses IEEE 754 floating point (which is all of them).

Rule: Never compare floating-point numbers with ==. Use a tolerance.

```
#include <cmath>    // for std::abs

// Returns true if a and b are within tolerance of each other
bool nearly_equal(double a, double b, double tolerance = 1e-9) {
    return std::abs(a - b) < tolerance;
}

if (nearly_equal(0.1 + 0.2, 0.3)) {
    std::cout << "equal (within tolerance)\n";    // this prints
}
```

Choose tolerance based on the scale of your values: - For values around 1.0: 1e-9 is reasonable - For values around 1,000,000: 1e-3 may be needed - For values that went through many operations: use larger tolerances

0.2.27 Logical Operators

```
bool x = true, y = false;

bool and_result = x && y;    // false (both must be true) -- Python: x and y
bool or_result  = x || y;    // true  (either must be true) -- Python: x or y
bool not_result = !x;        // false (flip)                -- Python: not x
```

0.2.27.1 Short-Circuit Evaluation

&& stops evaluating as soon as it finds a false. || stops as soon as it finds a true. This is identical to Python and is critically important for safety:

```

int* p = nullptr;

// Safe: if p is null, the right side is NEVER evaluated
if (p != nullptr && *p > 0) {
    std::cout << "positive pointee\n";
}
// Without short-circuit: *p when p is null = crash

// Common optimization: cheap check first, expensive check second
if (is_in_cache(key) || compute_and_cache(key)) {
    // compute_and_cache only called when not in cache
}

```

Short-circuit evaluation is relied upon for correctness, not just optimization.

0.2.28 Bitwise Operators

These operate on individual bits within integers. They appear constantly in systems programming, graphics, embedded code, and game engines.

Op	Name	Meaning
&	AND	Output bit is 1 only if BOTH input bits are 1
\	OR	Output bit is 1 if EITHER input bit is 1
^	XOR	Output bit is 1 if the two input bits are DIFFERENT
~	NOT	Flips all bits
<<	Left shift	Shifts bits left, filling with 0 on the right
>>	Right shift	Shifts bits right (fills with 0 for unsigned, sign bit for signed)

Example: a = 5 = 0101 in binary
 b = 3 = 0011 in binary

```

a & b:  0101
        0011
        ----
        0001   = 1   (only bit 0 is set in both)

```

```

a | b:  0101
        0011
        ----
        0111   = 7   (any bit set in either)

```

```

a ^ b:  0101
        0011
        ----
        0110  = 6  (bits that differ)

```

```
~a:      ~0101 = 1111...11111010 = -6 (for 32-bit signed int)
```

Left shift by 1 = multiply by 2. Left shift by n = multiply by 2^n:

```

1 << 0 = 1
1 << 1 = 2
1 << 2 = 4
1 << 3 = 8
1 << 4 = 16
1 << 10 = 1024

```

0.2.28.1 Practical Use: Boolean Flags Packed in an Integer

Instead of a separate bool for each property, pack them into one integer – each bit represents one flag:

```

const uint32_t VISIBLE    = 1u << 0;  // bit 0: 00000001
const uint32_t SOLID     = 1u << 1;  // bit 1: 00000010
const uint32_t ACTIVE    = 1u << 2;  // bit 2: 00000100
const uint32_t HAS_HEALTH = 1u << 3;  // bit 3: 00001000

uint32_t entity_flags = 0;

// Set a flag (turn bit ON):
entity_flags |= VISIBLE;    // flags = 00000001
entity_flags |= ACTIVE;    // flags = 00000101

// Test a flag (check if bit is set):
if (entity_flags & VISIBLE) { // nonzero = true
    render(entity);
}
if (entity_flags & ACTIVE) {
    update(entity);
}

// Clear a flag (turn bit OFF):
entity_flags &= ~ACTIVE;    // ~ACTIVE = 11111011; AND clears bit 2
                           // flags = 00000001

// Toggle a flag (flip):
entity_flags ^= SOLID;     // XOR flips bit 1

```

This pattern is everywhere: OpenGL uses `GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT`, Vulkan uses `VkImageUsageFlags`, Linux uses `O_RDONLY | O_CREAT` for file open flags.

0.2.29 Operator Precedence

When operators are mixed, precedence determines the order of evaluation:

Highest priority (evaluated first)

() [] -> .	grouping, subscript, member access
! ~ ++x --x (unary)	unary operators (right-to-left)
* / %	multiplicative
+ -	additive
<< >>	bitwise shift
< <= > >=	relational comparison
== !=	equality comparison
&	bitwise AND
^	bitwise XOR
	bitwise OR
&&	logical AND
	logical OR
?:	ternary (right-to-left)
= += -= ...	assignment (right-to-left)

Lowest priority (evaluated last)

Common traps:

```
// Trap 1: bitwise AND vs equality
if (flags & MASK == 1) { }    // parsed as: flags & (MASK == 1)  -- WRONG
if ((flags & MASK) == 1) { }  // correct

// Trap 2: shift and arithmetic
int x = 2 + 3 << 1;    // parsed as: 2 + (3 << 1) = 2 + 6 = 8
int y = (2 + 3) << 1;  // = 5 << 1 = 10

// Rule: when in doubt, parenthesize
```

0.2.30 The Ternary Operator

```
# Python
label = "even" if n % 2 == 0 else "odd"
```

```
// C++
std::string label = (n % 2 == 0) ? "even" : "odd";
//                condition   true    false
```

The ternary is an *expression* that produces a value. Good for simple one-liners. Do not nest them – `a ? b ? c : d : e` is unreadable.

0.2.31 Common Mistakes in This Chapter

0.2.31.1 Mistake 1: Assignment Instead of Comparison

The bug:

```
if (x = 5) { ... } // assigns 5 to x, condition is always true (5 != 0)
```

The symptom: The `if` branch always runs; the `else` never runs; `x` silently changed. **The fix:** `if (x == 5)`. Compile with `-Wall` to get a warning about this. **Defensive coding:** Some people write `if (5 == x)` (“Yoda conditions”). Then a typo `if (5 = x)` is a compile error (can’t assign to a literal).

0.2.31.2 Mistake 2: Integer Division Surprise

The bug:

```
double bmi = weight / (height * height); // all ints -- integer division!
```

The fix: Cast one operand: `static_cast<double>(weight) / (height * height)`

0.2.31.3 Mistake 3: == on Floats

The bug: `if (result == expected_value)` where both are doubles **The fix:** `if (std::abs(result - expected_value) < 1e-9)`

0.2.31.4 Mistake 4: Bitwise vs Logical Operators

The bug:

```
bool a = is_ready();
bool b = can_proceed();
if (a & b) { ... } // bitwise AND -- works only if a and b are 0 or 1
// breaks for general integers
if (a && b) { ... } // logical AND -- correct for boolean conditions
```

The fix: Use `&&` and `||` for boolean logic. Use `&` and `|` for bit manipulation.

0.2.32 Exercises

Exercise 3.1 – Predict the output

```
int a = 17, b = 5;
std::cout << a / b << "\n";
std::cout << a % b << "\n";
std::cout << -a / b << "\n";
std::cout << -a % b << "\n";
```

Answer: 3, 2, -3, -2. C++ truncates toward zero; the sign of % follows the dividend.

Exercise 3.2 – Bit flag operations

Starting from `uint8_t flags = 0b00000000`: 1. Set bit 2 (value 4) 2. Set bit 5 (value 32) 3. Check if bit 2 is set (should be true) 4. Clear bit 2

Write the four operations and show the final value of `flags`.

Answer:

```
uint8_t flags = 0b00000000;

flags |= (1u << 2);           // 1. set bit 2: flags = 0b00000100
flags |= (1u << 5);           // 2. set bit 5: flags = 0b00100100

bool bit2_set = (flags & (1u << 2)) != 0; // 3. check bit 2: true

flags &= ~(1u << 2);          // 4. clear bit 2: flags = 0b00100000
// ~(1u << 2) = ~0b00000100 = 0b11111011
// 0b00100100 & 0b11111011 = 0b00100000 = 32
```

Exercise 3.3 – Floating-point tolerance

Write a function `are_equal(double a, double b)` that returns true when `a` and `b` are within 0.0001 of each other. Test it with `0.1 + 0.2` vs `0.3`.

Answer:

```
#include <cmath>
bool are_equal(double a, double b) {
    return std::abs(a - b) < 0.0001;
}
// are_equal(0.1 + 0.2, 0.3) returns true
// are_equal(0.1 + 0.2, 0.4) returns false
```

0.2.33 if/else if/else

```
# Python
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"
```

```
// C++
int score = 85;
std::string grade;

if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else {
    grade = "C";
}
```

Three syntax differences: 1. The condition must be in **parentheses**: `if (score >= 90)` not `if score >= 90` 2. The body uses **curly braces** `{}` instead of `:` and indentation 3. **else if** (two words, with a space) not `elif`

0.2.33.1 Always Use Curly Braces – A Real Cautionary Tale

Braces are technically optional when the body is a single statement:

```
if (error)
    std::cout << "Error occurred\n";    // technically fine
```

But do not do this. The Apple “goto fail” bug (2014) used this pattern and allowed attackers to bypass SSL certificate verification on millions of Apple devices. The vulnerable code looked like:

```
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
goto fail;    // This ALWAYS executes -- attacker bypasses verification
```

The second `goto fail` is not inside the `if`. The indentation lies. Always use braces.

0.2.33.2 if with Initializer (C++17)

Declare a variable scoped only to the `if/else` block:

```
// C++ before 2017:
int value = compute();
if (value > 0) {
    use(value);
}
// 'value' still exists here (wasted scope, potential confusion)

// C++17 and later:
if (int value = compute(); value > 0) {
    // ^-- initializer      ^-- condition
    use(value);
} else {
    handle_error(value);
}
// 'value' does not exist here -- scope is clean
```

Use this when the variable is only meaningful within the `if/else`.

0.2.34 switch

`switch` tests one integer or enum value against a list of constants:

```
int day = 3;

switch (day) {
    case 1:
        std::cout << "Monday\n";
        break;
    case 2:
        std::cout << "Tuesday\n";
        break;
    case 3:
        std::cout << "Wednesday\n";
        break;
    case 6:
    case 7:
        std::cout << "Weekend\n";    // cases 6 and 7 share the same body
        break;
    default:
        std::cout << "Weekday\n";
        break;
}
```

0.2.34.1 Fallthrough: The break Requirement

Without `break`, execution falls through into the next case:

```
int x = 1;
switch (x) {
    case 1:
        std::cout << "one\n";
        // no break!
    case 2:
        std::cout << "two\n";
        break;
    case 3:
        std::cout << "three\n";
        break;
}
```

Output when `x == 1`: one then two. The control falls through from case 1 into case 2.

This is almost always a bug. Always add `break`. If fallthrough is intentional, use `[[fallthrough]]` (C++17) to document it and silence the warning:

```
case 'A':
    [[fallthrough]]; // intentional: A and a are treated the same
case 'a':
    process_letter_a();
    break;
```

`switch` only works on integral types (`int`, `char`, `enum`). It does not work on strings or floating-point numbers.

0.2.35 while Loop

Checks the condition before each iteration. If the condition is false initially, the body never runs.

```
# Python
x = 0
while x < 5:
    print(x)
    x += 1
```

```
// C++
int x = 0;
while (x < 5) {
    std::cout << x << "\n";
    ++x;
}
// prints 0, 1, 2, 3, 4
```

0.2.36 do-while Loop

The body runs at least once. The condition is checked after the body.

```
int x = 100;
do {
    std::cout << x << "\n"; // prints 100 even though 100 >= 5
    ++x;
} while (x < 5);
// body ran once; condition was false; loop ends
```

When to use it: Input validation – you always want to ask once, then check.

```
#include <iostream>

int main() {
    int age;
    do {
        std::cout << "Enter your age (1-120): ";
        std::cin >> age;
    } while (age < 1 || age > 120);
    // Guaranteed: asked at least once; keeps asking until valid input

    std::cout << "Age accepted: " << age << "\n";
    return 0;
}
```

Python has no do-while. The equivalent Python idiom:

```
while True:
    age = int(input("Enter your age (1-120): "))
    if 1 <= age <= 120:
        break
```

0.2.37 for Loop: C-Style

```
for (initializer; condition; increment) {
    body;
}
```

```
for (int i = 0; i < 10; ++i) {
    std::cout << i << " "; // 0 1 2 3 4 5 6 7 8 9
}
// 'i' does not exist after the loop -- scoped to the for
```

Execution order:

```

for (int i = 0; i < 10; ++i)

    int i = 0      <-- runs ONCE before anything
        |
        v
    [check: i < 10?]  ----NO----> exit loop
        |
        YES
        |
        v
    [body: cout << i]
        |
        v
    [++i]
        |
        +-----> [check: i < 10?] (repeat)

```

Key point: the variable declared in the initializer (`int i`) exists only inside the `for` loop. This is better than Python, where loop variables leak out:

```

# Python: loop variable persists after loop
for i in range(5):
    pass
print(i)  # 4 -- still accessible

```

```

// C++: loop variable is gone after loop
for (int i = 0; i < 5; ++i) { }
std::cout << i;  // COMPILE ERROR: 'i' was not declared in this scope

```

Useful patterns:

```

// Count down:
for (int i = 10; i >= 0; --i) {
    std::cout << i << " ";  // 10 9 8 7 6 5 4 3 2 1 0
}

// Step by 3 (like range(0, 30, 3)):
for (int i = 0; i < 30; i += 3) {
    std::cout << i << " ";  // 0 3 6 9 12 15 18 21 24 27
}

// Multiple variables:
for (int i = 0, j = 10; i < j; ++i, --j) {
    std::cout << i << " " << j << "\n";
}
// 0 10, 1 9, 2 8, 3 7, 4 6

```

0.2.38 Range-Based for Loop (C++11)

The C++ equivalent of Python's `for x in collection`:

```
numbers = [10, 20, 30, 40, 50]
for n in numbers:
    print(n)
```

```
#include <vector>
std::vector<int> numbers = {10, 20, 30, 40, 50};
for (int n : numbers) {
    std::cout << n << "\n";
}
```

0.2.38.1 The Reference vs Copy Decision

This is the most important thing to understand about range-based for in C++.

By value (copy): Each iteration creates a copy. Modifying the loop variable does NOT change the container.

```
std::vector<int> v = {1, 2, 3};
for (int n : v) {
    n *= 10;    // modifies the copy, not the element in v
}
// v is still {1, 2, 3}
```

By reference (&): The loop variable IS the element. Modifying it DOES change the container.

```
for (int& n : v) {
    n *= 10;    // modifies the actual element in v
}
// v is now {10, 20, 30}
```

By const reference (const&): Read-only access to the element, no copy made. Use this for large objects you only need to read.

```
for (const int& n : v) {
    std::cout << n << "\n";    // read-only, no copy
    // n = 0;    // COMPILE ERROR: n is const
}
```

When to use which:

Type is small (int, double, bool, char, pointer):

```
for (T x : v)            -- copy is cheap, fine
for (const T& x : v)     -- also fine
```

Type is large (string, struct, vector, custom class):

```
for (const T& x : v)    -- ALWAYS prefer: no copy, no modify
for (T& x : v)         -- when you need to modify in place
```

You don't know the type or it's complex:

```
for (const auto& x : v) -- safe universal choice
```

0.2.39 break and continue

```
for (int i = 0; i < 10; ++i) {
    if (i == 5) {
        break;           // exit the loop immediately
    }
    if (i % 2 == 0) {
        continue;       // skip to the next iteration
    }
    std::cout << i << " "; // prints: 1 3
}
// loop ends because of break when i == 5
```

break and continue only affect the **innermost** enclosing loop. For nested loops, use a function return or a flag:

```
// Method: refactor into a function, use return
bool find_target(const std::vector<std::vector<int>>& grid, int target) {
    for (int row = 0; row < (int)grid.size(); ++row) {
        for (int col = 0; col < (int)grid[row].size(); ++col) {
            if (grid[row][col] == target) {
                return true; // exits both loops
            }
        }
    }
    return false;
}
```

0.2.40 Loop Execution Visualization

For `for (int i = 0; i < 3; ++i) { body; }:`

Start:

```
int i = 0    (initialization -- runs once)
```

Iteration 1:

```
i < 3? YES (0 < 3)
body executes
++i -> i = 1
```

Iteration 2:

```
i < 3? YES (1 < 3)
body executes
++i -> i = 2
```

Iteration 3:

```
i < 3? YES (2 < 3)
body executes
++i -> i = 3
```

Check:

```
i < 3? NO (3 is not < 3)
Loop exits
```

0.2.41 Common Mistakes in This Chapter

0.2.41.1 Mistake 1: Off-By-One Error

The bug:

```
std::vector<int> v = {10, 20, 30};
for (int i = 0; i <= v.size(); ++i) { // <= instead of <
    std::cout << v[i] << "\n";      // accesses v[3] -- does not exist!
}
```

The symptom: Crash, garbage output, or silent memory corruption on the last iteration.

The fix: Use `i < v.size()` (strictly less than). Or better: use range-based for, which cannot have this bug.

0.2.41.2 Mistake 2: Forgetting break in switch

The bug:

```
switch (command) {
    case 'q':
        prepare_to_quit();
        // forgot break -- falls through to 'h'!
    case 'h':
        show_help();
        break;
}
// When command == 'q': prepare_to_quit() runs, THEN show_help() runs
```

The fix: Add break after every case.

0.2.41.3 Mistake 3: Modifying a Container While Iterating Over It

The bug:

```
std::vector<int> v = {1, 2, 3, 4, 5};
for (int n : v) {
    if (n == 3) v.erase(v.begin() + 2); // modifies v while iterating!
}
// undefined behavior -- the range-based for's internal iterator is now invalid
```

The fix: Collect things to remove first, then remove them after the loop. Or use `std::remove_if` (Chapter 29).

0.2.41.4 Mistake 4: Using = Instead of == in a Condition

The bug:

```
int x = 5;
while (x = 10) { // assigns 10 to x, condition is always true (infinite loop!)
    std::cout << x << "\n";
}
```

The symptom: Infinite loop.

The fix: `while (x == 10)`. Compile with `-Wall` to get a warning.

0.2.42 Exercises

Exercise 4.1 – Predict the output

```
for (int i = 1; i <= 10; ++i) {  
    if (i % 3 == 0) continue;  
    if (i > 7) break;  
    std::cout << i << " ";  
}
```

Answer: 1 2 4 5 7 – skips multiples of 3 (3, 6), breaks when $i > 7$ (stops before 8).

Exercise 4.2 – FizzBuzz

Print numbers 1 to 20. Multiples of 3: print “Fizz”. Multiples of 5: print “Buzz”. Multiples of both 3 and 5: print “FizzBuzz”.

Answer:

```
#include <iostream>  
int main() {  
    for (int i = 1; i <= 20; ++i) {  
        if (i % 15 == 0) std::cout << "FizzBuzz\n";  
        else if (i % 3 == 0) std::cout << "Fizz\n";  
        else if (i % 5 == 0) std::cout << "Buzz\n";  
        else std::cout << i << "\n";  
    }  
}
```

Check % 15 first. If you check % 3 first, 15 would print “Fizz” instead of “FizzBuzz”.

Exercise 4.3 – Sum with while

Use a `while` loop to compute the sum of integers from 1 to 100.

Answer:

```
int sum = 0, i = 1;  
while (i <= 100) {  
    sum += i;  
    ++i;  
}  
std::cout << sum << "\n"; // 5050
```

Exercise 4.4 – Triangle

Print this triangle with nested loops:

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Answer:

```
for (int row = 1; row <= 5; ++row) {  
    for (int col = 1; col <= row; ++col) {  
        std::cout << "* ";  
    }  
    std::cout << "\n";  
}
```

Exercise 4.5 – do-while validation

Write a program that asks the user to enter a password. Keep asking until they type “secret”. (Use `std::cin >> password` to read.)

Answer:

```
#include <iostream>  
#include <string>  
  
int main() {  
    std::string password;  
    do {  
        std::cout << "Enter password: ";  
        std::cin >> password;  
    } while (password != "secret");  
    std::cout << "Access granted.\n";  
    return 0;  
}
```

0.2.43 What Happens When You Call a Function

Before the syntax, understand what a function call does to memory. This mental model makes pass-by-value, scope, and the declaration/definition split all click at once.

Your program uses a region of memory called the **call stack**. It operates like a stack of plates: each function call pushes a new **frame** (plate) on top; when the function returns, that frame is popped off and gone.

Each frame holds: - The function's parameters (copies of the arguments) - The function's local variables - The return address (where to continue after returning)

```
int multiply(int a, int b) {
    int result = a * b;
    return result;
}

int square(int x) {
    int s = multiply(x, x);
    return s;
}

int main() {
    int answer = square(5);
    std::cout << answer << "\n"; // 25
}
```

When `square(5)` is called, then `multiply(5, 5)` is called inside `square`:

CALL STACK (top = currently executing function)

+-----+	
multiply's frame	<-- currently executing
a = 5	COPY of square's x
b = 5	COPY of square's x
result = 25	
+-----+	
square's frame	
x = 5	COPY of main's argument 5
s = ?	not yet assigned
+-----+	
main's frame	
answer = ?	not yet assigned
+-----+	

When `multiply` finishes: 1. It returns 25 2. Its entire frame is popped – `a`, `b`, `result` are gone from memory 3. Back in `square`, `s` receives the returned 25 4. `square` returns 25, its frame is popped – `x`, `s` are gone 5. In `main`, `answer` receives 25

Critical insight: `a` and `b` inside `multiply` are COPIES of `x` from `square`. They live at different memory addresses. Changing `a` inside `multiply` has absolutely no effect on `x` in `square`. This is **pass by value**.

0.2.44 Function Syntax

```
# Python -- no return type, no parameter types  
def add(a, b):  
    return a + b
```

```
// C++ -- return type required, parameter types required  
int add(int a, int b) {  
    return a + b;  
}
```

What you must provide: 1. **Return type** before the function name. `int` means the function returns an `int`. 2. **Type for each parameter**. Each parameter is `type name`. 3. A return statement if the return type is not `void`.

```
// Returns nothing:  
void print_banner() {  
    std::cout << "=====\n";  
} // no return needed for void  
  
// Returns bool:  
bool is_palindrome(std::string s) {  
    std::string rev = s;  
    std::reverse(rev.begin(), rev.end());  
    return s == rev;  
}  
  
// Multiple parameters:  
double distance(double x1, double y1, double x2, double y2) {  
    double dx = x2 - x1;  
    double dy = y2 - y1;  
    return std::sqrt(dx*dx + dy*dy);  
}  
  
// No parameters:  
int get_screen_width() {  
    return 1920;  
}
```

0.2.45 Declarations vs Definitions

This is a concept Python does not have. It exists because C++ compiles each `.cpp` file independently, top-to-bottom.

The problem: If you call a function before the compiler has seen it, the compiler does not know: - Does this function exist? - How many parameters does it take? - What types are the parameters? - What type does it return?

Without this information, the compiler cannot type-check the call. It refuses.

A declaration (also called a forward declaration or prototype) gives the compiler the function's signature without the body:

```
int add(int a, int b); // declaration: signature only, ends with semicolon
```

A definition is the full function with its body:

```
int add(int a, int b) { // definition: has the body in braces
    return a + b;
}
```

Every definition is implicitly also a declaration. A declaration without a body is not a definition.

0.2.45.1 The Three Solutions

Problem: function used before it is defined

```
int main() {
    int r = add(3, 4); // ERROR: 'add' not declared yet
}

int add(int a, int b) { return a + b; }
```

Solution 1: Move the definition above the call

```
int add(int a, int b) { return a + b; } // defined first

int main() {
    int r = add(3, 4); // OK -- compiler has seen add
}
```

Works for simple, single-file programs. Becomes unmanageable with circular dependencies (function A calls B, B calls A).

Solution 2: Forward declaration

```
int add(int a, int b); // declaration -- tells compiler the signature

int main() {
    int r = add(3, 4); // OK -- compiler knows signature, trusts you to define it
}

int add(int a, int b) { return a + b; } // definition can appear anywhere after
```

Works in a single file. For multi-file projects, use headers.

Solution 3: Header file (the real solution for multi-file projects)

```
// math.h -- declarations only
#pragma once
int add(int a, int b);
int subtract(int a, int b);

// math.cpp -- definitions
#include "math.h"
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }

// main.cpp -- uses the functions
#include "math.h" // gives main.cpp the declarations it needs
int main() {
    int r = add(3, 4); // compiler checks types via the declaration in math.h
    int s = subtract(10, 3);
}
```

We cover headers fully in Chapter 11.

0.2.46 Pass by Value: What Is Copied and Why It Matters

When you pass a variable to a function, C++ copies it by default. The function gets its own independent copy.

```
void zero_it(int x) {
    x = 0; // modifies the local copy only
    std::cout << "Inside: " << x << "\n";
}

int main() {
    int n = 42;
    zero_it(n);
    std::cout << "Outside: " << n << "\n";
}
```

Output:

```
Inside: 0
Outside: 42    <-- n was NOT changed
```

The stack during the call:


```

main's frame:                                zero_it's frame:
+-----+                                +-----+
| n = 42      |                                | x = 42      | <-- a COPY at a different address
| (at 0x7fff1000) |                                | (at 0x7fff0ff0) |
+-----+                                +-----+
                                           |
                                           | x = 0 (modifies copy)
                                           |
                                           | zero_it returns, frame popped
main's frame:
+-----+
| n = 42      | <-- unchanged
+-----+

```

This applies to every type: `int`, `double`, `std::string`, structs, classes. Passing a `std::string` with 10,000 characters by value copies all 10,000 characters. This is why Chapter 6 (References) and `const&` parameters matter.

0.2.47 Default Parameters

```

# Python
def connect(host, port=8080, timeout=30):
    print(f"Connecting to {host}:{port} (timeout={timeout}s)")

```

```

// C++
void connect(std::string host, int port = 8080, int timeout = 30) {
    std::cout << "Connecting to " << host << ":" << port
              << " (timeout=" << timeout << "s)\n";
}

connect("localhost");           // port=8080, timeout=30
connect("localhost", 9000);     // port=9000, timeout=30
connect("localhost", 9000, 60); // port=9000, timeout=60

```

Rules: 1. Default parameters must be at the **end** of the parameter list. 2. You cannot skip a middle argument: `connect("host", , 60)` is illegal. 3. Specify defaults in the **declaration** (header file), not in the definition:

```

// math.h -- put defaults here
void connect(std::string host, int port = 8080, int timeout = 30);

// math.cpp -- no defaults here (would be a redefinition error)
void connect(std::string host, int port, int timeout) {
    // implementation
}

```

0.2.48 Function Overloading

C++ lets you define multiple functions with the same name, as long as they have different parameter types. The compiler picks the right one at each call site.

```
int    abs_val(int x)    { return x < 0 ? -x : x; }
double abs_val(double x) { return x < 0.0 ? -x : x; }
float  abs_val(float x)  { return x < 0.0f ? -x : x; }

abs_val(5);           // calls int version
abs_val(-3.14);        // calls double version
abs_val(-2.0f);        // calls float version (f suffix = float literal)
```

The compiler resolves overloads at compile time using these rules (simplified): 1. **Exact match** – preferred 2. **Trivial conversion** (e.g., non-const to const) 3. **Promotion** (e.g., float to double, short to int) 4. **Standard conversion** (e.g., int to double) 5. **Ambiguous** (two overloads equally good) – compile error

```
void foo(int x)    { std::cout << "int\n"; }
void foo(double x) { std::cout << "double\n"; }

foo(5);           // "int"    -- exact match
foo(5.0);          // "double" -- exact match
foo(5.0f);         // "double" -- float promotes to double (closer than int)
foo('A');          // "int"    -- char promotes to int
// foo(true);      -- ambiguous: bool can promote to int OR double equally well
```

Python does not have overloading. You achieve similar behavior with type checks (`isinstance`) or default parameters. The C++ approach is resolved entirely at compile time with zero runtime cost.

0.2.48.1 What Overloading Is NOT

Overloading is based on **parameter types only**. You cannot overload on return type:

```
int    get() { return 5; }
double get() { return 5.0; } // ERROR: same parameters, only return type differs
```

The compiler uses the call arguments to pick the overload. If two functions take the same arguments, there is no way to distinguish them.

0.2.49 Compiler Messages for Function Errors

Understanding compiler error messages makes debugging much faster. Here are the most common function-related errors:

0.2.49.1 No Matching Function

```
void greet(std::string name) { }
```

```
int main() {  
    greet(42);    // passing int, but greet takes string  
}
```

```
error: no matching function for call to 'greet(int)'  
    note: candidate: 'void greet(std::string)'  
    note:   no known conversion from 'int' to 'std::string'
```

The compiler tells you what it found and why it didn't match.

0.2.49.2 Ambiguous Overload

```
void process(int x) { }  
void process(long x) { }
```

```
process(42);    // 42 is int -- exact match for int version, fine  
process(42L);   // 42L is long -- exact match for long version, fine  
// process(true); -- ambiguous: bool can convert to int OR long
```

```
error: call to 'process' is ambiguous  
    note: candidate: 'void process(int)'  
    note: candidate: 'void process(long)'
```

0.2.49.3 Wrong Number of Arguments

```
void foo(int a, int b) { }
```

```
foo(1, 2, 3);    // too many
```

```
error: too many arguments to function 'void foo(int, int)'  
    note: declared here: void foo(int a, int b)
```

0.2.50 Common Mistakes in This Chapter

0.2.50.1 Mistake 1: Expecting Pass-by-Value to Modify the Original

The bug:

```
void set_to_zero(int x) { x = 0; }

int count = 100;
set_to_zero(count);
std::cout << count;    // 100 -- NOT 0
```

Why it fails: `x` is a copy. Setting `x = 0` changes only the copy.

The fix: Use a reference (Chapter 6): `void set_to_zero(int& x) { x = 0; }`

0.2.50.2 Mistake 2: Missing Return Statement

The bug:

```
int max(int a, int b) {
    if (a > b) return a;
    // forgot: return b;
}
```

The symptom: Compiler warning “control may reach end of non-void function.” Calling `max` when `a <= b` returns garbage.

The fix: All code paths must return a value.

0.2.50.3 Mistake 3: Defining a Function Twice

The bug:

```
int add(int a, int b) { return a + b; }
int add(int a, int b) { return a + b; } // second definition
```

error: redefinition of 'int add(int, int)'

The fix: Declarations can appear multiple times (they're just type information). Definitions can appear exactly once.

0.2.50.4 Mistake 4: Forgetting to Compile the File Containing the Definition

The bug:

```
$ g++ -std=c++23 -o program main.cpp # forgot greet.cpp
```

undefined reference to `greet(std::string const&)'

The fix: Include all .cpp files that contain definitions you need:

```
$ g++ -std=c++23 -o program main.cpp greet.cpp
```

0.2.51 Exercises

Exercise 5.1 – Declaration or definition?

For each, say whether it is a declaration, a definition, or both:

```
double sqrt(double x); // (a)
double square(double x) { return x*x; } // (b)
void print(); // (c)
void print() { std::cout << "!\n"; } // (d)
```

Answer: - (a): declaration only (no body) - (b): definition (has body) – and also implicitly a declaration - (c): declaration only - (d): definition (and implicitly a declaration)

Exercise 5.2 – Stack trace

Draw the call stack when `inner` is executing:

```
int inner(int x) { return x * 2; }
int middle(int x) { return inner(x + 1); }
int outer(int x) { return middle(x + 1); }
int main() { int r = outer(5); }
```

Answer:

```
inner's frame:  x = 7    (outer 5 -> middle 6 -> inner 7)
middle's frame: x = 6
outer's frame:  x = 5
main's frame:   r = ?
```

inner returns $7 * 2 = 14$. Pops. middle returns 14. Pops. outer returns 14. Pops. main's $r = 14$.

Exercise 5.3 – Overloaded describe

Write three overloaded functions named `describe` that: - Take `int` and print "Integer: N" - Take `double` and print "Double: N.NN" - Take `std::string` and print "String: S (length L)"

Answer:

```
#include <iostream>
#include <string>
#include <iomanip>

void describe(int n) {
    std::cout << "Integer: " << n << "\n";
}

void describe(double d) {
    std::cout << "Double: " << std::fixed << std::setprecision(2) << d << "\n";
}

void describe(std::string s) {
    std::cout << "String: " << s << " (length " << s.size() << ")\n";
}

int main() {
    describe(42);
    describe(3.14159);
    describe(std::string{"hello"});
}
```

Output:

```
Integer: 42
Double: 3.14
String: hello (length 5)
```

Exercise 5.4 – power with default

Write `power (base, exponent=2)` that computes $\text{base}^{\text{exponent}}$ using a loop. It should work for non-negative integer exponents. Verify: `power (3)` returns 9, `power (2, 10)` returns 1024.

Answer:

```
double power(double base, int exponent = 2) {
    double result = 1.0;
    for (int i = 0; i < exponent; ++i) {
        result *= base;
    }
    return result;
}
```

Part I is complete. You now have the foundations: the compilation model, types and memory, operators, control flow, and functions.

Part II begins with the concepts that most distinguish C++ from Python: references, pointers, the stack vs the heap, and const correctness. These are where C++ becomes its own language. Ask to continue.

0.3 Part II – Core C++ (the part that is not like Python)

The next six chapters cover the concepts that have no real equivalent in Python. Python hides all of this behind its object model and garbage collector. C++ exposes it – and once you understand it, you understand why both languages make the choices they do.

Chapter 6: References – Aliases for Variables

0.3.1 The Problem References Solve

At the end of Chapter 5 you saw that passing a variable to a function copies it. That is usually fine for `int` and `double`. It is expensive for large objects and sometimes flat-out wrong when you need the function to modify the original.

```
// Costs nothing to copy:
void print_count(int n) { std::cout << n << "\n"; }

// Copying a 50,000-byte buffer is painful:
void compress(HugeBuffer data) { ... } // copies 50,000 bytes on every call

// Needs to modify the original -- copy is useless:
void swap(int a, int b) {
    int tmp = a;
    a = b;      // modifies the copy, not main's variables
    b = tmp;
}
int x = 10, y = 20;
swap(x, y);
// x is still 10, y is still 20 -- nothing swapped
```

References solve all three problems.

0.3.2 What Is a Reference?

A reference is an **alias** – another name for an existing variable. It is not a copy. It is not a pointer (though it is implemented as one). It is a second name that refers to the exact same memory location.

```
int x = 42;
int& ref = x;    // ref is a reference to x. & after the type = reference.

std::cout << x    << "\n";    // 42
std::cout << ref  << "\n";    // 42 -- same memory, same value

ref = 100;        // modifying ref modifies x (they ARE the same thing)
std::cout << x    << "\n";    // 100
std::cout << ref  << "\n";    // 100
```

Memory picture:

Before: `ref = x` declared

Address	Name	Value
0x1000	x	42

\

- +-- ref is another name for this same location
- | No new memory is allocated for ref itself.
- | (The compiler may use a pointer internally,
- | but you never see it.)

Rules for references: 1. A reference **must** be initialized when declared. `int& r;` is a compile error. 2. A reference **cannot be reseated** – once bound to a variable, it refers to that variable for its entire life. 3. After initialization, using `ref` is exactly like using the original variable.

```
int a = 1, b = 2;
int& r = a;    // r refers to a
r = b;        // this does NOT make r refer to b
              // it assigns b's value (2) TO a via r
std::cout << a; // 2
// r still refers to a, not b
```


0.3.3 Pass by Reference

To let a function modify its argument, pass it by reference:

```
void swap(int& a, int& b) {    // & in parameter = reference parameter
    int tmp = a;
    a = b;
    b = tmp;
}

int x = 10, y = 20;
swap(x, y);
std::cout << x << " " << y;    // 20 10 -- actually swapped
```

Inside swap, a IS x and b IS y. They share the same memory.

main's frame:		swap's frame:	
+-----+		+-----+	
x = 10	<--->	a (reference)	a and x ARE the same memory
y = 20	<--->	b (reference)	b and y ARE the same memory
+-----+		+-----+	
		a = 20, b = 10 executed	
+-----+			
x = 20		x changed through a	
y = 10		y changed through b	
+-----+			

Contrast with pass-by-value from Chapter 5:

```
void swap_broken(int a, int b) {    // no &, copies made
    int tmp = a; a = b; b = tmp;    // swaps the copies only
}

swap_broken(x, y);    // x and y unchanged
```

0.3.4 Pass by const Reference

When you want to avoid copying a large object but do NOT need to modify it, use const&:

```
// BAD: copies the entire string (potentially many bytes)
void print_name(std::string name) {
    std::cout << name << "\n";
}

// GOOD: no copy, but cannot modify (const ensures this)
```

```
void print_name(const std::string& name) {
    std::cout << name << "\n";
    // name = "Bob"; // COMPILE ERROR -- const reference, cannot assign
}
```

`const std::string& name` means: - & – reference (no copy) - const – cannot be modified through this reference

The caller's string is not copied. The function gets direct read-only access to the original. This is the most common parameter style for anything larger than a few bytes.

0.3.4.1 The Decision Table for Parameter Style

Parameter type	When to use
<code>int x</code>	Small types (int, double, bool, char, pointer) that are cheap to copy and you don't need to modify.
<code>int& x</code>	Small or large type that the function MUST modify. (signals to the caller: "I will change your variable")
<code>const std::string& x</code>	Large type (string, vector, struct) that you only need to read, not modify. No copy. Fastest.
<code>std::string& x</code>	Large type that the function must modify. (rare -- usually return a new value instead)

0.3.5 References Cannot Be Null

Unlike pointers (next chapter), a reference is guaranteed to refer to a valid object. You cannot have a “null reference” in well-formed C++.

```
int& bad_ref = *nullptr; // undefined behavior -- DO NOT DO THIS
                       // dereferencing null pointer gives "reference to nothing"
```

If validity is uncertain at the time of binding, use a pointer instead. References are for when you know the object exists.

0.3.6 Returning References

A function can return a reference to give the caller direct access to something inside the function... but only if that thing outlives the function call.

```
// CORRECT: returning a reference to a member of an object that outlives the call
int& get_element(std::vector<int>& v, int index) {
    return v[index];    // v exists outside this function; safe
}

std::vector<int> nums = {10, 20, 30};
get_element(nums, 1) = 99;    // assigns through the returned reference
// nums is now {10, 99, 30}
```

```
// WRONG: returning a reference to a local variable
int& bad() {
    int x = 5;
    return x;    // x is destroyed when bad() returns -- dangling reference!
}
int& r = bad();    // r refers to memory that no longer exists
r = 10;    // undefined behavior -- corrupts memory
```

The compiler often warns about this:

```
warning: reference to local variable 'x' returned [-Wreturn-local-addr]
    7 |         return x;
      |         ^
```

0.3.7 Common Mistakes in This Chapter

0.3.7.1 Mistake 1: Expecting pass-by-reference from a non-reference parameter

The bug:

```
void double_it(int n) { n *= 2; }    // n is a copy
int x = 5;
double_it(x);
std::cout << x;    // 5, not 10
```

The fix: `void double_it(int& n) { n *= 2; }`

0.3.7.2 Mistake 2: Forgetting `const` on read-only reference parameters

The bug:

```
void print_sum(std::string& s) { // non-const reference
    std::cout << s << "\n";
}
print_sum("hello"); // COMPILE ERROR: cannot bind non-const reference to rvalue
```

Why it fails: Temporary values (like string literals) cannot bind to non-const references because the compiler cannot take their address in a meaningful way. **The fix:** `void print_sum(const std::string& s)`

0.3.7.3 Mistake 3: Returning a reference to a local variable

The bug: As shown above – the local variable is destroyed on return, leaving a dangling reference. **The fix:** Only return references to objects that outlive the function (parameters, class members, statics).

0.3.8 Exercises

Exercise 6.1 – Trace the output

```
void increment(int& x) { ++x; }

int a = 5;
int& b = a;
increment(b);
std::cout << a << " " << b;
```

Answer: 6 6. `b` is a reference to `a`, so they are the same variable. `increment(b)` increments `a` (via `b` via the parameter `x`). Both names show the new value.

Exercise 6.2 – Fix the swap

```
void swap(double a, double b) {
    double tmp = a;
    a = b;
    b = tmp;
}

double x = 1.5, y = 2.5;
swap(x, y);
// x and y are unchanged -- fix swap so they actually swap
```

Answer:

```
void swap(double& a, double& b) {
    double tmp = a;
    a = b;
    b = tmp;
}
```

Exercise 6.3 – const& parameter

Write a function `print_stats(const std::vector<int>& v)` that prints the size, first element, and last element of a vector. It should not copy the vector.

Answer:

```
#include <iostream>
#include <vector>

void print_stats(const std::vector<int>& v) {
    std::cout << "Size: " << v.size() << "\n";
    std::cout << "First: " << v.front() << "\n";
    std::cout << "Last: " << v.back() << "\n";
}

int main() {
    std::vector<int> nums = {10, 20, 30, 40, 50};
    print_stats(nums); // no copy of nums made
}
```

Chapter 7: Pointers and Memory Addresses

0.3.9 The Address of a Variable

Every variable lives at a specific location in memory – a numbered address. On a 64-bit system, addresses are 64-bit numbers, usually displayed in hexadecimal.

```
int x = 42;
std::cout << &x << "\n"; // prints something like: 0x7ffee4b3c5ac
```

The `&` operator (when used in an expression, not a declaration) is the **address-of** operator. It gives you the memory address where a variable lives.

Variable `x`:

Address	Value
0x7fff10	[00][00][00][2A] <- &x is 0x7fff10; *(&x) is 42

A **pointer** is a variable that stores an address. The `*` in the type means “pointer to”:

```
int x = 42;           // x is an int
int* p = &x;          // p is a pointer to int, stores x's address

std::cout << p        << "\n";    // 0x7fff10 -- the address (the pointer value)
std::cout << *p        << "\n";    // 42      -- the value at that address (dereference)
std::cout << &p        << "\n";    // 0x7fff08 -- address of the pointer itself
```

The `*` when used with an existing pointer is the **dereference** operator – it follows the address to get the value stored there.

Memory picture:

Address	Variable	Value
0x7fff10	x (int)	42
0x7fff08	p (int*)	0x7fff10 <- p stores x's address

p is the address 0x7fff10.
*p follows that address and gives 42.

0.3.10 Modifying Through a Pointer

```
int x = 42;
int* p = &x;

*p = 100; // write 100 to the memory location p points to
           // same as: x = 100

std::cout << x;    // 100
std::cout << *p;   // 100
```

This is the lower-level version of references. Pointers and references both allow indirect modification, but:

	Reference	Pointer
Syntax	int& r = x	int* p = &x
Must be initialized	Yes	No (but you should)
Can be null	No	Yes (nullptr)

	Reference	Pointer
Can be reseated	No	Yes
Dereference syntax	just use r	*p
When to use	Aliases, function parameters	Optional relationships, arrays, dynamic memory

0.3.11 nullptr: The Null Pointer

A pointer that points to nothing is called a null pointer. Always use `nullptr` (C++11), never `NULL` (old C macro) or `0`:

```
int* p = nullptr; // p points to nothing

if (p != nullptr) {
    std::cout << *p; // safe -- only dereference if not null
}

// Dereferencing a null pointer is undefined behavior (usually a crash):
int* q = nullptr;
std::cout << *q; // CRASH -- segmentation fault
```

Null pointer:

Address	Variable	Value
0x7fff08	p (int*)	0x0000000000000000 <- nullptr (address zero)

Dereferencing: `*p` means "go to address 0 and read from there"
OS protects address 0 -- your program gets `SIGSEGV` (segfault)

Always check pointers before dereferencing if they might be null.

0.3.12 Pointer Arithmetic

Pointers support arithmetic that advances by the size of the pointed-to type:

```
int arr[5] = {10, 20, 30, 40, 50};
int* p = arr;    // p points to arr[0]

std::cout << *p      << "\n";    // 10
std::cout << *(p + 1) << "\n";    // 20 (moves 4 bytes forward, one int)
std::cout << *(p + 2) << "\n";    // 30

p++;             // p now points to arr[1]
std::cout << *p      << "\n";    // 20
```

Memory (each int is 4 bytes):

Address	Value	
0x1000	10	<- arr[0], p points here initially
0x1004	20	<- arr[1], p+1 points here
0x1008	30	<- arr[2]
0x100C	40	<- arr[3]
0x1010	50	<- arr[4]

$p + 1$ moves by `sizeof(int) = 4` bytes, landing on the next integer. This is why arrays are contiguous in memory – pointer arithmetic only works correctly because of that contiguity.

0.3.13 Pointers vs Python References

In Python, every variable is a reference (pointer) to an object. Python programmers are already using pointers – they just don't see the mechanics.

```
# Python: all "variables" are pointers
a = [1, 2, 3]
b = a    # b and a point to the SAME list
b.append(4)
print(a)    # [1, 2, 3, 4] -- a was affected through b

# Python explicitly shows this with id():
print(id(a) == id(b))    # True -- same memory address
```

```
// C++: copying by default, pointer explicitly
std::vector<int> a = {1, 2, 3};
std::vector<int> b = a;    // b is a COPY -- different memory
b.push_back(4);
std::cout << a.size();    // 3 -- a is unaffected

std::vector<int>* p = &a;    // p points to a
p->push_back(4);    // modifies a through the pointer
std::cout << a.size();    // 4
```

In Python, `b = a` for a list makes two labels for one object. In C++, `b = a` copies everything. To share, you explicitly use a pointer or reference.

0.3.14 The `->` Operator

When you have a pointer to an object and want to access its members, `->` is shorthand for `(*p).member`:

```
struct Point {
    double x, y;
};

Point pt{3.0, 4.0};
Point* p = &pt;

std::cout << (*p).x << "\n";    // 3.0 -- dereference then member access
std::cout << p->x << "\n";      // 3.0 -- same thing, cleaner syntax
p->x = 10.0;                      // modifies pt.x through the pointer
```

`->` is the idiomatic way to access members through a pointer. You will see it constantly in C++ code.

0.3.15 `const` and Pointers

There are two things that can be `const` with a pointer: the pointer itself (where it points) and the value it points to.

```
int a = 1, b = 2;

// Pointer to const int: cannot change the pointed-to value
const int* p1 = &a;
*p1 = 10;    // ERROR: *p1 is const
p1 = &b;     // OK: p1 itself can be changed (points to b now)

// Const pointer to int: cannot change where it points
int* const p2 = &a;
*p2 = 10;    // OK: the int it points to can be changed
p2 = &b;     // ERROR: p2 is const (cannot reset)

// Const pointer to const int: cannot change either
const int* const p3 = &a;
*p3 = 10;    // ERROR
p3 = &b;     // ERROR

// Memory trick: read the type right-to-left
// const int*      --> pointer to const int
// int* const      --> const pointer to int
// const int* const --> const pointer to const int
```

0.3.16 Common Mistakes in This Chapter

0.3.16.1 Mistake 1: Dereferencing a Null or Uninitialized Pointer

The bug:

```
int* p;           // uninitialized -- contains garbage address
*p = 5;          // writes to garbage address -- undefined behavior
```

The symptom: Immediate crash (segfault), or silent memory corruption that crashes later.

The fix: Always initialize pointers: `int* p = nullptr;` or `int* p = &some_variable;`

0.3.16.2 Mistake 2: Using a Pointer After the Pointed-To Variable is Destroyed

The bug:

```
int* get_ptr() {
    int local = 42;
    return &local; // local is destroyed when function returns
}
int* p = get_ptr(); // p now points to freed stack memory
std::cout << *p;    // undefined behavior -- dangling pointer
```

The fix: Never return a pointer to a local variable. Return the value, or make the variable static, or allocate on the heap (Chapter 8).

0.3.16.3 Mistake 3: Confusing & as Address-Of vs & as Reference Type

The confusion:

```
int x = 5;
int& r = x; // & in declaration = reference type
int* p = &x; // & in expression = address-of operator
```

The position matters: & after a type in a declaration means “reference.” & before a variable in an expression means “take the address of.”

0.3.17 Exercises

Exercise 7.1 – Trace the pointer

```
int a = 10, b = 20;
int* p = &a;
*p = 30;
p = &b;
*p += 5;
std::cout << a << " " << b;
```

Answer: 30 25. Step by step: `*p = 30` sets a to 30. `p = &b` makes p point to b. `*p += 5` adds 5 to b ($20 + 5 = 25$).

Exercise 7.2 – Pointer parameter

Rewrite `double_it` to take a pointer parameter instead of a reference:

```
void double_it(int* p) {
    *p *= 2;
}
int x = 7;
double_it(&x);    // must pass address explicitly
std::cout << x;   // 14
```

Which style (reference or pointer) is more idiomatic in modern C++ for this use case? Why?

Answer: Reference is more idiomatic. It is simpler (`double_it(x)` vs `double_it(&x)`), cannot be null, and cannot be reseated. Pointers are used when nullability or reseating is needed.

Exercise 7.3 – const correctness with pointers

Declare a pointer p to `double` such that: - The value at p cannot be changed through p - p itself can be pointed at a different `double`

Then declare q such that: - q is fixed to always point to the same `double` - The value at q CAN be changed through q

Answer:

```
double a = 1.0, b = 2.0;
const double* p = &a;    // ptr to const double
p = &b;                  // OK -- p can be reseated
// *p = 5.0;            // ERROR -- cannot modify through p

double* const q = &a;    // const pointer to double
*q = 5.0;                // OK -- can modify value
// q = &b;              // ERROR -- cannot reseat q
```

Chapter 8: The Stack and the Heap

0.3.18 Two Regions of Memory

Every running program has two main regions where variables can live:

+-----+	
Stack	
- Fast (just moves a pointer)	
- Automatic (compiler manages lifetime)	
- Limited size (~1-8 MB)	
- Local variables, function parameters	
+-----+	
(other memory: code, globals...)	
+-----+	
Heap	
- Slower (OS/allocator involved)	
- Manual (YOU manage lifetime in C++)	
- Huge (limited by RAM)	
- Dynamic allocation: new, malloc	
+-----+	

Python hides this entirely – all objects live on the heap, managed by the garbage collector. C++ exposes both regions and lets you choose where memory comes from.

0.3.19 The Stack in Detail

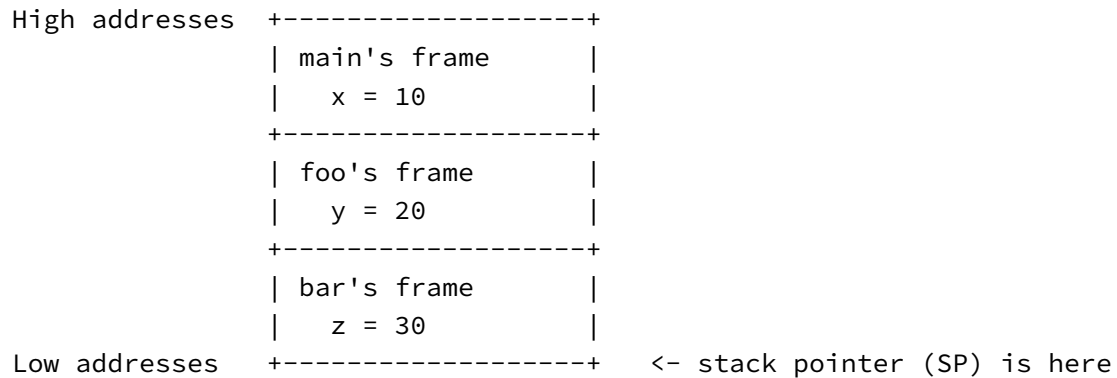
The stack is a LIFO (Last In, First Out) structure. Local variables and function parameters live here.

```
void bar() {
    int z = 30;
    // z lives on the stack while bar() is running
}

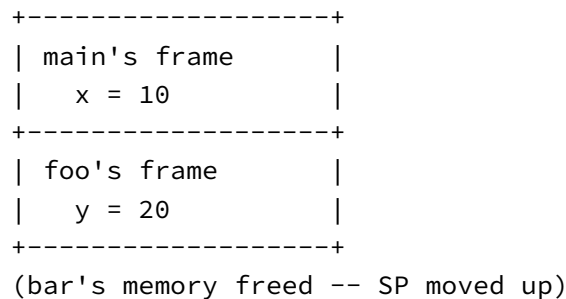
void foo() {
    int y = 20;
    bar();    // bar's frame pushed on top of foo's
    // z is gone (bar's frame popped); y is back on top
}

int main() {
    int x = 10;
    foo();
    // y is gone; x is still here
}
```

Call stack during bar():



After bar() returns:



Allocating stack memory is nearly free: the CPU just decrements the stack pointer register by the number of bytes needed. Freeing stack memory is also free: the CPU increments the stack pointer back.

Stack overflow: If you allocate too much on the stack (e.g., declaring a 10MB array locally, or infinitely deep recursion), the stack runs into other memory and the OS kills your program.

```
// Stack overflow example:
void recurse() { recurse(); } // infinite recursion
recurse(); // segfault when stack space exhausted

// Stack overflow with large local array:
void dangerous() {
    int huge[2000000]; // ~8 MB on the stack -- likely crashes
}
```

0.3.20 The Heap in Detail

The heap is a large pool of memory managed by the OS and your allocator. You request chunks with `new` and release them with `delete`.

```
int* p = new int{42};    // allocates 4 bytes on the heap; p stores the address
std::cout << *p << "\n"; // 42
delete p;                // releases the 4 bytes back to the allocator
p = nullptr;             // good practice: prevents accidental reuse
```

After `int* p = new int{42}`:

Stack:		Heap:
+-----+		+-----+
p = 0x555f2a1b10	----->	42 (4 bytes)
+-----+		+-----+

After `delete p`:

Stack:		Heap:
+-----+		(memory returned to heap pool)
p = 0x555f2a1b10	---X-->	(no longer valid to access)
+-----+		

After `p = nullptr`:

+-----+		
p = 0x0		(safe -- cannot accidentally dereference)
+-----+		

0.3.20.1 Heap Arrays

```
int n = 1000;
int* arr = new int[n]{};    // allocate n ints on the heap, zero-initialized
arr[0] = 10;
arr[n-1] = 99;

delete[] arr; // MUST use delete[] for arrays (not delete)
arr = nullptr;
```

The size does not need to be a compile-time constant – you can compute it at runtime. This is the main use case for heap allocation.

0.3.21 Why Python Uses the Heap for Everything

Python allocates every object on the heap and uses reference counting to track when objects can be freed. This is why Python is flexible but slower:

```
x = [1, 2, 3]    # list allocated on heap
y = x           # y is another reference to the same heap object
                # reference count goes from 1 to 2
del x           # reference count goes from 2 to 1 -- NOT freed
del y           # reference count goes from 1 to 0 -- freed!
```

C++ gives you the choice: - Stack: fast, automatic, size known at compile time - Heap: flexible, manual (or smart pointers), any size

```
// Stack -- automatic lifetime, fast
std::vector<int> v = {1, 2, 3}; // v is on the stack
// vector's CONTENTS are on the heap (internally), but you don't manage that
// v is automatically destroyed when it goes out of scope

// Heap -- manual lifetime (rare in modern C++ -- use smart pointers instead)
std::vector<int>* vp = new std::vector<int>{1, 2, 3};
delete vp; // you must remember to do this
```

0.3.22 Memory Leaks

A memory leak is heap memory that was allocated but never freed. In C++, the OS reclaims all memory when the process exits, so short-lived programs don't suffer from leaks permanently. But long-running programs (servers, games) slowly consume more and more memory until they crash or the machine runs out.

```
void leaky() {
    int* p = new int{42}; // allocates heap memory
    // forgot: delete p;
    // function returns -- p (the pointer) is gone
    // the heap memory at p's address is still allocated, now unreachable
}

for (int i = 0; i < 1000000; ++i)
    leaky(); // leaks 4 bytes per call = 4 MB leaked
```

Detection: Run with `valgrind ./program` or compile with `-fsanitize=address`:

```
==12345== LEAK SUMMARY:
==12345==    definitely lost: 4 bytes in 1 blocks
==12345==    at 0x4C2FB0F: operator new(unsigned long) (vg_replace_malloc.c:334)
==12345==    by 0x10868B: leaky() (example.cpp:2)
```

The real fix is to never use raw `new/delete`. Use `std::vector`, `std::string`, and smart pointers (Chapter 14), which manage heap memory automatically through RAII.

0.3.23 Stack vs Heap: Which to Use

Use the stack (local variables) when:

- Size is known at compile time
- Lifetime matches the current scope
- Performance is critical (tight loops, small structures)

Examples: `int`, `double`, small structs, `std::array<int, 10>`

Use the heap (via containers or smart pointers) when:

- Size is determined at runtime (user input, file content)
- Lifetime must extend beyond the creating function
- Data is very large (millions of elements)

Examples: `std::vector`, `std::string`, objects stored in maps

In modern C++:

- You almost never write `new/delete` directly
- `std::vector` and `std::string` manage their heap memory internally
- smart pointers (`unique_ptr`, `shared_ptr`) handle ownership automatically
- Raw `new/delete` is a code smell in modern C++

0.3.24 Common Mistakes in This Chapter

0.3.24.1 Mistake 1: `delete` Instead of `delete[]` for Arrays

The bug:

```
int* arr = new int[100];
delete arr;    // undefined behavior! Should be delete[]
```

The symptom: Memory corruption or crash – the allocator uses the wrong size to free. **The fix:** `delete[] arr;`

0.3.24.2 Mistake 2: Double Delete

The bug:

```
int* p = new int[5];
delete p;
delete p;    // undefined behavior -- freeing already-freed memory
```

The symptom: Crash or silent heap corruption. **The fix:** Set pointer to `nullptr` after deleting. Deleting `nullptr` is a no-op.

0.3.24.3 Mistake 3: Using After Delete (Use-After-Free)

The bug:

```
int* p = new int{5};
delete p;
std::cout << *p;    // reads freed memory -- undefined behavior
```

Detection: `-fsanitize=address` reports “heap-use-after-free” with a stack trace.

0.3.25 Exercises

Exercise 8.1 – Where does it live?

For each variable, say whether it lives on the stack or heap:

```
int a = 5;
int* b = new int{10};
std::string s = "hello";
std::vector<int> v = {1, 2, 3};
```

Answer: - a: stack - b: stack (the pointer). The `int` it points to: heap. - s: stack (the `std::string` object itself). The character data it manages: heap (internally). - v: stack (the `std::vector` object). The `int` elements it manages: heap (internally).

Exercise 8.2 – Spot the leak

How many memory leaks does this code have?

```
void process() {
    int* a = new int{1};
    int* b = new int{2};
    if (*a > 0) return;    // early return!
    delete a;
    delete b;
}
```

Answer: Two leaks when `*a > 0` (which is always true here, since `a = 1`). The early return bypasses both `delete` calls. This is exactly why RAII (Chapter 12) and smart pointers (Chapter 14) exist – they clean up even when exceptions or early returns happen.

Exercise 8.3 – Heap array

Allocate an array of 5 doubles on the heap, set them to 1.1, 2.2, 3.3, 4.4, 5.5, print them, then free the memory correctly.

Answer:

```
double* arr = new double[5]{1.1, 2.2, 3.3, 4.4, 5.5};
for (int i = 0; i < 5; ++i)
    std::cout << arr[i] << "\n";
delete[] arr;
arr = nullptr;
```

Chapter 9: const Correctness

0.3.26 Why const Matters

const is not just a safety net for your own mistakes. It is a contract: you tell the compiler “this value will not change,” and the compiler enforces that contract everywhere the value is used. This contract propagates through the codebase, letting the compiler catch bugs at compile time that would otherwise be silent data corruption at runtime.

In Python, there is no const. You use naming conventions (ALL_CAPS) to signal “don’t change this,” but nothing enforces it. C++’s const is enforced.

0.3.27 const Variables

```
const int MAX_PLAYERS = 8;
MAX_PLAYERS = 10;    // COMPILER ERROR: assignment of read-only variable
```

The compiler rejects any assignment to MAX_PLAYERS after initialization. Every future reader of the code knows with certainty: this value never changes.

```
const double PI = 3.14159265358979;
const std::string GREETING = "Hello";
```

Prefer const for anything that does not need to change. It is documentation that is enforced.

0.3.28 const Function Parameters

A const parameter promises not to modify the argument:

```
void print_length(const std::string& s) {
    std::cout << s.size() << "\n";
    s = "modified";    // COMPILE ERROR -- s is const reference
}
```

This is important for const& parameters: you can pass temporaries (rvalues) to them:

```
void print_length(const std::string& s) { ... }

print_length("hello");           // OK: string literal binds to const&
print_length(std::string{"hi"}); // OK: temporary binds to const&

void modify(std::string& s) { ... }
modify("hello");                 // ERROR: non-const reference cannot bind to temporary
```

0.3.29 const Member Functions

When you write a class, member functions that do not modify the object's state should be marked const. This is the const at the end of the function signature:

```
class Rectangle {
public:
    Rectangle(double w, double h) : width{w}, height{h} {}

    double area() const {           // const: does not modify this object
        return width * height;
    }

    void scale(double factor) {     // non-const: modifies this object
        width *= factor;
        height *= factor;
    }

private:
    double width, height;
};

const Rectangle r{3.0, 4.0};
std::cout << r.area();             // OK -- area() is const, usable on const objects
r.scale(2.0);                     // ERROR -- scale() is non-const, not usable on const objects
```

The const after the parameter list means: “this function can be called on const objects, and it promises not to modify the object.”

Rules: - On a const object, only const member functions can be called - A non-const object can call both const and non-const member functions - Inside a const member function, you cannot assign to member variables

```
double area() const {
    width = 0;    // COMPILE ERROR -- modifying member in const function
    return width * height;
}
```

0.3.30 constexpr – Compile-Time Constants

constexpr goes further than const – the value must be computable at compile time:

```
constexpr int BOARD_SIZE = 8;
constexpr double TAX_RATE = 0.085;
constexpr int CELLS = BOARD_SIZE * BOARD_SIZE;    // 64, computed at compile time

// constexpr functions:
constexpr int factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}

constexpr int FACT_10 = factorial(10);    // 3628800, computed at compile time
// The result is embedded in the binary as a constant -- zero runtime cost
```

```
// Cannot be constexpr if value comes from runtime:
int n;
std::cin >> n;
constexpr int x = n;    // ERROR: n is not a compile-time constant
const int y = n;        // OK: const, but runtime value
```

constexpr is useful for: - Mathematical constants (PI, E, conversion factors) - Array sizes (int grid[BOARD_SIZE][BOARD_SIZE]; – array size must be compile-time) - Lookup tables computed at compile time - Performance: zero-cost named constants

0.3.31 Cascading const

const propagates: once you have a const object, everything you do with it must also be const.

```
std::vector<int> v1 = {1, 2, 3};
const std::vector<int> v2 = {4, 5, 6};

v1.push_back(4);    // OK -- v1 is not const
v2.push_back(7);    // ERROR -- v2 is const; push_back is non-const
```

```
int x = v1[0];           // OK
int y = v2[0];           // OK -- operator[] has a const overload (returns const ref)
v2[0] = 99;              // ERROR -- v2[0] returns const int&, cannot assign
```

When you pass by `const&`, you can only call `const` member functions on the object. The compiler checks the entire call chain.

0.3.32 Common Mistakes in This Chapter

0.3.32.1 Mistake 1: Omitting `const` on Member Functions That Should Be `const`

The bug:

```
class Circle {
    double radius;
public:
    double area() { // forgot const
        return 3.14159 * radius * radius;
    }
};

const Circle c{5.0};
c.area(); // ERROR: 'this' argument discards qualifiers (area is non-const)
```

The fix: `double area() const { ... }` – add `const` after the parameter list.

0.3.32.2 Mistake 2: Trying to Modify Through a `const` Reference

The bug:

```
void process(const std::string& s) {
    s += " world"; // ERROR: s is const, cannot modify
}
```

The fix: Take by value (`std::string s`) if you need a modifiable copy, or by non-`const` reference (`std::string& s`) if you intend to modify the original.

0.3.33 Exercises

Exercise 9.1 – Mark const correctly

Which of these should be const members?

```
class Counter {
    int count = 0;
public:
    void increment()    { ++count; }
    int get_count()    { return count; }
    void reset()        { count = 0; }
    bool is_zero()      { return count == 0; }
};
```

Answer: `get_count()` and `is_zero()` should be const – they read but do not modify `count`. `increment()` and `reset()` modify `count`, so they cannot be const.

```
void increment()    { ++count; }
int get_count() const { return count; }
void reset()        { count = 0; }
bool is_zero()      const { return count == 0; }
```

Exercise 9.2 – constexpr table

Write a constexpr function `celsius_to_fahrenheit(double c)` and use it to create a compile-time constant for the boiling point of water (100°C).

Answer:

```
constexpr double celsius_to_fahrenheit(double c) {
    return c * 9.0 / 5.0 + 32.0;
}
constexpr double WATER_BOILING_F = celsius_to_fahrenheit(100.0); // 212.0
```

Chapter 10: Arrays, `std::vector`, and `std::string`

0.3.34 C-Style Arrays (Understand, then Avoid)

The C language had arrays built in. C++ inherited them. They are the underlying model behind everything else, so understand them, then use `std::vector` instead.

```
int scores[5];           // array of 5 ints (uninitialized -- garbage!)
int primes[5] = {2, 3, 5, 7, 11}; // initialized
int zeros[100] = {};     // all zeros (zero-initialized with {})

std::cout << primes[0] << "\n"; // 2 -- zero-indexed
std::cout << primes[4] << "\n"; // 11
```

Memory layout of `primes[5]`:

Address	Value	
0x1000	2	<- <code>primes[0]</code>
0x1004	3	<- <code>primes[1]</code>
0x1008	5	<- <code>primes[2]</code>
0x100C	7	<- <code>primes[3]</code>
0x1010	11	<- <code>primes[4]</code>

The name 'primes' in expressions decays to a pointer to `primes[0]`.

0.3.34.1 Critical Weakness: No Bounds Checking

C++ does not check if your index is valid at runtime:

```
int arr[3] = {1, 2, 3};
std::cout << arr[5]; // reads memory past the array -- undefined behavior
arr[5] = 99;        // writes past the array -- corrupts other data!
```

This is one of the leading causes of security vulnerabilities. The compiler doesn't catch it and there is no runtime exception – it just reads or writes whatever is at that address.

0.3.34.2 Size Is Not Carried With the Array

```
int arr[5] = {1, 2, 3, 4, 5};
sizeof(arr); // 20 -- size in bytes (only works if arr is in scope as an array)

void bad_print(int arr[]) { // array decays to pointer when passed to function
    sizeof(arr);           // 8 -- size of pointer, NOT the array!
}
```

You must pass the size separately, which is error-prone. `std::vector` solves this.

0.3.35 `std::vector` – The Right Way to Do Dynamic Arrays

`std::vector` is C++'s resizable array. It is what Python's `list` most closely corresponds to:

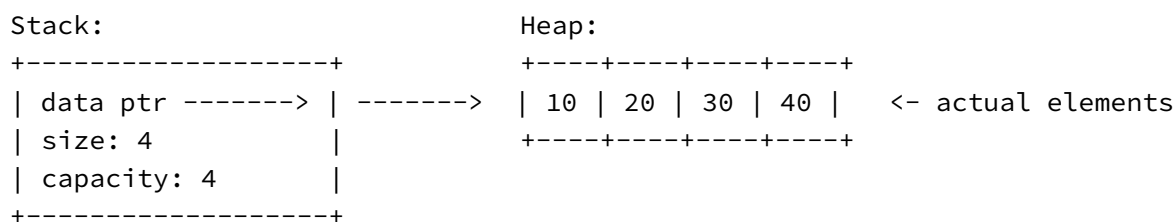
```
# Python list
numbers = [10, 20, 30]
numbers.append(40)
print(len(numbers))    # 4
print(numbers[1])      # 20
```

```
// C++ vector
#include <vector>
std::vector<int> numbers = {10, 20, 30};
numbers.push_back(40);
std::cout << numbers.size() << "\n";    // 4
std::cout << numbers[1] << "\n";        // 20
```

0.3.35.1 What's Inside `std::vector`

A `std::vector` is a small object (24 bytes typically) that internally manages a heap-allocated array:

```
std::vector<int> numbers = {10, 20, 30, 40};
```



- `size`: number of elements currently in the vector
- `capacity`: how many elements fit in the currently allocated heap space
- When `size == capacity` and you `push_back`, the vector allocates a bigger heap array (typically doubles), copies all elements, and frees the old array.

You never see any of this. The vector manages it transparently.

0.3.35.2 Key Operations

```
std::vector<int> v;           // empty vector
v.push_back(10);              // append 10: v = {10}
v.push_back(20);              // append 20: v = {10, 20}
v.push_back(30);              // append 30: v = {10, 20, 30}

v.size();                     // 3    -- number of elements
v.empty();                     // false -- is it empty?
v.front();                     // 10   -- first element
v.back();                      // 30   -- last element
```



```

v.pop_back();           // removes last: v = {10, 20}

v[0];                  // 10 -- no bounds check (unsafe, fast)
v.at(0);               // 10 -- bounds-checked (throws if out of range)

v.insert(v.begin() + 1, 99); // insert 99 at index 1: v = {10, 99, 20}
v.erase(v.begin() + 1);    // remove element at index 1: v = {10, 20}

v.clear();              // remove all elements (size = 0, capacity kept)
v.resize(5, 0);         // resize to 5 elements, new ones initialized to 0

std::vector<int> w(10, 42); // 10 elements all set to 42

```

0.3.35.3 Iterating Over a Vector

```

std::vector<int> v = {1, 2, 3, 4, 5};

// Range-based for (preferred):
for (int n : v)      std::cout << n << " "; // copies each element
for (int& n : v)     n *= 2;                // modifies in place
for (const int& n : v) std::cout << n << " "; // read-only, no copy

// Index-based (when you need the index):
for (int i = 0; i < (int)v.size(); ++i) {
    std::cout << i << ": " << v[i] << "\n";
}

// Note: v.size() returns size_t (unsigned). Comparing int i < size_t is
// technically a warning. Cast to (int)v.size() or use size_t i.

```

0.3.35.4 2D Vectors

```

// 3 rows, 4 columns, all initialized to 0:
std::vector<std::vector<int>> grid(3, std::vector<int>(4, 0));

grid[1][2] = 99;
std::cout << grid[1][2]; // 99

```

0.3.36 std::string – The Right Way to Handle Text

```

# Python strings
s = "hello"
s += " world"
print(len(s))      # 11
print(s.upper())   # HELLO WORLD
print(s[1:5])      # ello

```

```
// C++ strings
#include <string>
std::string s = "hello";
s += " world";
std::cout << s.size() << "\n";          // 11
// (no built-in upper() -- use a loop or std::transform)

std::string sub = s.substr(1, 4);        // "ello" (start=1, length=4)
```

`std::string` internally manages a heap-allocated buffer of characters, similar to `std::vector<char>`.

0.3.36.1 Key Operations

```
std::string s = "Hello, World!";

s.size();           // 13 -- number of characters
s.empty();          // false
s[0];               // 'H' -- no bounds check
s.at(0);            // 'H' -- bounds-checked
s.front();          // 'H'
s.back();           // '!'

s.find("World");     // 7 -- index where found
s.find("xyz");       // std::string::npos -- not found

s.substr(7, 5);      // "World" (start=7, length=5)

s.replace(7, 5, "C++"); // "Hello, C++"

std::string t = "hello";
s.compare(t);        // negative (s < t lexicographically)

// Concatenation:
std::string a = "foo", b = "bar";
std::string c = a + b; // "foobar"
a += "!";             // "foo!"

// Convert to/from numbers:
#include <string>
std::string num_str = std::to_string(42); // "42"
int n = std::stoi("123");                // 123
double d = std::stod("3.14");             // 3.14
```

0.3.36.2 String Comparison

Unlike Python where `==` compares value, C++ `std::string` also uses `==` for value comparison:

```
std::string a = "hello", b = "hello";
if (a == b) std::cout << "equal\n"; // equal -- compares content
```

0.3.36.3 Raw C-Style Strings

You will see C-style strings (`const char*`) in older code and C interfaces:

```
const char* cs = "hello";    // points to read-only character array
std::string s = cs;         // convert: C-string to std::string (fine)

// std::string to C-string (for C library functions):
const char* c = s.c_str();  // null-terminated char array
// valid only as long as s is alive and unmodified
```

Always use `std::string` for your own code. Use `const char*` only when a C API requires it, and immediately pass it through – do not store it.

0.3.37 `std::array` – Fixed-Size Array With Safety

When the size is known at compile time, use `std::array` instead of a C-style array:

```
#include <array>
std::array<int, 5> primes = {2, 3, 5, 7, 11};

primes.size();           // 5 -- size is always available
primes[2];               // 5
primes.at(10);           // throws std::out_of_range -- bounds checked
primes.front();          // 2
primes.back();           // 11
```

`std::array` knows its own size, can be passed to functions without separately passing the size, and supports all the standard iteration patterns.

0.3.38 Common Mistakes in This Chapter

0.3.38.1 Mistake 1: Off-By-One / Out-of-Bounds Access

The bug:

```
std::vector<int> v = {1, 2, 3};
for (int i = 0; i <= (int)v.size(); ++i) // <= instead of <
    std::cout << v[i] << "\n";          // v[3] is past the end
```

Detection: `-fsanitize=address` reports “heap-buffer-overflow.” **The fix:** Use `<` not `<=`. Better: use range-based for.

0.3.38.2 Mistake 2: Modifying a Vector While Iterating With Indices

The bug:

```
std::vector<int> v = {1, 2, 3, 4, 5};
for (int i = 0; i < (int)v.size(); ++i) {
    if (v[i] % 2 == 0) v.erase(v.begin() + i);
    // After erasing index 1 (value 2), index 1 now holds 3 (shifted)
    // The loop increments i to 2, skipping 3
}
```

The fix: Iterate backward, or use the erase-remove idiom (Chapter 29).

0.3.38.3 Mistake 3: Using + to Concatenate Non-String Literals

The bug:

```
std::string result = "Count: " + 42; // ERROR: no + between const char* and int
```

The fix:

```
std::string result = "Count: " + std::to_string(42);
// or better (C++23):
std::string result = std::format("Count: {}", 42);
```

0.3.38.4 Mistake 4: Accessing .c_str() Pointer After Modifying the String

The bug:

```
std::string s = "hello";
const char* p = s.c_str();
s += " world"; // may reallocate the internal buffer
printf("%s\n", p); // p may now be dangling
```

The fix: Call .c_str() immediately before passing it to the C function, never store it.

0.3.39 Exercises

Exercise 10.1 – Vector operations

Starting from an empty `std::vector<int>`:

1. Push back 5, 10, 15, 20, 25
2. Print the size (should be 5)
3. Remove the last element
4. Insert the value 99 at index 2
5. Print all elements

Answer:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    v.push_back(5);
    v.push_back(10);
    v.push_back(15);
    v.push_back(20);
    v.push_back(25);

    std::cout << v.size() << "\n";    // 5

    v.pop_back();                      // removes 25; v = {5,10,15,20}

    v.insert(v.begin() + 2, 99);      // v = {5,10,99,15,20}

    for (int n : v)
        std::cout << n << " ";      // 5 10 99 15 20
    std::cout << "\n";
}
```

Exercise 10.2 – String manipulation

Given `std::string sentence = "the quick brown fox"`: 1. Print its length 2. Extract and print the substring “quick” (starts at index 4, length 5) 3. Find where “brown” appears 4. Replace “fox” with “cat” 5. Print the final string

Answer:

```
std::string sentence = "the quick brown fox";
std::cout << sentence.size() << "\n";    // 19
std::cout << sentence.substr(4, 5) << "\n"; // quick
std::cout << sentence.find("brown") << "\n"; // 10

size_t pos = sentence.find("fox");
sentence.replace(pos, 3, "cat");
std::cout << sentence << "\n";           // the quick brown cat
```

Exercise 10.3 – 2D grid

Create a 4x4 grid (vector of vectors), fill position `[row][col]` with the value `row * 4 + col`, then print it as a square.

Answer:

```
std::vector<std::vector<int>> grid(4, std::vector<int>(4));
for (int r = 0; r < 4; ++r)
    for (int c = 0; c < 4; ++c)
        grid[r][c] = r * 4 + c;

for (const auto& row : grid) {
    for (int n : row)
        std::cout << std::setw(3) << n;
    std::cout << "\n";
}
```

Output:

```
0  1  2  3
4  5  6  7
8  9 10 11
12 13 14 15
```

Chapter 11: Scope, Lifetime, and Organizing Code into Files

0.3.40 Scope

Scope is the region of code where a name is visible. In C++, every pair of `{ }` creates a new scope.

```
int x = 1;           // x in outer scope

{
    int x = 2;        // x in inner scope -- SHADOWS the outer x
    std::cout << x;    // 2 -- inner x
}                    // inner x destroyed here

std::cout << x;        // 1 -- outer x, back in scope
```

Shadowing (declaring an inner variable with the same name as an outer one) is legal but confusing. Avoid it. Compile with `-Wshadow` to get warnings.

0.3.40.1 Scope in Loops and Conditionals

```
for (int i = 0; i < 10; ++i) {
    // i only visible inside the loop body
}
// i does NOT exist here

if (int n = get_value(); n > 0) { // C++17 initializer
    // n only visible inside the if/else
}
```

```

} else {
    // n also visible here
}
// n does NOT exist here

```

0.3.41 Lifetime

Lifetime is the period during which a variable's storage is valid. For stack variables, it matches scope. For heap variables, it starts at `new` and ends at `delete`.

```

{
    std::string s = "hello"; // s constructed, storage allocated
    // ... use s ...
} // s destructed, storage freed
// using s here is undefined behavior (it no longer exists)

```

The key insight: in C++, when a variable goes out of scope, its **destructor** is called automatically. For `std::string`, the destructor frees the heap-allocated character buffer. For `std::vector`, it frees the element array. This automatic cleanup is called **RAII** (Chapter 12) and is C++'s answer to garbage collection.

0.3.42 Organizing Code: The Header/Source Split

As programs grow, you split code into multiple files. The organization:

```

projectname/
  include/
    math_utils.h      <- declarations (the public interface)
  src/
    math_utils.cpp    <- definitions (the implementation)
    main.cpp          <- uses the functions

```

0.3.42.1 Why Headers Exist

C++ compiles each `.cpp` file separately. When `main.cpp` calls `multiply(3, 4)`, the compiler must know (at compile time) what `multiply` looks like – its parameter types and return type – to type-check the call.

The linker resolves where `multiply`'s actual code is. But the compiler must see the declaration.

The solution: put declarations in a header file. Every `.cpp` that needs to call `multiply` includes the header.

```
// math_utils.h -- declarations only
#pragma once

int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);
double divide(double a, double b);
```

`#pragma once` tells the preprocessor: include this file at most once per compilation unit, even if it is `#include`d multiple times. It prevents duplicate declaration errors from diamond-shaped include chains.

```
// math_utils.cpp -- definitions
#include "math_utils.h" // include own header to verify declarations match

int add(int a, int b)      { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
double divide(double a, double b) { return a / b; }
```

```
// main.cpp -- uses the functions
#include <iostream>
#include "math_utils.h" // gets the declarations; tells compiler the signatures

int main() {
    std::cout << add(3, 4) << "\n"; // 7
    std::cout << multiply(6, 7) << "\n"; // 42
    std::cout << divide(10.0, 4.0) << "\n"; // 2.5
}
```

Build:

```
$ g++ -std=c++23 -Wall -o program src/main.cpp src/math_utils.cpp -I include/
```

The `-I include/` tells the compiler to look in the `include/` directory for headers.

0.3.42.2 The Compilation Model With Multiple Files

```
math_utils.cpp --> [compiler] --> math_utils.o  --+
main.cpp       --> [compiler] --> main.o        --+--> [linker] --> program
```

Each `.cpp` is compiled independently. The compiler only needs to see the header (declarations) to compile `main.cpp`. It trusts that `math_utils.cpp` will provide the actual definitions. The linker connects everything.

0.3.42.3 What Goes In Headers vs Source Files

Headers (.h):

- Function declarations
- Class declarations (but not the bodies of non-inline methods)
- Type definitions (structs, enums, using aliases)
- #include directives needed by the declarations
- inline functions (small utility functions that are called often)
- constexpr and const variable definitions

Source (.cpp):

- Function definitions (the actual code)
- Class method definitions
- Global variable definitions
- #include of their own header + other headers needed for the implementation
- Implementation details not exposed to users

Do NOT put in headers: - using namespace std; - it pollutes every file that includes your header - Non-inline function definitions - causes “multiple definition” linker errors if included in multiple .cpp files - Definitions of non-const global variables - also causes linker errors

0.3.42.4 Include Guards (Alternative to #pragma once)

Older code uses include guards instead of #pragma once:

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

// ... declarations ...

#endif // MATH_UTILS_H
```

If math_utils.h is included twice in the same compilation unit, the second inclusion sees that MATH_UTILS_H is already defined and skips everything inside. #pragma once does the same thing more concisely and is supported by all modern compilers.

0.3.43 Namespaces

Namespaces prevent name collisions between libraries. When two libraries both define Matrix, they can put it in different namespaces.

```

namespace geometry {
    struct Point { double x, y; };
    double distance(Point a, Point b);
}

namespace graphics {
    struct Point { float x, y, z; }; // different Point, no conflict
    void draw(Point p);
}

// Use with ::
geometry::Point p1{1.0, 2.0};
graphics::Point p2{3.0f, 4.0f, 5.0f};

```

The standard library lives in `std`. That is why everything is `std::cout`, `std::vector`, `std::string`.

You can pull specific names into scope:

```

using std::cout; // just cout, not everything
using std::vector;

cout << "hello\n"; // OK
vector<int> v; // OK

```

Do this inside functions, not at file scope (it affects the rest of the file, which may cause surprises in headers).

0.3.43.1 Writing Your Own Namespace

```

// mylib.h
#pragma once

namespace mylib {

int clamp(int value, int lo, int hi);
double lerp(double a, double b, double t);

} // namespace mylib

// mylib.cpp
#include "mylib.h"

namespace mylib {

int clamp(int value, int lo, int hi) {
    if (value < lo) return lo;
    if (value > hi) return hi;
    return value;
}

double lerp(double a, double b, double t) {
    return a + t * (b - a);
}

```

```
}  
  
} // namespace mylib
```

0.3.44 Static Local Variables

A static local variable is initialized once and persists across calls:

```
int counter() {  
    static int count = 0; // initialized only once (first call)  
    ++count;  
    return count;  
}  
  
std::cout << counter() << "\n"; // 1  
std::cout << counter() << "\n"; // 2  
std::cout << counter() << "\n"; // 3
```

The variable `count` lives for the entire program duration (like a global), but is only accessible inside `counter`. This is useful for functions that need to remember state between calls without using a class.

0.3.45 Common Mistakes in This Chapter

0.3.45.1 Mistake 1: Defining a Non-Inline Function in a Header

The bug:

```
// utils.h  
int add(int a, int b) { return a + b; } // DEFINITION in header  
  
// main.cpp includes utils.h  
// utils.cpp also includes utils.h  
// Both .cpp files define add() -- linker error!
```

linker error: multiple definition of `'add(int, int)'`

The fix: Declarations in headers, definitions in `.cpp` files. Or mark the function `inline` (which tells the linker to allow multiple identical definitions).

0.3.45.2 Mistake 2: Missing `#pragma once` (or Include Guard)

The bug:

```
// a.h includes b.h and c.h
// b.h includes c.h
// c.h has no include guard

// When a.h is processed: c.h is included twice, all declarations in c.h appear twice
error: redefinition of 'class Foo'
```

The fix: Start every header with `#pragma once`.

0.3.45.3 Mistake 3: using namespace std; in a Header

The bug:

```
// utils.h
#include <string>
using namespace std;    // pollutes every file that includes utils.h

// Now every file including utils.h gets all of std:: imported
// Causes conflicts with user-defined names like 'vector', 'string', 'max', 'min'
```

The fix: Never put `using namespace X;` in a header. In `.cpp` files it is acceptable (though `using std::cout;` for specific names is better style).

0.3.46 Exercises

Exercise 11.1 – Trace scope

Predict the output:

```
int x = 10;
{
    int x = 20;
    {
        int x = 30;
        std::cout << x << "\n";
    }
    std::cout << x << "\n";
}
std::cout << x << "\n";
```

Answer: 30, 20, 10. Each inner `x` shadows the outer one. When the inner scope ends, the outer `x` becomes visible again.

Exercise 11.2 – Split into files

Split this single-file program into `main.cpp`, `greeting.h`, and `greeting.cpp`:

```
#include <iostream>
#include <string>

std::string make_greeting(const std::string& name) {
    return "Hello, " + name + "!";
}

int main() {
    std::cout << make_greeting("Alice") << "\n";
    std::cout << make_greeting("Bob") << "\n";
}
```

Answer:

```
// greeting.h
#pragma once
#include <string>

std::string make_greeting(const std::string& name);
```

```
// greeting.cpp
#include "greeting.h"

std::string make_greeting(const std::string& name) {
    return "Hello, " + name + "!";
}
```

```
// main.cpp
#include <iostream>
#include "greeting.h"

int main() {
    std::cout << make_greeting("Alice") << "\n";
    std::cout << make_greeting("Bob") << "\n";
}
```

Build: `g++ -std=c++23 -o program main.cpp greeting.cpp`

Exercise 11.3 – Static local counter

Write a function `next_id()` that returns 1 on the first call, 2 on the second, and so on, without using any global variable.

Answer:

```
int next_id() {
    static int id = 0;
    return ++id;
}
// next_id() == 1, next_id() == 2, next_id() == 3 ...
```

Exercise 11.4 – Namespace

Write a namespace `convert` with two functions: `km_to_miles(double km)` and `miles_to_km(double miles)`. 1 km = 0.621371 miles.

Answer:

```
// convert.h
#pragma once

namespace convert {
    double km_to_miles(double km);
    double miles_to_km(double miles);
}
```

```
// convert.cpp
#include "convert.h"

namespace convert {
    double km_to_miles(double km)    { return km * 0.621371; }
    double miles_to_km(double miles) { return miles / 0.621371; }
}
```

Usage:

```
std::cout << convert::km_to_miles(100.0) << "\n";    // 62.1371
std::cout << convert::miles_to_km(62.0) << "\n";    // 99.79...
```

Part II is complete. You now understand the concepts that have no Python equivalent: references as aliases, pointers and memory addresses, the stack vs heap memory model, const correctness enforced by the compiler, the standard library containers, and how C++ organizes multi-file projects.

Part III covers ownership and memory management – the RAII pattern, smart pointers, and move semantics. These are what make modern C++ safe while staying fast. Ask to continue.

0.4 Part III – Ownership and Memory Management

This part covers what separates experienced C++ programmers from beginners: understanding *who owns a resource* and *when it gets cleaned up*. Python’s garbage collector answers both questions automatically. C++ makes you – or your types – answer them explicitly.

The central pattern is RAII. Everything in this part flows from it.

Chapter 12: RAII – The Core Idea That Replaces Garbage Collection

0.4.1 The Problem: Resources Need Cleanup

Some things you acquire must eventually be released:

Resource	Acquire	Release
Heap memory	<code>new</code>	<code>delete</code>
File	<code>fopen / open</code>	<code>fclose / close</code>
Mutex lock	<code>lock()</code>	<code>unlock()</code>
Network socket	<code>socket() / connect()</code>	<code>close()</code>
Database connection	<code>connect()</code>	<code>disconnect()</code>

If you forget to release, you get leaks, deadlocks, or corruption. If an exception or early return happens between acquire and release, the release never runs.

```
void risky() {
    int* data = new int[1000];

    if (something_failed()) {
        return;           // LEAK: data never deleted
    }

    if (something_else_failed()) {
        throw std::runtime_error("oops"); // LEAK: data never deleted
    }

    delete[] data; // only reached on the happy path
}
```

This is the problem RAII solves.

0.4.2 What RAII Is

RAII stands for **Resource Acquisition Is Initialization**. The name is cryptic; the idea is simple:

Tie a resource's lifetime to the lifetime of an object. Acquire the resource in the constructor. Release the resource in the destructor.

When the object goes out of scope, C++ automatically calls its destructor. The destructor releases the resource. This happens even if an exception is thrown, even if there is an early return. The cleanup is guaranteed.

```
class IntArray {
    int* data;
    int size;
public:
    IntArray(int n) : data{new int[n]{}}, size{n} {
        // constructor: acquires the resource (heap memory)
    }

    ~IntArray() {           // destructor: ~ prefix, no return type
        delete[] data;      // releases the resource
    }

    int& operator[](int i) { return data[i]; }
    int get_size() const { return size; }
};

void safe() {
    IntArray arr{1000};     // constructor runs: allocates 1000 ints

    if (something_failed()) {
        return;             // destructor runs: delete[] data -- NO LEAK
    }

    if (something_else_failed()) {
        throw std::runtime_error("oops"); // destructor runs -- NO LEAK
    }

    // destructor runs at end of scope -- NO LEAK
}
```

The destructor runs at scope exit **no matter how the scope exits**. Return, exception, fall-through – the destructor always runs.

0.4.3 The Destructor

A destructor is a special member function:


```

class Foo {
public:
    Foo() { std::cout << "constructed\n"; } // constructor
    ~Foo() { std::cout << "destroyed\n"; } // destructor (~ prefix)
};

{
    Foo a;
    Foo b;
    std::cout << "inside block\n";
} // b destructs first (LIFO), then a

```

Output:

```

constructed
constructed
inside block
destroyed    <- b (last in, first out)
destroyed    <- a

```

Destructors run in **reverse order of construction** (last in, first out – like the stack). This matters when objects depend on each other.

0.4.4 RAI for File Handling

Without RAI:

```

void write_data_bad(const std::string& filename) {
    FILE* f = fopen(filename.c_str(), "w");
    if (!f) return; // ok, no file to close

    // ... write stuff ...

    if (error_condition) {
        return; // LEAK: file never closed!
    }

    fclose(f); // only reached on happy path
}

```

With RAI (using `std::fstream` from the standard library):

```

#include <fstream>

void write_data_good(const std::string& filename) {
    std::ofstream f{filename}; // constructor opens the file
    if (!f) return;
}

```

```

    // ... write stuff ...

    if (error_condition) {
        return;           // destructor closes the file -- always
    }

} // destructor closes the file -- always

```

`std::ofstream`'s destructor closes the file handle. You never call `fclose`. The file is always closed, regardless of how the function exits.

0.4.5 RAII for Mutex Locks

```

#include <mutex>

std::mutex mtx;

void bad_worker() {
    mtx.lock();
    if (error) {
        return;           // DEADLOCK: mutex never unlocked!
    }
    mtx.unlock();
}

void good_worker() {
    std::lock_guard<std::mutex> guard{mtx}; // constructor: locks mtx
    if (error) {
        return;           // destructor: unlocks mtx -- always
    }
} // destructor: unlocks mtx -- always

```

`std::lock_guard` is a tiny RAII wrapper. It locks on construction and unlocks on destruction. The mutex is always released, even if an exception is thrown.

0.4.6 Python's Equivalent: Context Managers

Python's `with` statement provides similar guarantees:

```

# Python RAII equivalent: context manager
with open("file.txt", "w") as f:
    f.write("hello")
# f.__exit__() called here -- file closed even on exception

```

C++'s RAI is more general and automatic: it applies to every object with a destructor, with no special syntax at the call site. In Python you must explicitly write `with`.

0.4.7 The Lifetime Guarantee Visualized

Scope:

```
{
    IntArray arr{1000};    <- constructor: allocates memory
    process(arr);
    if (fail) return;      <- early return?  destructor still runs!
    if (err) throw ...;    <- exception?      destructor still runs!
}
```

<- normal exit: destructor runs

Stack unwind on exception:

When an exception propagates, C++ calls the destructor of every local object as it unwinds each stack frame. No resource is leaked.

0.4.8 Common Mistakes in This Chapter

0.4.8.1 Mistake 1: Not Writing a Destructor for Resource-Owning Classes

The bug:

```
class Buffer {
    int* data;
public:
    Buffer(int n) { data = new int[n]; }
    // forgot: ~Buffer() { delete[] data; }
};
// Every Buffer that goes out of scope leaks its allocation.
```

The fix: If a class owns a raw pointer acquired with `new`, it needs a destructor with `delete`.

0.4.8.2 Mistake 2: Destructors That Throw

The bug:

```
~MyClass() {
    if (!cleanup()) throw std::runtime_error("cleanup failed"); // DANGEROUS
}
```

Why it's dangerous: If a destructor throws while the stack is already unwinding from another exception, `std::terminate` is called and the program crashes with no recovery. **The fix:** Never let destructors throw. Swallow or log errors inside destructors.

0.4.9 Exercises

Exercise 12.1 – Trace the destructor order

```
struct Log {
    std::string name;
    Log(std::string n) : name{n} { std::cout << "create " << name << "\n"; }
    ~Log() { std::cout << "destroy " << name << "\n"; }
};

int main() {
    Log a{"A"};
    {
        Log b{"B"};
        Log c{"C"};
    }
    Log d{"D"};
}
```

What is the output?

Answer:

```
create A
create B
create C
destroy C
destroy B
create D
destroy D
destroy A
```

C and B are destroyed in reverse order when the inner block ends. D is created after the block. Then A and D are destroyed in reverse order when main ends.

Exercise 12.2 – Design a RAII wrapper

Design (declarations only, no need to implement) an RAII class `FileHandle` that wraps `FILE*`. What members does it need? What should the constructor and destructor do?

Answer:

```

class FileHandle {
    FILE* file;
public:
    FileHandle(const char* path, const char* mode); // opens: file = fopen(path, mode)
    ~FileHandle(); // closes: if (file) fclose(file)
    bool is_open() const; // returns file != nullptr
    FILE* get() const; // returns the raw FILE* for C API use

    // Disable copy (two wrappers closing the same file = double-close bug):
    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;
};

```

Chapter 13: Dynamic Allocation: new, delete, and Why You Avoid Them

0.4.10 new and delete Revisited

We introduced new and delete in Chapter 8. Here is the complete picture.

```

// Single object:
int* p = new int{42}; // allocate one int on heap, initialize to 42
delete p; // free it
p = nullptr; // prevent accidental reuse

// Array:
int* arr = new int[100]{}; // allocate 100 ints, zero-initialized
delete[] arr; // MUST use delete[] for arrays
arr = nullptr;

// Default-initialized (no value specified):
int* q = new int; // value is garbage (same as uninitialized local)
delete q;

// Value-initialized (zero):
int* r = new int{}; // value is 0
delete r;

```

0.4.10.1 What new Actually Does

1. Calls operator new to ask the allocator for N bytes of heap memory
2. Constructs the object in that memory (runs the constructor)
3. Returns a typed pointer to the constructed object

0.4.10.2 What delete Actually Does

1. Calls the destructor of the object
2. Calls operator delete to return the memory to the allocator

If you use delete on an array (allocated with new[]), the destructor for each element is called, and then the memory is freed. If you use delete instead of delete[], only one destructor is called and the allocator is given the wrong size – undefined behavior.

0.4.11 The Problems With Raw new/delete

0.4.11.1 Problem 1: Exception Safety

```
int* a = new int{1};  
int* b = new int{2};    // if this throws (out of memory), a leaks  
process(a, b);          // if this throws, a and b leak  
delete a;  
delete b;
```

0.4.11.2 Problem 2: Ownership Ambiguity

```
int* create() { return new int{42}; }    // who must call delete?  
  
void use(int* p) {  
    // Am I supposed to delete p? Is it heap-allocated? Who owns it?  
}
```

When raw pointers are passed around, ownership becomes unclear. This leads to either leaks (nobody deletes) or double-frees (two places both delete).

0.4.11.3 Problem 3: Paired Delete is Fragile

Every new must be matched by exactly one delete. This is an invariant you must maintain manually across hundreds or thousands of lines of code, through exceptions and early returns. One mistake = leak or crash.

0.4.12 When Raw new/delete Is Acceptable

Almost never in modern C++. The standard library provides better alternatives:

Old pattern	Modern replacement
<code>new int[n]</code>	<code>std::vector<int></code>
<code>new SomeClass(...)</code>	<code>std::make_unique<SomeClass>(...)</code>
Shared ownership	<code>std::make_shared<SomeClass>(...)</code>
Fixed-size array	<code>std::array<T, N></code>

The only legitimate uses of raw new/delete are: - Writing a custom allocator or container - Interfacing with a C API that expects you to free() memory it returned - Writing operator new/operator delete for a class

If you find yourself writing new in application code, stop and use a smart pointer or container.

0.4.13 std::bad_alloc – Out of Memory

When new cannot get memory (system is out of RAM), it throws std::bad_alloc. Most programs do not handle this because there is little useful you can do, but for critical systems:

```
#include <new>
try {
    int* p = new int[1'000'000'000'000]; // 1 trillion ints ~ 4TB
} catch (const std::bad_alloc& e) {
    std::cerr << "Out of memory: " << e.what() << "\n";
}
```

If you want new to return nullptr instead of throwing:

```
int* p = new(std::nothrow) int[1'000'000'000'000];
if (p == nullptr) {
    // handle out of memory
}
```

0.4.14 Common Mistakes in This Chapter

0.4.14.1 Mistake 1: delete on Stack Memory

The bug:

```
int x = 5;
int* p = &x;
delete p; // undefined behavior -- x is on the stack, not heap
```

The symptom: Crash or heap corruption. **The rule:** Only delete memory that came from new.

0.4.14.2 Mistake 2: Accessing Memory After delete

```
int* p = new int{10};
delete p;
std::cout << *p; // use-after-free: undefined behavior
```

Detection: -fsanitize=address reports “heap-use-after-free.” **The fix:** Set `p = nullptr` immediately after delete.

0.4.15 Exercises

Exercise 13.1 – Manual heap string

Without using `std::string`, allocate a char array on the heap large enough to hold “Hello, C++!” (including the null terminator), fill it using `strcpy`, print it, then free it correctly.

Answer:

```
#include <cstring>
const char* src = "Hello, C++!";
int len = strlen(src) + 1; // +1 for null terminator
char* buf = new char[len];
strcpy(buf, src);
std::cout << buf << "\n";
delete[] buf;
buf = nullptr;
```

Exercise 13.2 – Identify the errors

How many bugs does this code have? Classify each as leak, double-free, or use-after-free:


```
int* p = new int{5};
int* q = p;
delete p;
delete q;           // (a)
std::cout << *p;    // (b)
int x = 10;
delete &x;          // (c)
```

Answer: - (a): double-free – p and q point to the same memory; freeing both is double-free. - (b): use-after-free – p was deleted; reading *p is undefined behavior. - (c): delete on stack memory – x is a local variable, not heap-allocated.

Chapter 14: Smart Pointers: `unique_ptr`, `shared_ptr`, `weak_ptr`

0.4.16 The Core Idea

Smart pointers are RAII wrappers around raw pointers. They manage ownership: when the smart pointer is destroyed, it automatically deletes the object it owns.

```
#include <memory>

// Instead of:
int* raw = new int{42};
// ... possibly forget to delete ...
delete raw;

// Use:
auto smart = std::make_unique<int>(42);
// ... no delete needed -- automatically freed when smart goes out of scope ...
```

There are three kinds, each for a different ownership pattern.

0.4.17 `std::unique_ptr` – Exclusive Ownership

`unique_ptr` represents **sole ownership**: exactly one `unique_ptr` owns the object at any time. When the `unique_ptr` is destroyed, it deletes the owned object.

```
#include <memory>

auto p = std::make_unique<int>(42); // allocates int{42} on heap
                                     // p is the sole owner

std::cout << *p << "\n";           // 42 -- dereference like a raw pointer
std::cout << p.get() << "\n";       // raw address (for C API use)
```

```
// p is automatically deleted when it goes out of scope
// No manual delete. No leaks.
```

`make_unique<T>(args...)` is the right way to create a `unique_ptr`. It: 1. Allocates the object on the heap 2. Constructs it with the given arguments 3. Returns a `unique_ptr` owning it

0.4.17.1 Unique Ownership Enforced at Compile Time

```
auto p = std::make_unique<int>(42);
auto q = p;    // COMPILE ERROR: unique_ptr cannot be copied
```

You cannot copy a `unique_ptr` – that would create two owners for one object (a contradiction of unique ownership). The compiler enforces this.

You CAN move it (transfer ownership):

```
auto p = std::make_unique<int>(42);
auto q = std::move(p);    // ownership transferred from p to q
// p is now empty (nullptr); q owns the int
std::cout << *q << "\n";    // 42
// *p would be undefined behavior -- p is now empty
```

`std::move` is covered in Chapter 15. For now, understand that `std::move(p)` says “I am done with p; transfer what it owns to q.”

0.4.17.2 Unique Pointer to a Custom Class

```
struct Player {
    std::string name;
    int health;
    Player(std::string n, int h) : name{n}, health{h} {}
    ~Player() { std::cout << name << " destroyed\n"; }
};

{
    auto hero = std::make_unique<Player>("Alice", 100);
    hero->health -= 20;    // -> accesses members (same as raw pointer)
    std::cout << hero->name << " has " << hero->health << " HP\n";
}    // hero goes out of scope -- Player destructor called, memory freed
// prints: "Alice has 80 HP" then "Alice destroyed"
```

0.4.17.3 `unique_ptr` as a Function Parameter

```
// Takes ownership (caller cannot use the pointer after this):
void consume(std::unique_ptr<Player> p) {
    std::cout << "consuming " << p->name << "\n";
} // p destroyed here

// Borrows (caller keeps ownership, function just uses the object):
void use(const Player& p) { // prefer this
    std::cout << "using " << p.name << "\n";
}
void use_ptr(const Player* p) { // raw ptr = borrow, no ownership
    if (p) std::cout << "using " << p->name << "\n";
}

auto hero = std::make_unique<Player>("Bob", 80);
use(*hero); // dereference to get Player&
use_ptr(hero.get()); // .get() returns raw pointer, no ownership transfer
consume(std::move(hero)); // hero is empty after this
```

The guideline: pass `unique_ptr` only when you intend to transfer ownership. For “just using” the object, pass a reference or raw pointer (raw pointer = borrow, no ownership implied).

0.4.18 `std::shared_ptr` – Shared Ownership

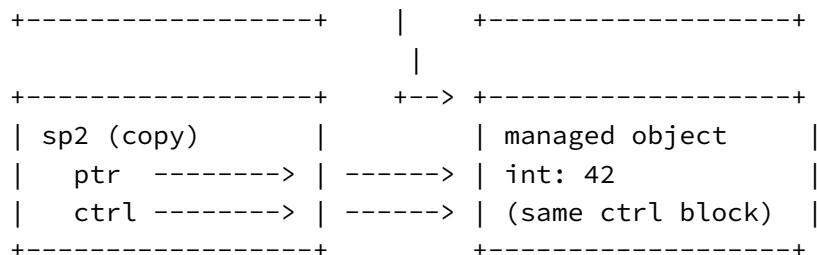
`shared_ptr` uses **reference counting**: multiple `shared_ptr`s can all own the same object. The object is deleted when the last `shared_ptr` to it is destroyed.

```
# Python reference counting -- the same idea Python uses for all objects
a = [1, 2, 3] # ref count = 1
b = a        # ref count = 2
del a        # ref count = 1
del b        # ref count = 0 --> freed
```

```
auto sp1 = std::make_shared<int>(42); // ref count = 1
{
    auto sp2 = sp1; // ref count = 2 (COPY IS ALLOWED for shared_ptr)
    std::cout << *sp2 << "\n"; // 42
    std::cout << sp1.use_count() << "\n"; // 2
} // sp2 destroyed, ref count = 1
// int still alive (sp1 holds it)
std::cout << sp1.use_count() << "\n"; // 1
// sp1 goes out of scope, ref count = 0, int deleted
```

Memory layout of `shared_ptr`:

sp1	ptr	ctrl	control block	ref count: 2	weak count: 0
-----	-----	------	---------------	--------------	---------------



0.4.18.1 When to Use shared_ptr

Use `shared_ptr` when multiple objects genuinely need to share ownership and you cannot determine statically which one will outlive the others:

- Scene graph nodes that can be referenced from multiple places
- Cache entries referenced by many users
- Callbacks registered with multiple event systems

Do NOT use `shared_ptr` by default. It has overhead (atomic ref-count increment/decrement on every copy and destroy). If one owner makes sense, use `unique_ptr`. If no ownership is needed (just borrowing), use a reference or raw pointer.

0.4.19 std::weak_ptr – Non-Owning Observer

A `weak_ptr` holds a non-owning reference to an object managed by `shared_ptr`. It does not affect the ref count. You must convert it to a `shared_ptr` to actually use the object, and that conversion can fail if the object was already deleted.

The main use: breaking reference cycles.

```

// Without weak_ptr: a cycle keeps both objects alive forever (memory leak)
struct Node {
    std::shared_ptr<Node> next;    // strong reference
};
auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b;
b->next = a;    // cycle: a holds b, b holds a -- neither ever freed

// With weak_ptr: the cycle is broken
struct Node {
    std::weak_ptr<Node> next;    // weak reference (non-owning)
};
// Now b->next does not keep a alive. When a goes out of scope, it is freed.

```

Using a `weak_ptr`:

```

auto sp = std::make_shared<int>(99);
std::weak_ptr<int> wp = sp;

if (auto locked = wp.lock()) { // lock() returns shared_ptr if alive, empty if dead
    std::cout << *locked << "\n"; // 99
} else {
    std::cout << "object was deleted\n";
}

sp.reset(); // delete the object

if (auto locked = wp.lock()) {
    std::cout << *locked << "\n";
} else {
    std::cout << "object was deleted\n"; // this prints
}

```

0.4.20 Choosing the Right Smart Pointer

Is there exactly one owner that will definitely outlive all users?

--> `std::unique_ptr` (default choice)

Do multiple owners genuinely need to share the object's lifetime?

--> `std::shared_ptr`

Do you need to observe a `shared_ptr`-managed object without affecting its lifetime?

--> `std::weak_ptr`

Are you just borrowing -- the object's owner is clear and nearby?

--> `T&` (reference) or `const T&` (const reference)

--> `T*` (raw pointer) if nullability is needed

90% of cases: use `unique_ptr`. 9%: `shared_ptr`. 1%: `weak_ptr`.

0.4.21 Common Mistakes in This Chapter

0.4.21.1 Mistake 1: Creating a `shared_ptr` From a Raw Pointer Twice

The bug:

```

int* raw = new int{42};
auto sp1 = std::shared_ptr<int>(raw);
auto sp2 = std::shared_ptr<int>(raw); // two independent shared_ptrs own raw!
// When both are destroyed: double-free

```

The fix: Always use `std::make_shared`. If you must wrap an existing pointer, do it once and copy the `shared_ptr`.

0.4.21.2 Mistake 2: Calling `.get()` and Storing the Result

The bug:

```
auto sp = std::make_shared<int>(5);
int* raw = sp.get(); // raw is a non-owning pointer -- fine so far
sp.reset();          // sp destroys the int
std::cout << *raw;   // use-after-free
```

The fix: Never store the result of `.get()` beyond the lifetime of the smart pointer.

0.4.21.3 Mistake 3: Passing `unique_ptr` by Value When Borrowing

The bug:

```
void display(std::unique_ptr<Player> p) { ... } // takes ownership!
auto hero = std::make_unique<Player>("Alice", 100);
display(hero); // ERROR: cannot copy unique_ptr
display(std::move(hero)); // compiles, but hero is now empty -- likely a bug
```

The fix: For borrowing, pass `const Player&` or `Player*` (via `.get()`).

0.4.22 Exercises

Exercise 14.1 – `unique_ptr` basics

Rewrite this raw-pointer code using `unique_ptr`:

```
double* compute() {
    double* result = new double{3.14};
    return result; // caller must delete
}

int main() {
    double* r = compute();
    std::cout << *r << "\n";
    delete r;
}
```

Answer:

```
#include <memory>
#include <iostream>

std::unique_ptr<double> compute() {
    return std::make_unique<double>(3.14);
}

int main() {
    auto r = compute(); // unique_ptr takes ownership
    std::cout << *r << "\n"; // 3.14
    // r automatically deleted at end of main
}
```

Exercise 14.2 – shared_ptr ref count

Predict the ref count at each comment:

```
auto a = std::make_shared<int>(10); // (1)
{
    auto b = a; // (2)
    auto c = a; // (3)
    c.reset(); // (4)
} // (5)
// (6)
```

Answer: - (1): 1 - (2): 2 - (3): 3 - (4): 2 (c . reset () releases c's ownership) - (5): 1 (b goes out of scope) - (6): 1 (a is still alive outside the block)

Exercise 14.3 – Choose the smart pointer

For each scenario, say whether to use `unique_ptr`, `shared_ptr`, `weak_ptr`, or a raw reference:

- A game entity that owns a weapon (weapon lives exactly as long as the entity)
- A texture loaded into a cache, referenced by hundreds of sprites
- A parent node in a tree that wants to observe its own child (child is managed by parent via `shared_ptr`)
- A function that needs to read-only access an object whose lifetime is certain to outlast the function

Answer: - a: `unique_ptr<Weapon>` – one owner, no sharing. - b: `shared_ptr<Texture>` – many owners, object lives until last sprite is done. - c: `weak_ptr<Node>` – parent already holds a `shared_ptr` to child; back-reference via `weak_ptr` avoids cycle. - d: `const T&` (reference) – just borrowing, no ownership needed.

Chapter 15: Move Semantics, lvalues, and rvalues

0.4.23 The Performance Problem With Copies

Consider what happens when you return a `std::vector` from a function:

```
std::vector<int> make_million() {  
    std::vector<int> v(1'000'000, 0); // 1 million ints on heap: ~4 MB  
    return v;  
}  
  
std::vector<int> result = make_million();
```

Naively, this would copy 4 MB from `v` into `result`. That is a lot of work. But modern C++ avoids this copy entirely. To understand how, you need lvalues and rvalues.

0.4.24 lvalues and rvalues

An **lvalue** is an expression that has a stable memory address – you can take its address with `&`, and it persists beyond the current expression. Named variables are lvalues.

An **rvalue** is a temporary – it has no persistent address. Literals, arithmetic results, and return values are rvalues.

```
int x = 5;  
int y = x + 3;  
  
// x is an lvalue: it has an address, it persists  
// 5 is an rvalue: it is a temporary value with no address of its own  
// x + 3 is an rvalue: the result is a temporary  
  
int* p = &x; // OK: x is an lvalue, can take its address  
int* q = &(x + 3); // ERROR: x+3 is an rvalue, no persistent address
```

The shorthand: if you can put it on the left side of an assignment, it is an lvalue. Rvalues cannot be assigned to.

```
x = 42; // OK: x is an lvalue  
x + 3 = 42; // ERROR: x+3 is an rvalue, you cannot assign to it
```


0.4.25 The Move Operation

An rvalue is a temporary that is about to be discarded. If you are initializing a new object from a temporary, there is no need to copy the temporary's data – you can just steal it.

This is the **move operation**: instead of copying data from source to destination and then destroying the source, move hands ownership of the source's resources directly to the destination, then puts the source in a valid-but-empty state.

Copy:

```
[Source] --> makes a duplicate --> [Destination]
[Source] still valid and full      [Destination] is a new copy
```

Move:

```
[Source] --> hands over resources --> [Destination]
[Source] is now empty (valid but empty) [Destination] owns the data
```

For a `std::vector` with 1 million elements:

Copy: allocate 4MB, copy 1M integers, now two 4MB allocations exist

Move: copy 3 pointers (data, size, capacity), set source to empty state
-- essentially free

0.4.26 `std::move` – Casting to rvalue

By default, named variables are lvalues and are copied. To tell C++ “treat this lvalue as a temporary so it can be moved,” use `std::move`:

```
std::vector<int> a(1'000'000, 42); // a: 4MB

std::vector<int> b = a;           // COPY: 4MB allocated, 1M ints copied
std::vector<int> c = std::move(a); // MOVE: 3 pointers copied, a is now empty
// a is empty after the move -- do not use a anymore
```

`std::move` does NOT move anything. It just casts `a` to an rvalue reference, signaling that the move constructor (instead of the copy constructor) should be used.

0.4.27 Move Semantics and Returned Values

```
std::vector<int> make_million() {
    std::vector<int> v(1'000'000, 0);
    return v;    // v is a local variable, about to be destroyed
                // compiler applies NRVO (named return value optimization)
                // or the implicit move -- either way, no copy
}

auto result = make_million();    // no copy of 4MB
```

The compiler applies **NRVO** (Named Return Value Optimization) or uses the implicit move. In practice, returning a local variable from a function is always efficient in modern C++. Do not write `return std::move(v)` – that actually disables NRVO.

0.4.28 Move Constructor and Move Assignment

You can define how your own class moves:

```
class Buffer {
    int* data;
    int size;

public:
    // Constructor
    Buffer(int n) : data{new int[n]{}}, size{n} {}

    // Destructor
    ~Buffer() { delete[] data; }

    // Copy constructor (deep copy -- expensive)
    Buffer(const Buffer& other) : data{new int[other.size]{}}, size{other.size} {
        std::copy(other.data, other.data + size, data);
    }

    // Move constructor (steal the pointer -- cheap)
    Buffer(Buffer&& other) noexcept    // && = rvalue reference
        : data{other.data}, size{other.size} {
        other.data = nullptr;    // source is now empty
        other.size = 0;
    }

    // Move assignment
    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data;    // free current data
            data = other.data;    // steal other's data
            size = other.size;
            other.data = nullptr;    // leave other empty
            other.size = 0;
        }
        return *this;
    }
}
```

```
    }
};
```

The `&&` in `Buffer(Buffer&& other)` is an **rvalue reference** – a reference that binds only to temporaries (rvalues). This overload is chosen when moving from a temporary.

`noexcept` tells the compiler “this function will not throw.” Move operations should be `noexcept` whenever possible – some standard library algorithms (like `std::vector` reallocation) can only use the move constructor if it is `noexcept`.

0.4.29 `std::string` and `std::vector` Move in Action

```
std::string s1 = "This is a long string that lives on the heap";
std::string s2 = s1;           // copy: s2 gets its own copy of the chars
std::string s3 = std::move(s1); // move: s3 steals the char buffer from s1
                               // s1 is now an empty string ""

std::cout << s2 << "\n"; // "This is a long string..."
std::cout << s3 << "\n"; // "This is a long string..."
std::cout << s1 << "\n"; // "" (empty -- moved-from state)
```

0.4.30 When C++ Moves Automatically

You do not always need `std::move` explicitly. Moves happen automatically:

1. **Returning a local variable from a function** (NRVO or implicit move)
2. **Initializing from a temporary** (rvalue): `auto s = get_string();`
3. **Passing a temporary to a function:** `process(std::vector<int>{1,2,3});`

You need explicit `std::move` when: 1. You want to transfer ownership from one named variable to another 2. You want to move-insert into a container

```
std::vector<std::string> words;
std::string word = "hello";
words.push_back(word);           // copy: word still valid
words.push_back(std::move(word)); // move: word is now empty, faster
```

0.4.31 Common Mistakes in This Chapter

0.4.31.1 Mistake 1: Using a Moved-From Object

The bug:

```
std::vector<int> src = {1, 2, 3};
auto dst = std::move(src);
for (int n : src) { ... } // src is empty -- loop runs zero times
                          // (well-defined but probably wrong)
```

The rule: After `std::move(x)`, treat `x` as if it were default-constructed (empty). Reassign before using.

0.4.31.2 Mistake 2: `return std::move(local)` Disabling NRVO

The bug:

```
std::vector<int> make() {
    std::vector<int> v = {1, 2, 3};
    return std::move(v); // PESSIMIZATION: disables NRVO
}
```

The fix: Just write `return v;`. The compiler already knows to move or elide.

0.4.32 Exercises

Exercise 15.1 – lvalue or rvalue?

Classify each expression as lvalue or rvalue:

```
int x = 5;
int arr[3] = {1,2,3};

x           // (a)
x + 1       // (b)
arr[0]      // (c)
42          // (d)
std::string{"hello"} // (e)
```

Answer: - (a) lvalue – `x` is a named variable with a persistent address - (b) rvalue – `x + 1` is a temporary result - (c) lvalue – `arr[0]` refers to a specific array element with an address - (d) rvalue – `42` is a literal, no persistent address - (e) rvalue – a temporary `std::string` constructed inline

Exercise 15.2 – Move vs copy performance

Explain in your own words why moving a `std::vector<int>` with 1 million elements is faster than copying it.

Answer: A `std::vector` consists of three things: a pointer to the heap-allocated element array, a size, and a capacity. Copying requires allocating a new heap array and copying all 1 million integers – $O(N)$ work proportional to the number of elements. Moving just copies the three pointer/size/capacity values and sets the source to empty – $O(1)$ work, regardless of the number of elements. The move does not touch the elements at all; it transfers ownership of the existing heap array.

Exercise 15.3 – Efficient string collection

Write a function that takes a `std::string` and pushes it into a `std::vector<std::string>`. Write two versions: one that copies, one that moves. When would you use each?

Answer:

```
std::vector<std::string> words;

// Version 1: copy (use when you need to keep the original)
void add_copy(const std::string& s) {
    words.push_back(s);           // copies s into the vector
}

// Version 2: move (use when you are done with the original)
void add_move(std::string s) {    // takes by value (already a copy or move from caller)
    words.push_back(std::move(s)); // moves into the vector
}

std::string word = "hello";
add_copy(word);           // word still valid ("hello")
add_move(std::move(word)); // word is now empty
```

Chapter 16: The Rule of 0, 3, and 5

0.4.33 The Problem: Special Member Functions

C++ classes have six **special member functions** that the compiler can generate automatically:

Function	Signature	What it does
Default constructor	<code>T()</code>	Creates an object with no arguments
Copy constructor	<code>T(const T&)</code>	Creates a copy of another object
Copy assignment	<code>T& operator=(const T&)</code>	Overwrites this object with a copy

Function	Signature	What it does
Destructor	<code>~T()</code>	Cleans up when the object goes out of scope
Move constructor	<code>T(T&&)</code>	Creates object by stealing from a temporary
Move assignment	<code>T& operator=(T&&)</code>	Overwrites this object by stealing a temporary

The compiler generates these automatically if you do not write them. The compiler-generated versions do **memberwise** operations – they copy/move/destroy each member in turn.

The problem: the compiler-generated versions are wrong when your class **owns a resource** (raw pointer, file handle, socket, etc.).

0.4.34 The Rule of Zero

If your class does **not** own any raw resources, do not define any special member functions. Let the compiler generate them all. The compiler's versions will correctly copy/move/destroy each member.

```
// Rule of Zero: no raw resources, no user-defined special members
struct PlayerStats {
    std::string name;           // std::string manages its own memory
    int score{0};
    std::vector<int> history;    // std::vector manages its own memory
    // No raw pointers, no new/delete -- compiler generates correct defaults
};

PlayerStats a{"Alice", 100, {90, 95, 100}};
PlayerStats b = a;             // correct memberwise copy
PlayerStats c = std::move(a);  // correct memberwise move
```

This is the best outcome. Use `std::string`, `std::vector`, and smart pointers as members, so the Rule of Zero applies.

0.4.35 The Rule of Three

If you define **any** of: destructor, copy constructor, or copy assignment – define all three.

Why: If you need a custom destructor, your class probably manages a resource. If it manages a resource, the compiler's copy operations (which do a shallow copy of the pointer) are wrong.

```

class Buffer {
    int* data;
    int size;

public:
    Buffer(int n) : data{new int[n]{}}, size{n} {}

    // 1. Destructor: free the resource
    ~Buffer() { delete[] data; }

    // 2. Copy constructor: deep copy
    Buffer(const Buffer& other) : data{new int[other.size]{}}, size{other.size} {
        std::copy(other.data, other.data + size, data);
    }

    // 3. Copy assignment: free old, deep copy new
    Buffer& operator=(const Buffer& other) {
        if (this == &other) return *this; // self-assignment guard
        delete[] data;
        size = other.size;
        data = new int[size]{};
        std::copy(other.data, other.data + size, data);
        return *this;
    }
};

```

Without copy constructor and copy assignment, the compiler generates shallow copies – both objects' data pointer points to the same heap memory. When both destructors run, `delete[]` is called twice on the same pointer. Crash.

0.4.36 The Rule of Five

C++11 added move semantics. If you are defining the Rule of Three, also define the move constructor and move assignment for efficiency: the **Rule of Five**.

```

class Buffer {
    int* data;
    int size;

public:
    Buffer(int n) : data{new int[n]{}}, size{n} {}

    // 1. Destructor
    ~Buffer() { delete[] data; }

    // 2. Copy constructor (deep copy)
    Buffer(const Buffer& other) : data{new int[other.size]{}}, size{other.size} {
        std::copy(other.data, other.data + size, data);
    }

    // 3. Copy assignment (free old, deep copy)

```

```

Buffer& operator=(const Buffer& other) {
    if (this == &other) return *this;
    delete[] data;
    size = other.size;
    data = new int[size]{};
    std::copy(other.data, other.data + size, data);
    return *this;
}

// 4. Move constructor (steal pointer)
Buffer(Buffer&& other) noexcept
    : data{other.data}, size{other.size} {
    other.data = nullptr;
    other.size = 0;
}

// 5. Move assignment (free old, steal pointer)
Buffer& operator=(Buffer&& other) noexcept {
    if (this == &other) return *this;
    delete[] data;
    data = other.data;
    size = other.size;
    other.data = nullptr;
    other.size = 0;
    return *this;
}
};

```

0.4.37 = delete and = default

You can explicitly suppress or request the default implementation:

```

class Unique {
public:
    Unique() = default; // use compiler's default constructor

    Unique(const Unique&) = delete; // copying is forbidden
    Unique& operator=(const Unique&) = delete; // copy assignment is forbidden

    Unique(Unique&&) = default; // move is fine, use default
    Unique& operator=(Unique&&) = default;
};

Unique a;
Unique b = a; // COMPILE ERROR: copy constructor is deleted
Unique c = std::move(a); // OK: move is allowed

```

= delete is how `std::unique_ptr` prevents copying. It causes a clear compile error rather than a silent wrong-copy.

= default explicitly requests the compiler-generated version, which is useful when you need to declare one special member (which inhibits some compiler-generated ones) but still want the defaults for others.

0.4.38 Compiler-Generated Functions Are Suppressed When You Define Others

The rules for when the compiler generates special members are complex. Key interactions:

You define	Compiler suppresses
Destructor	Move constructor, move assignment (still generates copy ops)
Copy constructor	Default constructor, move constructor, move assignment
Copy assignment	Move constructor, move assignment
Move constructor	Copy constructor, copy assignment
Move assignment	Copy constructor, copy assignment

This is why the Rule of Five says: if you define any one, define all five. Defining just a destructor silently disables move semantics, forcing expensive copies everywhere.

0.4.39 The Practical Takeaway

In priority order:

1. **Rule of Zero** (best): use standard containers and smart pointers as members. The compiler generates correct defaults. No special member functions needed.
2. **Rule of Five** (when you must own raw resources): write all five explicitly. This is mainly for implementing containers and RAII wrappers, not typical application code.
3. Never write only a destructor without the others. The compiler's default copy does a shallow copy of raw pointers, leading to double-free crashes.

0.4.40 Common Mistakes in This Chapter

0.4.40.1 Mistake 1: Shallow Copy of Owing Pointer (Forgetting Rule of Three)

The bug:

```

class Buffer {
    int* data;
public:
    Buffer(int n) { data = new int[n]{}; }
    ~Buffer() { delete[] data; }
    // forgot copy constructor and copy assignment
};

Buffer a{10};
Buffer b = a;    // compiler shallow-copies: b.data == a.data (same pointer!)
// both destructors run: double-free crash

```

The fix: Implement copy constructor and copy assignment (or use `= delete` to forbid copying).

0.4.40.2 Mistake 2: Forgetting the Self-Assignment Guard

The bug:

```

Buffer& operator=(const Buffer& other) {
    delete[] data;           // frees data
    size = other.size;
    data = new int[size]{};
    std::copy(other.data, ...); // if other IS *this, other.data was just deleted!
}
// buf = buf; --> crash

```

The fix: if (`this == &other`) return `*this`; at the top of copy assignment.

0.4.41 Exercises

Exercise 16.1 – Rule of Zero or Three?

For each class, say whether the Rule of Zero (no user-defined specials needed) or Rule of Three/Five applies:

```

struct Point { double x, y; }; // (a)
class Socket { int fd; public: Socket(int f):fd{f}{} // (b)
    ~Socket() { close(fd); } };
struct Config { std::string name; std::vector<int> v; }; // (c)
class RawArr { int* p; int n; public: RawArr(int n):p{new int[n]{}},n{n}{}
    ~RawArr(){delete[]p;} }; // (d)

```

Answer: - (a): Rule of Zero – double members, no resource ownership. Compiler generates correct copy/move. - (b): Rule of Five – owns a file descriptor (OS resource). Needs explicit copy/move handling (probably `= delete` for copy, custom move that sets `fd = -1`). - (c): Rule of Zero – `std::string` and `std::vector` manage their own resources. Compiler generates correct defaults. - (d): Rule of Five – owns a raw `int*`. Needs destructor (done), copy constructor (deep copy), copy assignment, move constructor (steal pointer), move assignment.

Exercise 16.2 – Complete the Rule of Five

Complete `RawArr` from the exercise above with all five special members. The copy should make a deep copy; the move should steal the pointer.

Answer:

```
class RawArr {
    int* p;
    int n;
public:
    RawArr(int sz) : p{new int[sz]{}}, n{sz} {}

    ~RawArr() { delete[] p; }

    RawArr(const RawArr& o) : p{new int[o.n]{}}, n{o.n} {
        std::copy(o.p, o.p + n, p);
    }

    RawArr& operator=(const RawArr& o) {
        if (this == &o) return *this;
        delete[] p;
        n = o.n;
        p = new int[n]{};
        std::copy(o.p, o.p + n, p);
        return *this;
    }

    RawArr(RawArr&& o) noexcept : p{o.p}, n{o.n} {
        o.p = nullptr; o.n = 0;
    }

    RawArr& operator=(RawArr&& o) noexcept {
        if (this == &o) return *this;
        delete[] p;
        p = o.p; n = o.n;
        o.p = nullptr; o.n = 0;
        return *this;
    }
};
```

Part III is complete. You now understand the ownership model that makes C++ both safe and fast: RAII ties resource lifetime to object lifetime, smart pointers automate ownership, move semantics avoid unnecessary copies, and the Rule of Zero/Five ensures correct copy and move behavior for your own types.

Part IV covers object-oriented programming – classes, constructors, inheritance, virtual functions, and polymorphism. Ask to continue.

0.5 Part IV – Object-Oriented Programming

C++ OOP looks similar to Python OOP on the surface – both have classes, inheritance, and virtual dispatch. The differences are in where they live (stack vs heap), how they are initialized (constructor member initializer lists), what you must declare explicitly (virtual, override, abstract), and the performance implications of each choice.

Chapter 17: Classes, Objects, and Encapsulation

0.5.1 Python Classes vs C++ Classes

```
# Python class
class Rectangle:
    def __init__(self, width, height):
        self.width = width      # no access control -- everything is public
        self.height = height

    def area(self):
        return self.width * self.height

r = Rectangle(3, 4)
r.width = -5      # Python cannot stop this
print(r.area())   # -20 -- corrupt state
```

```
// C++ class
class Rectangle {
public:                                // public interface
    Rectangle(double w, double h);    // constructor declaration
    double area() const;              // method declaration

private:                              // hidden implementation
    double width;                    // cannot be accessed from outside
    double height;

};
```

The key difference: **access specifiers**. C++ divides class members into:

- `public`: accessible by anyone
- `private`: accessible only by the class's own member functions
- `protected`: accessible by the class and its derived classes (Chapter 19)

In Python everything is public by convention (with `_` as a hint). In C++, `private` is enforced by the compiler.

0.5.2 Defining a Class: Full Example

```
// Rectangle.h
#pragma once
#include <string>

class Rectangle {
public:
    // Constructor
    Rectangle(double width, double height);

    // Const methods (do not modify the object)
    double area() const;
    double perimeter() const;
    std::string describe() const;

    // Getters and setters with validation
    double get_width() const;
    double get_height() const;
    void set_width(double w);
    void set_height(double h);

private:
    double width;
    double height;
};
```

```
// Rectangle.cpp
#include "Rectangle.h"
#include <stdexcept>
#include <string>

// Constructor: ClassName::method_name(params) { body }
Rectangle::Rectangle(double w, double h) {
    if (w <= 0 || h <= 0)
        throw std::invalid_argument("Dimensions must be positive");
    width = w;
    height = h;
}

double Rectangle::area() const { return width * height; }
double Rectangle::perimeter() const { return 2.0 * (width + height); }

std::string Rectangle::describe() const {
    return "Rectangle(" + std::to_string(width) + " x "
           + std::to_string(height) + ")";
}

double Rectangle::get_width() const { return width; }
double Rectangle::get_height() const { return height; }

void Rectangle::set_width(double w) {
    if (w <= 0) throw std::invalid_argument("Width must be positive");
    width = w;
}

void Rectangle::set_height(double h) {
```

```

    if (h <= 0) throw std::invalid_argument("Height must be positive");
    height = h;
}

```

```

// main.cpp
#include <iostream>
#include "Rectangle.h"

int main() {
    Rectangle r{3.0, 4.0};           // calls constructor
    std::cout << r.area()           << "\n"; // 12
    std::cout << r.perimeter()      << "\n"; // 14
    std::cout << r.describe()       << "\n"; // Rectangle(3.000000 x 4.000000)

    r.set_width(5.0);
    std::cout << r.area() << "\n"; // 20

    // r.width = -1; // COMPILE ERROR: 'width' is private
    // r.set_width(-1); // throws std::invalid_argument at runtime
}

```

0.5.3 struct vs class

In C++, `struct` and `class` are almost identical. The only difference: `struct` members are public by default; `class` members are private by default.

```

struct Point {           // members are public by default
    double x, y;
};

class Point2 {           // members are private by default
    double x, y;         // these are private
};

Point p{1.0, 2.0};      // p.x accessible
Point2 q{1.0, 2.0};     // q.x NOT accessible (private)

```

Convention: - Use `struct` for simple data aggregates with no invariants to protect (Point, Color, Size) - Use `class` when you need access control and invariants (Rectangle must have positive dimensions)

0.5.4 Memory Layout of a Class

```
class Vec2 {
    float x;    // 4 bytes
    float y;    // 4 bytes
};
```

```
Vec2 v{1.0f, 2.0f};
```

Stack (or wherever `v` lives):

Address	Member	Bytes
0x1000	x (float)	[00][00][80][3F] <- 1.0f in IEEE 754
0x1004	y (float)	[00][00][00][40] <- 2.0f in IEEE 754

`sizeof(Vec2) == 8` (same as two separate floats)

There is no hidden per-object overhead from member functions. Functions exist once in the `.text` section of the executable. The object just holds data.

Code (compiled once, shared by all `Vec2` instances):

```
.text:
Vec2::area():
    mov eax, [this+0]    <- reads this->x
    ...
```

The `this` pointer (implicit first parameter of every member function) tells the function which object to operate on.

0.5.5 `this` Pointer

Inside any non-static member function, `this` is a pointer to the current object:

```
class Counter {
    int count{0};
public:
    Counter& increment() {
        ++count;    // same as: ++(this->count)
        return *this; // return reference to self (enables chaining)
    }
    int get() const { return count; }
};

Counter c;
c.increment().increment().increment(); // method chaining
std::cout << c.get(); // 3
```

`return *this` returns a reference to the object itself, allowing `.method()` to be called immediately on the result. This pattern (fluent interface / method chaining) is common in builder classes.

0.5.6 Static Members

static data members and methods belong to the **class**, not to any individual object:

```
class Player {
    std::string name;
    static int player_count;    // shared across ALL Player objects

public:
    Player(std::string n) : name{n} { ++player_count; }
    ~Player()                { --player_count; }

    static int get_count() { return player_count; }
    // static methods have no 'this' -- cannot access non-static members
};

int Player::player_count = 0;    // definition outside the class (required)

Player p1{"Alice"};
Player p2{"Bob"};
std::cout << Player::get_count() << "\n";    // 2 (call via class name)
{
    Player p3{"Carol"};
    std::cout << Player::get_count() << "\n";    // 3
}    // p3 destroyed
std::cout << Player::get_count() << "\n";    // 2
```

0.5.7 Common Mistakes in This Chapter

0.5.7.1 Mistake 1: Forgetting the :: Scope Resolution in the .cpp File

The bug:

```
// Rectangle.cpp
double area() const {    // ERROR: this defines a FREE function named area,
    return width * height;    // not Rectangle::area. 'width' is not in scope.
}
```

The fix: `double Rectangle::area() const { return width * height; }`

0.5.7.2 Mistake 2: Calling a Non-const Method on a Const Object

The bug:


```
const Rectangle r{3.0, 4.0};
r.set_width(5.0); // ERROR: set_width is non-const (it modifies the object)
```

Compiler message:

error: passing 'const Rectangle' as 'this' argument discards qualifiers

The fix: Either remove const from the object, or don't call mutating methods on const objects.

0.5.7.3 Mistake 3: Forgetting to Define Static Members

The bug:

```
class Foo { static int count; };
// Forgot: int Foo::count = 0;
// Linker error: undefined reference to 'Foo::count'
```

The fix: Static data members must be defined exactly once in a .cpp file.

0.5.8 Exercises

Exercise 17.1 – Design a class

Design a BankAccount class with: - Private balance (double, starts at 0) - deposit(double amount) – adds to balance (reject negative amounts) - withdraw(double amount) – subtracts (reject if insufficient funds) - get_balance() const – returns balance

Answer:

```
class BankAccount {
    double balance{0.0};
public:
    void deposit(double amount) {
        if (amount <= 0) throw std::invalid_argument("Amount must be positive");
        balance += amount;
    }
    void withdraw(double amount) {
        if (amount <= 0) throw std::invalid_argument("Amount must be positive");
        if (amount > balance) throw std::runtime_error("Insufficient funds");
        balance -= amount;
    }
    double get_balance() const { return balance; }
};
```

Exercise 17.2 – Const correctness

Which of these methods should be const?

```
class Circle {
    double radius;
public:
    void    set_radius(double r) { radius = r; }
    double  get_radius()        { return radius; }
    double  area()               { return 3.14159 * radius * radius; }
    bool    is_unit_circle()     { return radius == 1.0; }
};
```

Answer: `get_radius()`, `area()`, and `is_unit_circle()` should all be const – none of them modify `radius`. `set_radius()` cannot be const because it assigns to `radius`.

Chapter 18: Constructors, Destructors, and Initialization

0.5.9 The Member Initializer List

C++ constructors have a special syntax for initializing members **before the constructor body runs**. This is the **member initializer list**, the `:` clause between the parameter list and `{`:

```
class Particle {
    double x, y;
    double vx, vy;
    std::string name;

public:
    // WITHOUT member initializer list (suboptimal):
    Particle(double px, double py, std::string n) {
        x    = px;    // assignment (not initialization)
        y    = py;    // members are default-constructed first, then assigned
        vx    = 0.0;
        vy    = 0.0;
        name = n;    // name is default-constructed to "" then copy-assigned
    }

    // WITH member initializer list (correct and efficient):
    Particle(double px, double py, std::string n)
        : x{px}, y{py}, vx{0.0}, vy{0.0}, name{std::move(n)}
    {    // body runs after all members are initialized
    }
};
```

The member initializer list directly constructs each member with the given value. The assignment version first default-constructs each member, then overwrites it – two operations instead of one.

For `std::string`, `std::vector`, and any class type, always use the initializer list. For const members and reference members, the initializer list is not optional – they **cannot** be assigned after construction:

```
class Fixed {
    const int id;
    int& ref;
public:
    Fixed(int i, int& r) : id{i}, ref{r} {} // MUST use initializer list
    // id = i; // COMPILE ERROR: cannot assign to const member
};
```

0.5.9.1 Initialization Order

Members are initialized in the **order they are declared in the class**, not the order they appear in the initializer list:

```
class Tricky {
    int a;
    int b;
public:
    Tricky(int x) : b{x}, a{b} // WARNING: a is initialized before b!
    {}
    // a is initialized with b's value, but b hasn't been initialized yet
    // 'a' gets garbage
};
```

Rule: Put initializers in the same order as the declarations. Most compilers warn about order mismatches with `-Wall`.

0.5.10 Delegating Constructors (C++11)

One constructor can call another constructor of the same class:

```
class Vec3 {
    double x, y, z;
public:
    Vec3(double x, double y, double z) : x{x}, y{y}, z{z} {}

    // Delegate to the main constructor:
    Vec3() : Vec3{0.0, 0.0, 0.0} {} // zero vector
    Vec3(double uniform) : Vec3{uniform, uniform, uniform} {}
};

Vec3 origin; // (0, 0, 0)
Vec3 ones{1.0}; // (1, 1, 1)
Vec3 point{1.0, 2.0, 3.0};
```

0.5.11 Constructor Kinds

0.5.11.1 Default Constructor

Called when no arguments are provided:

```
class Foo {
public:
    Foo() { std::cout << "default constructed\n"; }
};
Foo f;           // calls Foo()
Foo g{};         // also calls Foo()
std::vector<Foo> v(5); // calls Foo() five times
```

If you declare **any** constructor, the compiler no longer generates a default constructor automatically. Add `Foo() = default;` to get it back.

0.5.11.2 Explicit Constructor (Prevent Implicit Conversion)

```
class Radius {
    double value;
public:
    Radius(double v) : value{v} {} // implicit: Radius r = 5.0; works
};

class SafeRadius {
    double value;
public:
    explicit SafeRadius(double v) : value{v} {} // explicit: no implicit conversion
};

Radius    r1 = 5.0;           // OK: implicit conversion double -> Radius
SafeRadius r2 = 5.0;          // ERROR: explicit constructor cannot convert implicitly
SafeRadius r3{5.0};          // OK: direct initialization always works
SafeRadius r4(5.0);           // OK: direct initialization

void process(Radius r) {}
process(5.0);                  // OK for Radius (implicit conversion allowed)
process(SafeRadius{5.0});      // OK: explicit construction then pass
```

Use `explicit` for single-argument constructors to prevent surprising implicit conversions. `std::vector(int)` is `explicit`—`std::vector<int> v = 5;` would be confusing.

0.5.12 Destructor Revisited: The Full Rules

```
class Resource {
    int* data;
public:
    Resource() : data{new int[100]{} } { std::cout << "acquired\n"; }
    ~Resource() { delete[] data; std::cout << "released\n"; }
};
```

Destructors are called: 1. **When a local variable goes out of scope** (stack unwinding) 2. **When delete is called** on a heap-allocated object 3. **When a container element is removed** (vector::pop_back, etc.) 4. **When a member variable's owner is destroyed** (member destructors run after the owner's destructor body)

The destructor call order for members is the **reverse of construction order**.

0.5.13 In-Class Member Initializers (C++11)

Members can have default values directly in the class definition:

```
class Config {
    int width{1920}; // default: 1920
    int height{1080}; // default: 1080
    bool fullscreen{false}; // default: false
    std::string title{"My App"};

public:
    Config() = default; // uses all defaults above

    Config(int w, int h) // overrides width and height, keeps other defaults
        : width{w}, height{h} {}
};

Config default_cfg; // 1920x1080, windowed, "My App"
Config custom{800, 600}; // 800x600, windowed, "My App"
```

This is the cleanest way to set defaults. Prefer it over setting values in the constructor body.

0.5.14 Common Mistakes in This Chapter

0.5.14.1 Mistake 1: Initializer List Order Different From Declaration Order

The bug:

```
class Pair {
    int second;    // declared first
    int first;     // declared second
public:
    Pair(int f, int s) : first{f}, second{s} {}
    // first is initialized with f -- but second is initialized FIRST (declaration order)
    // and second{s} is fine since s is not dependent on first
    // If instead: Pair(int x) : first{x}, second{first*2}{}
    // then second = first*2 but first hasn't been initialized yet!
};
```

The fix: Keep initializer list order matching declaration order.

0.5.14.2 Mistake 2: Missing `explicit` on Converting Constructor

The bug:

```
class Seconds { double val; public: Seconds(double v):val{v}{} };
void wait(Seconds s) { ... }
wait(100);    // Did you mean 100 seconds? Or was 100 an int you forgot to convert?
```

The fix: `explicit Seconds(double v)` forces `wait(Seconds{100})` – the intent is clear.

0.5.15 Exercises

Exercise 18.1 – Rewrite with initializer list

Rewrite this constructor to use a member initializer list:

```
class Player {
    std::string name;
    int hp;
    int max_hp;
public:
    Player(std::string n, int health) {
        name = n;
        hp = health;
        max_hp = health;
    }
};
```

Answer:

```
Player(std::string n, int health)
    : name{std::move(n)}, hp{health}, max_hp{health} {}
```

`std::move(n)` avoids copying the string (since `n` is a value parameter, it's a local copy we can move from).

Exercise 18.2 – Delegating constructors

Write a `Color` class with `r`, `g`, `b` components (0.0-1.0 doubles). Provide: - A full 3-argument constructor - A grayscale constructor taking one value (sets `r=g=b`) - A default constructor (black: 0,0,0)

Use delegating constructors.

Answer:

```
class Color {
    double r, g, b;
public:
    Color(double r, double g, double b) : r{r}, g{g}, b{b} {}
    Color(double gray) : Color{gray, gray, gray} {}
    Color() : Color{0.0} {}
};
```

Chapter 19: Inheritance and Composition

0.5.16 Two Kinds of Reuse

When one class needs functionality from another, you have two options:

- **Inheritance (“is-a”)**: Dog is-a Animal. Dog inherits from Animal.
- **Composition (“has-a”)**: Car has-a Engine. Car has an Engine as a member.

Python programmers often overuse inheritance. Prefer composition in C++ (and generally): it is more flexible and avoids the tight coupling that inheritance creates.

0.5.17 Inheritance Syntax

```
# Python
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."
```

```
class Dog(Animal):
    def speak(self):
        return "Woof"
```

```
// C++
class Animal {
protected:           // accessible by Animal and derived classes
    std::string name;
public:
    Animal(const std::string& n) : name{n} {}
    std::string get_name() const { return name; }
    virtual std::string speak() const { return "..."; }
    virtual ~Animal() {} // virtual destructor -- explained in Chapter 20
};

class Dog : public Animal { // Dog inherits publicly from Animal
public:
    Dog(const std::string& n) : Animal{n} {} // must call base constructor

    std::string speak() const override { return "Woof"; }
    //          ^--- override keyword (C++11): verifies this
    //          actually overrides a virtual base method
};
```

0.5.17.1 The public Inheritance Keyword

class Dog : public Animal – the public here controls how the base class’s access specifiers are inherited:

Inheritance type	public becomes	protected becomes	private becomes
public	public	protected	inaccessible
protected	protected	protected	inaccessible
private	private	private	inaccessible

Almost always use public inheritance. private and protected inheritance are rare, specialized tools.

0.5.18 Constructors in Derived Classes

A derived class constructor **must** call the base class constructor:

```
class Shape {
    std::string color;
public:
    Shape(const std::string& c) : color{c} {}
    std::string get_color() const { return color; }
};

class Circle : public Shape {
    double radius;
```



```
public:
    Circle(double r, const std::string& c)
        : Shape{c}          // must explicitly call base constructor
        , radius{r}         // then initialize own members
    {}
    double area() const { return 3.14159 * radius * radius; }
};
```

If the base class has no default constructor, the derived class **must** explicitly call a base constructor in its initializer list. The base part is always constructed first, before the derived members.

0.5.19 Memory Layout With Inheritance

```
class Base {
    int x;    // 4 bytes
    int y;    // 4 bytes
};

class Derived : public Base {
    int z;    // 4 bytes
};

Derived d;
```

Memory layout of d (Derived):

Address	Member	Size	
0x1000	x (Base)	4 bytes	<- Base part comes first
0x1004	y (Base)	4 bytes	
0x1008	z (Derived)	4 bytes	<- Derived's own members after

`sizeof(Derived) == 12`

The base class subobject lives at the beginning of the derived object's memory. A pointer to `Derived` is also a valid pointer to `Base` (just points to the same address – the `Base` subobject is right there).

0.5.20 Slicing: The Object-Oriented Trap

```
Animal a = Dog{"Rex"};    // BAD: Dog is sliced into an Animal!
a.speak();                // calls Animal::speak, not Dog::speak -- "..."
```

When you copy a `Dog` into an `Animal` variable, only the `Animal` part is copied. The `Dog`-specific data is sliced off. The object is now truly just an `Animal`.

Before slice:		After slice:
+-----+		+-----+
Dog		Animal
name: "Rex"	-->	name: "Rex"
(Dog data)		(Dog data GONE)
+-----+		+-----+

To avoid slicing, use pointers or references to the base class:

```
// Correct: pointer to Animal, no slicing
std::unique_ptr<Animal> a = std::make_unique<Dog>("Rex");
a->speak(); // "Woof" -- polymorphic dispatch (Chapter 20)

// Correct: reference to Animal, no slicing
Dog dog{"Rex"};
Animal& ref = dog;
ref.speak(); // "Woof" -- polymorphic dispatch
```

0.5.21 Composition vs Inheritance

Most “is-a” relationships in the real world are actually “can-do” or “has-a”. Prefer composition:

```
// Inheritance: Engine IS-A Vehicle? No. Engine is part of Vehicle.
class Engine { void start(); void stop(); };
class CarBad : public Engine {}; // wrong: Car is NOT an Engine

// Composition: Car HAS-A Engine
class CarGood {
    Engine engine; // Engine is a component
public:
    void start() { engine.start(); }
};
```

Prefer composition when: - You need the functionality of another class but are not that class - You want to change the implementation later without affecting callers - You want to combine multiple behaviors (C++ has single inheritance)

Use inheritance when: - There is a genuine “is-a” relationship - You need polymorphic dispatch (Chapter 20) through a common base pointer

0.5.22 Common Mistakes in This Chapter

0.5.22.1 Mistake 1: Forgetting to Call the Base Constructor

The bug:

```
class Derived : public Base {
public:
    Derived(int x) { ... } // Base's constructor never called!
    // If Base has no default constructor: compile error
    // If Base has a default constructor: it is called, possibly not what you want
};
```

The fix: `Derived(int x) : Base{...} { ... }`

0.5.22.2 Mistake 2: Object Slicing

The bug:

```
void process(Animal a) { a.speak(); } // takes by value -- slices!
process(Dog{"Rex"}); // Dog is sliced to Animal; speak() returns "..."
```

The fix: `void process(const Animal& a)` or `void process(Animal* a)` – reference/pointer avoids slicing.

0.5.23 Exercises

Exercise 19.1 – Class hierarchy

Design a small hierarchy: `Shape` (base), `Rectangle` and `Circle` (derived). Each should have `area()` `const` and `perimeter()` `const`. Use composition or inheritance as appropriate.

Answer (abbreviated):

```
class Shape {
public:
    virtual double area() const = 0; // pure virtual (Chapter 21)
    virtual double perimeter() const = 0;
    virtual ~Shape() {}
};

class Rectangle : public Shape {
    double w, h;
public:
    Rectangle(double w, double h) : w{w}, h{h} {}
    double area() const override { return w * h; }
    double perimeter() const override { return 2*(w+h); }
```

```
};

class Circle : public Shape {
    double r;
public:
    Circle(double r) : r{r} {}
    double area() const override { return 3.14159265 * r * r; }
    double perimeter() const override { return 2 * 3.14159265 * r; }
};
```

Chapter 20: Virtual Functions and Polymorphism

0.5.24 The Problem Without Virtual

```
class Animal {
public:
    std::string speak() const { return "..."; } // not virtual
};

class Dog : public Animal {
public:
    std::string speak() const { return "Woof"; } // hides, not overrides
};

Animal* a = new Dog{};
std::cout << a->speak(); // "..." -- calls Animal::speak, not Dog::speak!
```

Without `virtual`, the function called is determined by the **type of the pointer** at compile time. `a` is `Animal*`, so `Animal::speak` is called. The fact that the object is actually a `Dog` is ignored.

This is called **static dispatch** (compile-time resolution).

0.5.25 `virtual` Enables Dynamic Dispatch

Adding `virtual` tells the compiler to resolve the call at **runtime** based on the actual type of the object:

```
class Animal {
public:
    virtual std::string speak() const { return "..."; }
    virtual ~Animal() {} // always virtual destructor in polymorphic base classes
};

class Dog : public Animal {
public:
    std::string speak() const override { return "Woof"; }
```

```

};

class Cat : public Animal {
public:
    std::string speak() const override { return "Meow"; }
};

Animal* a1 = new Dog{};
Animal* a2 = new Cat{};
std::cout << a1->speak() << "\n"; // "Woof" -- dynamic dispatch to Dog::speak
std::cout << a2->speak() << "\n"; // "Meow" -- dynamic dispatch to Cat::speak
delete a1;
delete a2;

```

The same pointer type (`Animal*`), the same call (`->speak()`), different behavior based on what the pointer actually points to. This is **runtime polymorphism**.

0.5.26 How Virtual Dispatch Works: The vtable

For each class with virtual functions, the compiler creates a **vtable** (virtual function table): an array of function pointers, one per virtual function.

Each object of a polymorphic class has a hidden **vp_{tr}** (vtable pointer) pointing to its class's vtable:

Memory layout of a Dog object (with virtual `speak`):

```

+-----+
| vptr -----> Dog's vtable:
|           | [0] --> Dog::speak()
| name (string) | [1] --> Dog::~Dog()
+-----+

```

Memory layout of a Cat object:

```

+-----+
| vptr -----> Cat's vtable:
|           | [0] --> Cat::speak()
| name (string) | [1] --> Cat::~Cat()
+-----+

```

When you call `a->speak()`: 1. Load `a` (the pointer) 2. Follow `a` to the object 3. Load `vptr` from the object (first bytes of the object) 4. Index into the vtable: `vptr[0]` 5. Call the function pointer found there

The overhead: one pointer dereference and one indirect call. Tiny but non-zero. For hot inner loops, this matters (Chapter 38 covers alternatives).

0.5.27 override and final (C++11)

`override` tells the compiler to verify that this function actually overrides a virtual base function. Without it, a typo silently creates a new function instead:

```
class Animal {
    virtual std::string speak() const { return "..."; }
};

class Dog : public Animal {
    std::string speak() const { return "Woof"; } // typo! No error without override.
    // Does not override speak(). Dog::speak is a new function.
    // Animal::speak is still called for Dog objects via base pointer.
};

class DogCorrect : public Animal {
    std::string speak() const override { return "Woof"; }
    // ERROR: 'speak' does not override any virtual function
    // Compiler catches the typo.
};
```

Always use `override`. No downside, catches bugs.

`final` prevents further overriding (for a method) or prevents inheritance (for a class):

```
class SafeAnimal : public Animal {
    std::string speak() const override final { return "safe"; }
    // no derived class can override speak() anymore
};

class Terminal final : public Animal { ... };
// class Sub : public Terminal {}; // ERROR: Terminal is final
```

0.5.28 Virtual Destructor: Why It Is Required

```
class Base {
public:
    ~Base() { std::cout << "Base destroyed\n"; } // NOT virtual
};

class Derived : public Base {
    int* data;
public:
    Derived() : data{new int[100]} {}
    ~Derived() { delete[] data; std::cout << "Derived destroyed\n"; }
};

Base* p = new Derived{};
delete p; // calls Base::~~Base ONLY -- Derived::~~Derived never called!
          // 'data' is leaked every time
```

When you delete through a base class pointer and the destructor is not virtual, only the base class destructor is called. The derived class's destructor (and its cleanup code) is bypassed.

```
class Base {
public:
    virtual ~Base() { std::cout << "Base destroyed\n"; } // virtual
};
// Now: delete p; calls Derived::~Derived first (via vtable), then Base::~Base
```

Rule: If a class has any virtual functions, make its destructor virtual. The overhead is one vtable slot – negligible.

0.5.29 Polymorphism With unique_ptr

Modern C++ polymorphism uses smart pointers, not raw pointers:

```
#include <memory>
#include <vector>

std::vector<std::unique_ptr<Animal>> animals;
animals.push_back(std::make_unique<Dog>("Rex"));
animals.push_back(std::make_unique<Cat>("Whiskers"));
animals.push_back(std::make_unique<Dog>("Buddy"));

for (const auto& a : animals) {
    std::cout << a->get_name() << " says: " << a->speak() << "\n";
}
// Rex says: Woof
// Whiskers says: Meow
// Buddy says: Woof
// All Animals deleted automatically when vector is destroyed
```

No manual delete. No memory leaks. Full polymorphism.

0.5.30 Python Polymorphism vs C++ Polymorphism

Python uses **duck typing**: no inheritance required. Any object with a `speak()` method works.

```
class Dog: def speak(self): return "Woof"
class Cat: def speak(self): return "Meow"
class Rock: pass # no speak

animals = [Dog(), Cat()]
for a in animals:
    print(a.speak()) # works because both have speak()
```

C++ polymorphism requires: 1. A common base class with `virtual` functions 2. The object accessed through a base pointer or reference 3. `override` in derived classes

C++ offers **static polymorphism** via templates (Chapter 23/25) for zero-overhead duck typing.

0.5.31 Common Mistakes in This Chapter

0.5.31.1 Mistake 1: Non-Virtual Base Destructor

Already covered above. Always make the destructor `virtual` in a polymorphic base class.

0.5.31.2 Mistake 2: Forgetting `override`

The bug:

```
class Derived : public Base {
    void render() { ... } // Base has: virtual void Render() { ... }
    // typo: render vs Render -- silent new function, not an override
};
```

The fix: Always write `override`. The compiler will tell you if the name doesn't match.

0.5.31.3 Mistake 3: Calling Virtual Functions in Constructors/Destructors

The bug:

```
class Base {
public:
    Base() { initialize(); } // calls virtual -- WRONG
    virtual void initialize() { std::cout << "Base\n"; }
};
class Derived : public Base {
public:
    void initialize() override { std::cout << "Derived\n"; }
};
Derived d; // prints "Base", not "Derived" -- vtable not fully set up yet
```

Why: During the base constructor, the object is still a Base. The vtable points to Base's functions. Dynamic dispatch is suspended during construction and destruction. **The fix:** Never call virtual functions in constructors or destructors. Use a factory function or a two-phase initialization if needed.

0.5.32 Exercises

Exercise 20.1 – Virtual dispatch trace

```
struct A {
    virtual void f() { std::cout << "A\n"; }
    void g()         { std::cout << "A::g\n"; }
};
struct B : A {
    void f() override { std::cout << "B\n"; }
    void g()         { std::cout << "B::g\n"; }
};
A* p = new B{};
p->f();    // (a)
p->g();    // (b)
B* q = static_cast<B*>(p);
q->f();    // (c)
q->g();    // (d)
delete p;
```

Answer: - (a): B – f() is virtual, dynamic dispatch to B::f - (b): A::g – g() is not virtual, compile-time resolution via A* - (c): B – f() is virtual, q is B*, dispatch to B::f - (d): B::g – g() is not virtual, compile-time resolution via B*

Exercise 20.2 – Polymorphic zoo

Create a vector of `unique_ptr<Animal>`, add at least 3 different animal types, and print each one's `speak()` result using a range-based loop.

Answer:

```
#include <memory>
#include <vector>
#include <iostream>

// Using the Animal/Dog/Cat hierarchy from this chapter:
std::vector<std::unique_ptr<Animal>> zoo;
zoo.push_back(std::make_unique<Dog>("Rex"));
zoo.push_back(std::make_unique<Cat>("Whiskers"));
zoo.push_back(std::make_unique<Dog>("Buddy"));

for (const auto& animal : zoo) {
    std::cout << animal->get_name() << ": " << animal->speak() << "\n";
}
```

0.5.33 Pure Virtual Functions

A **pure virtual function** is a virtual function declared with `= 0`. It has no implementation in the base class. Any class with at least one pure virtual function is an **abstract class** – it cannot be instantiated.

```
class Shape {
public:
    virtual double area()      const = 0; // pure virtual
    virtual double perimeter() const = 0; // pure virtual
    virtual void draw()        const = 0; // pure virtual
    virtual ~Shape() {}
};

Shape s; // COMPILER ERROR: cannot instantiate abstract class 'Shape'
```

A class is abstract to express: “I define the interface; subclasses provide the implementation.”

0.5.34 Concrete Derived Classes

A derived class that provides implementations for all pure virtual functions is **concrete** (can be instantiated):

```
class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius{r} {}

    double area()      const override { return 3.14159265 * radius * radius; }
    double perimeter() const override { return 2 * 3.14159265 * radius; }
    void draw()        const override { std::cout << "O\n"; }
};

class Square : public Shape {
    double side;
public:
    Square(double s) : side{s} {}

    double area()      const override { return side * side; }
    double perimeter() const override { return 4 * side; }
    void draw()        const override { std::cout << "[\n"; }
};

// Now concrete:
Circle c{5.0};
Square s{3.0};
std::cout << c.area() << "\n"; // 78.5...
std::cout << s.area() << "\n"; // 9
```

If a derived class leaves any pure virtual functions unimplemented, it is also abstract:

```
class PartialShape : public Shape {
    double area() const override { return 0; }
    // perimeter() and draw() not implemented -- PartialShape is still abstract
};
PartialShape p;    // COMPILE ERROR: still abstract
```

0.5.35 Interfaces in C++

C++ does not have a built-in interface keyword (unlike Java or C#). An **interface** is a convention: an abstract class with only pure virtual functions and a virtual destructor – no data members, no implementation.

```
# Python "interface" via ABC
from abc import ABC, abstractmethod

class Drawable(ABC):
    @abstractmethod
    def draw(self) -> None: ...

    @abstractmethod
    def bounding_box(self) -> tuple: ...
```

```
// C++ interface convention
class IDrawable {
public:
    virtual void draw() const = 0;
    virtual std::pair<double,double> bounding_box() const = 0;
    virtual ~IDrawable() {}
};

class ISerializable {
public:
    virtual std::string serialize() const = 0;
    virtual void deserialize(const std::string&) = 0;
    virtual ~ISerializable() {}
};

// A class can implement multiple interfaces:
class Sprite : public IDrawable, public ISerializable {
    // must implement all pure virtuals from both interfaces
    void draw() const override { ... }
    std::pair<double,double> bounding_box() const override { ... }
    std::string serialize() const override { ... }
    void deserialize(const std::string&) override { ... }
};
```

Multiple inheritance from multiple **pure-interface** base classes is safe and common. Multiple inheritance from classes with data members is tricky (diamond problem) and usually avoided.

0.5.36 Partial Implementations (Partially Abstract)

A class can implement some pure virtuals and leave others:

```
class AbstractRenderer : public IDrawable {
public:
    // Provides a default implementation for draw:
    void draw() const override {
        set_up_context();
        do_draw();           // calls another pure virtual -- template method pattern
        tear_down_context();
    }
    // Subclasses must provide do_draw():
    virtual void do_draw() const = 0;
protected:
    void set_up_context() const { ... }
    void tear_down_context() const { ... }
};

class OpenGLRenderer : public AbstractRenderer {
    void do_draw() const override { ... } // only override what is necessary
};
```

This is the **Template Method** pattern: the base class defines the algorithm skeleton, derived classes fill in the specific steps.

0.5.37 Checking the Type at Runtime: `dynamic_cast`

When you have a base class pointer and need to find out the real type:

```
void process(Shape* s) {
    if (Circle* c = dynamic_cast<Circle*>(s)) {
        // s points to a Circle (or derived from Circle)
        std::cout << "radius: " << c->radius << "\n";
    } else if (Square* sq = dynamic_cast<Square*>(s)) {
        std::cout << "side: " << sq->side << "\n";
    }
}
```

`dynamic_cast<Circle*>(s)` returns a `Circle*` if `s` actually points to a `Circle`, or `nullptr` otherwise. It works via the vtable (requires at least one virtual function).

Caveat: Frequent `dynamic_cast` is usually a design smell. It means the code is asking “what type is this?” at runtime instead of using virtual dispatch. Prefer redesigning with virtual functions. `dynamic_cast` is legitimate for occasional introspection or when interacting with external code.

0.5.38 Common Mistakes in This Chapter

0.5.38.1 Mistake 1: Instantiating an Abstract Class

The bug:

```
Shape s{}; // or: Shape* p = new Shape{};
```

Compiler error:

```
error: cannot declare variable 's' to be of abstract type 'Shape'
note: because the following virtual functions are pure within 'Shape':
note: virtual double Shape::area() const
```

The fix: Instantiate a concrete derived class, not the abstract base.

0.5.38.2 Mistake 2: Missing `override` in Concrete Class

The bug:

```
class Rect : public Shape {
    double area() const { return w * h; } // forgot override
    // If Shape::area() signature changes, this silently stops overriding it
};
```

The fix: Always write `override`. If the signature in the base changes, the compiler will tell you.

0.5.39 Exercises

Exercise 21.1 – Design an interface

Design a `ILogger` interface with methods `log_info(std::string)`, `log_warning(std::string)`, `log_error(std::string)`. Then write two implementations: `ConsoleLogger` (prints to cout) and `NullLogger` (does nothing – useful for disabling logging in tests).

Answer:

```
class ILogger {
public:
    virtual void log_info(const std::string& msg) = 0;
    virtual void log_warning(const std::string& msg) = 0;
    virtual void log_error(const std::string& msg) = 0;
    virtual ~ILogger() {}
};
```

```

class ConsoleLogger : public ILogger {
public:
    void log_info(const std::string& msg)    override {
        std::cout << "[INFO] " << msg << "\n"; }
    void log_warning(const std::string& msg) override {
        std::cout << "[WARN] " << msg << "\n"; }
    void log_error(const std::string& msg)  override {
        std::cout << "[ERROR] " << msg << "\n"; }
};

class NullLogger : public ILogger {
public:
    void log_info(const std::string&)    override {}
    void log_warning(const std::string&) override {}
    void log_error(const std::string&)  override {}
};

// Usage: inject the logger as a pointer to interface
void do_work(ILogger& log) {
    log.log_info("Starting work");
    // ...
    log.log_warning("Something unusual");
}

ConsoleLogger cl;
do_work(cl);    // prints

NullLogger nl;
do_work(nl);    // prints nothing (testing/silent mode)

```

Chapter 22: Operator Overloading

0.5.40 What Operator Overloading Is

C++ lets you define what the built-in operators (+, -, ==, <, <<, [], etc.) do for your own types. This is called **operator overloading**.

Python calls these **dunder methods** (double underscore):

```

class Vec2:
    def __init__(self, x, y): self.x, self.y = x, y
    def __add__(self, other): return Vec2(self.x+other.x, self.y+other.y)
    def __repr__(self):      return f"Vec2({self.x}, {self.y})"

a = Vec2(1, 2)
b = Vec2(3, 4)
print(a + b)    # Vec2(4, 6)

```

```

struct Vec2 {
    double x, y;

    Vec2 operator+(const Vec2& other) const {
        return Vec2{x + other.x, y + other.y};
    }
};

// Non-member: output stream operator
std::ostream& operator<<(std::ostream& os, const Vec2& v) {
    return os << "Vec2(" << v.x << ", " << v.y << ")";
}

Vec2 a{1.0, 2.0};
Vec2 b{3.0, 4.0};
Vec2 c = a + b;
std::cout << c << "\n";    // Vec2(4, 6)

```

0.5.41 Which Operators to Overload

Not all operators make sense for all types. Overload only what has a clear, unsurprising meaning for your type.

```

struct Vec2 {
    double x, y;

    // Arithmetic (member: left operand is *this)
    Vec2 operator+(const Vec2& o) const { return {x+o.x, y+o.y}; }
    Vec2 operator-(const Vec2& o) const { return {x-o.x, y-o.y}; }
    Vec2 operator*(double s) const { return {x*s, y*s}; }
    Vec2 operator-() const { return {-x, -y}; } // unary minus

    // Compound assignment
    Vec2& operator+=(const Vec2& o) { x+=o.x; y+=o.y; return *this; }
    Vec2& operator-=(const Vec2& o) { x-=o.x; y-=o.y; return *this; }
    Vec2& operator*=(double s) { x*=s; y*=s; return *this; }

    // Comparison (C++20 spaceship operator generates all 6 at once)
    bool operator==(const Vec2& o) const { return x==o.x && y==o.y; }
    bool operator!=(const Vec2& o) const { return !(*this == o); }

    // Array subscript
    double& operator[(int i)] {
        if (i == 0) return x;
        if (i == 1) return y;
        throw std::out_of_range{"Vec2 index out of range"};
    }
    const double& operator[(int i)] const { // const version
        if (i == 0) return x;
        if (i == 1) return y;
        throw std::out_of_range{"Vec2 index out of range"};
    }
}

```

```
};

// Non-member: allows s * v as well as v * s
Vec2 operator*(double s, const Vec2& v) { return v * s; }

// Non-member: stream output (cannot be member because left operand is ostream)
std::ostream& operator<<(std::ostream& os, const Vec2& v) {
    return os << "(" << v.x << ", " << v.y << ")";
}
```

Usage:

```
Vec2 a{1.0, 2.0};
Vec2 b{3.0, 4.0};
Vec2 c = a + b;           // (4, 6)
Vec2 d = 2.0 * a;         // (2, 4)
Vec2 e = -a;              // (-1, -2)
a += b;                   // a = (4, 6)
std::cout << c << "\n"; // (4, 6)
std::cout << c[0] << "\n"; // 4
```

0.5.42 Member vs Non-Member Operators

Some operators must be members (`=`, `[]`, `()`, `->`). Others should be non-members when the left operand may not be your type.

Rule of thumb: - If the operator modifies the object (`+=`, `-=`, etc.): **member** - If the left operand might not be your type (`<<`, `*`, `+` with mixed types): **non-member** - Symmetric binary operators (`+`, `-`, `==`): **non-member** (for symmetry)

For `operator<<` specifically: the left operand is `std::ostream`, which is not your type. You cannot add a member to `std::ostream`. Therefore `operator<<` is always a non-member.

0.5.43 The Spaceship Operator (C++20)

C++20 introduced `operator<=>` (the three-way comparison operator), which generates all six comparison operators at once:

```
#include <compare>

struct Point {
    double x, y;

    auto operator<=>(const Point& o) const = default;
    // Generates: ==, !=, <, <=, >, >= all with memberwise comparison
```



```
};

Point a{1.0, 2.0};
Point b{1.0, 3.0};
bool less = (a < b);    // true: a.x == b.x, a.y < b.y
bool eq   = (a == a);   // true
```

= default tells the compiler to generate memberwise comparison in declaration order. For most value types, this is exactly right.

0.5.44 operator() – Making Objects Callable

```
struct Adder {
    int n;
    Adder(int n) : n{n} {}
    int operator()(int x) const { return x + n; }
};

Adder add5{5};
std::cout << add5(10) << "\n";    // 15
std::cout << add5(20) << "\n";    // 25
```

An object with operator() is called a **functor** or **function object**. They can store state (unlike raw function pointers) and are the foundation of lambdas (Chapter 30).

0.5.45 Rules and Guidelines for Operator Overloading

DO:

- Overload only when the semantics are obvious and unsurprising
- Keep operators consistent with each other (if + is defined, += should be too)
- Make operators non-throwing when possible
- Return *this from compound-assignment operators (enables a += b += c)

DON'T:

- Overload operators for unrelated meanings (e.g., using << for bit-shift on a list)
 - Overload &&, ||, or comma (,) -- they lose short-circuit/sequencing guarantees
 - Make operator+ modify the left operand (it should return a new value)
 - Overload more operators than your type needs
-

0.5.46 Common Mistakes in This Chapter

0.5.46.1 Mistake 1: Returning `*this` by Value Instead of Reference From `+=`

The bug:

```
Vec2 operator+=(const Vec2& o) { // returns by value
    x += o.x; y += o.y;
    return *this; // returns a COPY
}
(a += b) = c; // modifies the temporary copy, not a -- probably wrong
```

The fix: Return `Vec2&` (reference to `*this`): `Vec2& operator+=(const Vec2& o) { ... return *this; }`

0.5.46.2 Mistake 2: Missing the Const Overload for `operator[]`

The bug:

```
double& operator[](int i) { ... } // only non-const version
const Vec2 v{1.0, 2.0};
v[0]; // ERROR: no operator[] for const Vec2
```

The fix: Provide both `double& operator[](int i)` and `const double& operator[](int i) const`.

0.5.47 Exercises

Exercise 22.1 – Complete the vector class

Add the following to `Vec2`: `-length()` `const` returning `sqrt(x*x + y*y)` - `normalized()` `const` returning a unit vector (length 1.0) - `dot(Vec2)` `const` returning the dot product

Answer:

```
#include <cmath>

double length() const { return std::sqrt(x*x + y*y); }

Vec2 normalized() const {
    double len = length();
    if (len < 1e-12) throw std::runtime_error("Cannot normalize zero vector");
    return {x/len, y/len};
}

double dot(const Vec2& o) const { return x*o.x + y*o.y; }
```

Exercise 22.2 – Implement a Matrix2x2

Write a `Mat2` class (2x2 matrix of doubles) with: - operator* for matrix-matrix multiplication - operator* for matrix-vector multiplication (using `Vec2`) - operator==

Answer:

```
struct Mat2 {
    double a, b, c, d; // [a b; c d]

    Mat2 operator*(const Mat2& o) const {
        return {a*o.a + b*o.c, a*o.b + b*o.d,
                c*o.a + d*o.c, c*o.b + d*o.d};
    }

    Vec2 operator*(const Vec2& v) const {
        return {a*v.x + b*v.y, c*v.x + d*v.y};
    }

    bool operator==(const Mat2& o) const {
        return a==o.a && b==o.b && c==o.c && d==o.d;
    }
};
```

Part IV is complete. You can now design a full object-oriented C++ system: classes with proper encapsulation, constructors with initializer lists, inheritance hierarchies with virtual dispatch, abstract interfaces, and expressive operator overloading.

Part V covers generic programming – templates and concepts, which let you write type-safe code that works for any type that meets your requirements, with zero runtime overhead. Ask to continue.

0.6 Part V – Generic Programming

Python uses duck typing: if an object has the right methods, any algorithm works on it. C++ achieves the same expressiveness at compile time through **templates** – code that is parameterized over types. The key difference: Python resolves method calls at runtime; C++ generates specialized code for each type at compile time, producing no overhead.

0.6.1 The Problem Without Templates

You want a max function. You need it for `int`, `double`, `float`, `long` `long` – every numeric type:

```
int    max(int a,    int b)    { return a > b ? a : b; }
double max(double a, double b) { return a > b ? a : b; }
float  max(float a,  float b)  { return a > b ? a : b; }
// ... and so on for every type
```

The logic is identical for every type. Only the type changes. This is what templates solve.

0.6.2 Function Templates

A function template is a pattern. You write it once with a placeholder type, and the compiler generates concrete functions for each type you use it with.

```
# Python: works for any type automatically (duck typing)
def my_max(a, b):
    return a if a > b else b
```

```
my_max(3, 5)          # int
my_max(3.14, 2.7)     # float
my_max("b", "a")     # str
```

```
// C++: template -- compiler generates a version for each type
template <typename T>
T my_max(T a, T b) {
    return a > b ? a : b;
}

my_max(3, 5);          // compiler generates: int my_max(int, int)
my_max(3.14, 2.7);     // compiler generates: double my_max(double, double)
my_max('b', 'a');     // compiler generates: char my_max(char, char)
// my_max("b", "a");  // works too: const char* comparison (pointer comparison -- careful)
```

`template <typename T>` declares a **type parameter** `T`. Everywhere `T` appears in the function, the compiler substitutes the actual type when instantiating the template.

0.6.2.1 Template Type Deduction

The compiler deduces `T` from the arguments. You usually do not need to specify it explicitly:

```
my_max(3, 5);          // T deduced as int
my_max<double>(3, 5);  // T explicitly specified as double (3 and 5 converted)
my_max(3, 5.0);       // ERROR: ambiguous -- is T int or double?
my_max<double>(3, 5.0); // OK: explicit T=double
```

When arguments have different types, deduction fails (the compiler cannot pick one T). Specify T explicitly, or cast one argument.

0.6.2.2 Multiple Template Parameters

```
template <typename T, typename U>
auto add(T a, U b) {    // auto return type: deduced from the expression
    return a + b;
}

add(1, 2.0);           // returns double (int + double = double)
add(1, 2);              // returns int
```

0.6.3 How Templates Are Compiled

Templates are compiled differently from regular functions. This is why template errors look strange and why templates must be in headers.

When you write `my_max(3, 5)`, the compiler: 1. Finds the template definition 2. Substitutes `T = int` throughout 3. Compiles the resulting `int my_max(int, int)` as if you wrote it by hand 4. Places it in the object file

Source:

```
template<typename T> T my_max(T a, T b) { ... }
my_max(3, 5);           --> generates: int    my_max(int, int)    { ... }
my_max(3.0, 5.0);       --> generates: double my_max(double, double) { ... }
my_max('a', 'b');       --> generates: char   my_max(char, char)  { ... }
```

Each instantiation is a separate compiled function. No runtime type dispatch. Zero overhead compared to writing three functions by hand.

0.6.3.1 Why Templates Must Live in Headers

When the compiler processes `main.cpp` and sees `my_max(3, 5)`, it needs the template definition to generate the instantiation. It cannot use a declaration alone (unlike regular functions where the linker handles it later).

Therefore: **template definitions go in header files**, not `.cpp` files.

```
// math_utils.h -- template definitions here, not in a .cpp
#pragma once

template <typename T>
T my_max(T a, T b) {
    return a > b ? a : b;
}
```

If you put a template definition in a .cpp file and try to use it from another file, you get “undefined reference” at link time – the compiler in the other .cpp had no template definition to instantiate from.

0.6.4 Class Templates

The standard library containers (`std::vector<T>`, `std::map<K,V>`) are class templates. You can write your own:

```
// A simple fixed-size stack
template <typename T, int MaxSize = 16>    // T: type, MaxSize: non-type parameter
class Stack {
    T data[MaxSize];
    int top{0};

public:
    void push(const T& value) {
        if (top >= MaxSize) throw std::overflow_error{"Stack full"};
        data[top++] = value;
    }

    T pop() {
        if (top == 0) throw std::underflow_error{"Stack empty"};
        return data[--top];
    }

    const T& peek() const {
        if (top == 0) throw std::underflow_error{"Stack empty"};
        return data[top - 1];
    }

    bool empty() const { return top == 0; }
    int size() const { return top; }
};
```

Usage:

```
Stack<int>    int_stack;        // Stack of ints, max 16
Stack<double, 32> big_stack;    // Stack of doubles, max 32

int_stack.push(1);
int_stack.push(2);
int_stack.push(3);
```

```
std::cout << int_stack.pop() << "\n"; // 3
std::cout << int_stack.pop() << "\n"; // 2

Stack<std::string> str_stack;
str_stack.push("hello");
str_stack.push("world");
std::cout << str_stack.peek() << "\n"; // "world" (not popped)
```

Each combination of type arguments generates a completely separate class. `Stack<int>` and `Stack<double>` are two distinct types with no relationship to each other.

0.6.5 Non-Type Template Parameters

Templates can be parameterized by values (not just types):

```
template <int N>
struct Factorial {
    static constexpr int value = N * Factorial<N-1>::value;
};

template <> // specialization for base case
struct Factorial<0> {
    static constexpr int value = 1;
};

constexpr int f5 = Factorial<5>::value; // 120, computed entirely at compile time
constexpr int f10 = Factorial<10>::value; // 3628800
```

Array sizes, buffer sizes, and algorithm tuning parameters can all be non-type template parameters. This is how `std::array<int, 10>` works – the size 10 is a non-type template parameter.

0.6.6 Template Functions in the Standard Library

The standard library is built on templates. Some functions you will use constantly:

```
#include <algorithm>
#include <vector>

std::vector<int> v = {5, 3, 1, 4, 2};

// All work for any type that supports the required operations:
std::sort(v.begin(), v.end()); // {1,2,3,4,5}
std::reverse(v.begin(), v.end()); // {5,4,3,2,1}
auto it = std::find(v.begin(), v.end(), 3); // iterator to 3
int mx = *std::max_element(v.begin(), v.end()); // 5
```

```
std::vector<double> dv = {1.0, 2.0, 3.0};  
std::sort(dv.begin(), dv.end());           // same template, different type
```

0.6.7 Common Mistakes in This Chapter

0.6.7.1 Mistake 1: Putting Template Definitions in a .cpp File

The bug:

```
// math.cpp:  
template <typename T>  
T square(T x) { return x * x; }  
  
// main.cpp:  
template <typename T> T square(T x); // declaration only  
int main() { square(5); }           // linker error: no instantiation found
```

Linker error: undefined reference to 'int square<int>(int)' **The fix:** Move the template definition into the header.

0.6.7.2 Mistake 2: Mixed Types Without Explicit Instantiation

The bug:

```
template <typename T>  
T add(T a, T b) { return a + b; }  
  
add(1, 2.0); // ERROR: deduction fails -- is T int or double?
```

The fix: `add<double>(1, 2.0)` or `add(1.0, 2.0)` or `add(1, 2)`.

0.6.7.3 Mistake 3: Confusing Template Errors With Logic Errors

Template errors are notorious for long, cryptic messages because the error is reported after instantiation. Always look at the first error and the “required from” line that shows where you used the template.

0.6.8 Exercises

Exercise 23.1 – Write a swap template

Write `template_swap(a, b)` that swaps two values of any type. Test with `int`, `double`, and `std::string`.

Answer:

```
template <typename T>
void template_swap(T& a, T& b) {
    T tmp = std::move(a);
    a = std::move(b);
    b = std::move(tmp);
}

int a = 1, b = 2;
template_swap(a, b);    // a=2, b=1

double x = 1.5, y = 2.5;
template_swap(x, y);    // x=2.5, y=1.5

std::string s = "hello", t = "world";
template_swap(s, t);    // s="world", t="hello"
```

Exercise 23.2 – min/max template

Write `clamp<T>(value, lo, hi)` that returns `lo` if `value < lo`, `hi` if `value > hi`, otherwise `value`. The standard library has `std::clamp` – write your own.

Answer:

```
template <typename T>
T clamp(T value, T lo, T hi) {
    if (value < lo) return lo;
    if (value > hi) return hi;
    return value;
}

clamp(5, 0, 10);    // 5
clamp(-3, 0, 10);   // 0
clamp(15, 0, 10);   // 10
clamp(0.5, 0.0, 1.0); // 0.5
```

Exercise 23.3 – Class template Pair

Write a class template `Pair<A, B>` that stores two values of potentially different types. Provide: - Constructor `Pair(A, B)` - `first()` and `second()` getters (const) - Non-member `make_pair(a, b)` that deduces the types

Answer:

```
template <typename A, typename B>
class Pair {
    A first_val;
    B second_val;
public:
    Pair(A a, B b) : first_val{std::move(a)}, second_val{std::move(b)} {}
    const A& first() const { return first_val; }
    const B& second() const { return second_val; }
};

template <typename A, typename B>
Pair<A,B> make_pair(A a, B b) {
    return Pair<A,B>{std::move(a), std::move(b)};
}

auto p = make_pair(42, std::string{"hello"});
std::cout << p.first() << " " << p.second() << "\n"; // 42 hello
```

Chapter 24: Template Specialization and Variadic Templates

0.6.9 Template Specialization

Sometimes the generic template is wrong or inefficient for a specific type. **Full specialization** provides a completely custom implementation for a concrete type:

```
// Primary template: works for any T
template <typename T>
struct TypeInfo {
    static std::string name() { return "unknown"; }
};

// Full specialization for int:
template <> // <> means: no template parameters -- this is a specialization
struct TypeInfo<int> {
    static std::string name() { return "int"; }
};

// Full specialization for double:
template <>
struct TypeInfo<double> {
    static std::string name() { return "double"; }
};

std::cout << TypeInfo<int>::name() << "\n"; // "int"
std::cout << TypeInfo<double>::name() << "\n"; // "double"
std::cout << TypeInfo<char>::name() << "\n"; // "unknown" (uses primary template)
```

0.6.9.1 Partial Specialization

Partial specialization provides a custom implementation for a subset of type combinations, keeping some template parameters generic:

```
// Primary template
template <typename T, typename U>
struct IsSame { static constexpr bool value = false; };

// Partial specialization: both types are the same
template <typename T>
struct IsSame<T, T> { static constexpr bool value = true; };

IsSame<int, int>::value    // true
IsSame<int, double>::value // false
```

Another common example – specializing for pointers:

```
// Primary template: general case
template <typename T>
class Storage {
    T data;
public:
    void set(T v) { data = v; }
    T get()      { return data; }
};

// Partial specialization for pointer types
template <typename T>
class Storage<T*> {           // specialization matches any T*
    T* data{nullptr};
public:
    void set(T* p) { data = p; }
    T* get()      { return data; }
    bool valid()  { return data != nullptr; }
    // Extra: null check makes sense for pointers
};

Storage<int>    si;    // uses primary template
Storage<int*>   sp;    // uses pointer specialization -- has valid() method
```

0.6.10 if constexpr – Compile-Time Branching

Before concepts (next chapter), `if constexpr` lets you branch based on compile-time conditions inside templates:

```
#include <type_traits>

template <typename T>
void describe(T value) {
```

```

    if constexpr (std::is_integral_v<T>) {
        std::cout << "integer: " << value << "\n";
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << "float:   " << value << "\n";
    } else {
        std::cout << "other:   " << value << "\n";
    }
}

describe(42);           // "integer: 42"
describe(3.14);         // "float:   3.14"
describe("hello");      // "other:   hello"

```

Unlike a regular `if`, only the matching branch is compiled. The other branches can contain code that would fail to compile for this type – they are simply discarded.

```

template <typename T>
void print_info(T v) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "bits: " << sizeof(T) * 8 << "\n";
        std::cout << "max:  " << std::numeric_limits<T>::max() << "\n";
    } else {
        std::cout << "not an integer\n";
    }
}

```

0.6.11 Type Traits

`<type_traits>` provides compile-time predicates about types:

```

#include <type_traits>

std::is_integral_v<int>           // true
std::is_integral_v<double>        // false
std::is_floating_point_v<float>   // true
std::is_pointer_v<int*>           // true
std::is_reference_v<int&>         // true
std::is_const_v<const int>        // true
std::is_same_v<int, int>          // true
std::is_same_v<int, long>         // false (on most platforms!)
std::is_base_of_v<Animal, Dog>    // true
std::is_convertible_v<int, double> // true

```

These are the building blocks of generic code that adapts to the type it receives.

0.6.12 Variadic Templates

A variadic template accepts **any number of type arguments** (including zero). This is how `std::tuple`, `std::make_unique`, and `std::format` are implemented.

*# Python: *args handles any number of arguments*

```
def print_all(*args):
    for arg in args:
        print(arg)
```

```
print_all(1, "hello", 3.14)
```

// C++: variadic template with parameter pack

```
template <typename... Args> // Args is a "parameter pack" -- zero or more types
void print_all(Args... args) {
    // Expand the pack using fold expression (C++17):
    (std::cout << ... << args); // prints each arg with no separator
    std::cout << "\n";
}
```

```
print_all(1, " hello ", 3.14); // "1 hello 3.14"
```

The `...` is the pack expansion operator. `Args...` unpacks the types; `args...` unpacks the values.

0.6.12.1 Recursive Variadic Template (Classic Style)

Before C++17 fold expressions, variadic templates were handled recursively:

// Base case: zero arguments

```
void print_all() {}
```

// Recursive case: print first, then recurse on the rest

```
template <typename First, typename... Rest>
void print_all(First first, Rest... rest) {
    std::cout << first << " ";
    print_all(rest...); // recurse with one fewer argument
}
```

```
print_all(1, "hello", 3.14, true);
```

// Output: 1 hello 3.14 1

0.6.12.2 Fold Expressions (C++17)

More concise for common patterns:

// Sum of any number of values:

```
template <typename... Ts>
auto sum(Ts... args) {
    return (... + args); // left fold: ((arg1 + arg2) + arg3) + ...
}
```

```
sum(1, 2, 3, 4, 5);    // 15
sum(1.0, 2.5, 0.5);   // 4.0
sum(std::string{"a"}, std::string{"b"}, std::string{"c"}); // "abc"
```

Fold expression forms:

```
(... op pack)      // left fold: (((a op b) op c) op d)
(pack op ...)      // right fold: (a op (b op (c op d)))
(init op ... op pack) // left fold with initial value
(pack op ... op init) // right fold with initial value
```

0.6.13 Common Mistakes in This Chapter

0.6.13.1 Mistake 1: Full Specialization After the Template Is Already Instantiated

The bug:

```
template <typename T> void foo(T) { std::cout << "generic\n"; }
foo(5);                          // instantiates foo<int> as generic
template <> void foo<int>(int) { std::cout << "int\n"; }
// Too late: the specialization after the call point may not be seen by the compiler
```

The fix: Declare all specializations before any instantiations – usually by putting everything in the header, with specializations after the primary template.

0.6.13.2 Mistake 2: Forgetting `template <>` for Full Specialization

The bug:

```
template <typename T> struct Foo { };
struct Foo<int> { }; // ERROR: looks like a redefinition of a non-template class
```

The fix: `template <> struct Foo<int> { };`

0.6.14 Exercises

Exercise 24.1 – Specialize for bool

The `Stack<T>` from Chapter 23 stores `bool` values as full `T` data[MaxSize] which wastes space. Write a full specialization `Stack<bool>` that uses a `uint64_t` as a bitset (packing 64 bools into one `uint64_t`). Only implement `push`, `pop`, and `size`.

Answer (simplified):

```
template <>
class Stack<bool, 64> {
    uint64_t bits{0};
    int      top{0};
public:
    void push(bool v) {
        if (top >= 64) throw std::overflow_error{"Stack full"};
        if (v) bits |= (1ULL << top);
        else bits &= ~(1ULL << top);
        ++top;
    }
    bool pop() {
        if (top == 0) throw std::underflow_error{"Stack empty"};
        --top;
        return (bits >> top) & 1;
    }
    int size() const { return top; }
};
```

Exercise 24.2 – Variadic min

Write a variadic `vmin(args...)` that returns the minimum of any number of values.

Answer:

```
template <typename T>
T vmin(T only) { return only; }

template <typename T, typename... Rest>
T vmin(T first, Rest... rest) {
    T rest_min = vmin(rest...);
    return first < rest_min ? first : rest_min;
}

vmin(5, 3, 8, 1, 6); // 1
vmin(3.14, 2.71, 1.41); // 1.41
```

0.6.15 The Problem With Unconstrained Templates

```
template <typename T>
T my_max(T a, T b) {
    return a > b ? a : b;
}

struct Foo {};
my_max(Foo{}, Foo{}); // ERROR -- but the error is inside the template, very confusing
```

The error message:

```
error: no match for 'operator>' (operand types are 'Foo' and 'Foo')
  3 |         return a > b ? a : b;
    |                        ~~~^
note: in instantiation of function template specialization 'my_max<Foo>' requested here
  8 |         my_max(Foo{}, Foo{});
```

The error tells you the problem, but for complex templates it can span dozens of lines. More importantly, the error message talks about the template's internals – not about what the caller did wrong.

Concepts let you state requirements on template parameters up front, producing clear error messages at the call site.

0.6.16 Defining and Using Concepts

A **concept** is a named compile-time predicate over types:

```
# Python 3.12+ type hints (still runtime duck typing)
from typing import Protocol
```

```
class Comparable(Protocol):
    def __gt__(self, other) -> bool: ...
```

```
// C++20 concept: compile-time, enforced by compiler
#include <concepts>
```

```
template <typename T>
concept Comparable = requires(T a, T b) {
    { a > b } -> std::convertible_to<bool>; // expression 'a > b' must be valid
    { a < b } -> std::convertible_to<bool>; // and convertible to bool
};
```

```
// Use the concept to constrain the template:
```

```
template <Comparable T> // shorthand constraint syntax
T my_max(T a, T b) {
    return a > b ? a : b;
```



```

}

// Or equivalent with 'requires' clause:
template <typename T>
requires Comparable<T>
T my_max(T a, T b) { return a > b ? a : b; }

```

Now trying `my_max(Foo{}, Foo{})` gives:

```

error: no matching function for call to 'my_max(Foo, Foo)'
note: constraints not satisfied
note: 'Comparable<Foo>' evaluated to false
note: 'a > b' is not a valid expression for type 'Foo'

```

The error is at the call site and says exactly what is missing.

0.6.17 Standard Library Concepts (<concepts>)

C++20 provides many ready-made concepts:

```

#include <concepts>

std::integral<int>           // true: int is integral
std::integral<double>       // false
std::floating_point<double> // true
std::signed_integral<int>   // true
std::same_as<int, int>      // true
std::derived_from<Dog, Animal> // true (Dog derived from Animal)
std::convertible_to<int, double> // true
std::equality_comparable<int> // true (int has == and !=)
std::totally_ordered<int>    // true (int has <, >, <=, >=)
std::copyable<std::vector<int>> // true
std::movable<std::unique_ptr<int>> // true
std::callable<std::function<int()>> // true

```

Use these in your own templates:

```

#include <concepts>

template <std::integral T>           // T must be an integral type
T next(T n) { return n + 1; }

template <std::floating_point T>     // T must be a float type
T reciprocal(T n) { return T{1} / n; }

next(5);           // OK
next(3.14);        // ERROR: double does not satisfy 'integral'
reciprocal(2.0);   // OK
reciprocal(2);     // ERROR: int does not satisfy 'floating_point'

```

0.6.18 Writing Your Own Concepts

A concept is defined with `requires` expressions that describe what operations must be valid:

```
// A type T is Printable if it can be inserted into an ostream:
template <typename T>
concept Printable = requires(T v, std::ostream& os) {
    { os << v };    // this expression must compile
};

// A type T is Container if it has begin(), end(), and size():
template <typename T>
concept Container = requires(T c) {
    { c.begin() } -> std::input_iterator;
    { c.end()   } -> std::input_iterator;
    { c.size()  } -> std::convertible_to<std::size_t>;
};

// A type T is Numeric if it supports +, -, *, / with itself:
template <typename T>
concept Numeric = requires(T a, T b) {
    { a + b } -> std::same_as<T>;
    { a - b } -> std::same_as<T>;
    { a * b } -> std::same_as<T>;
    { a / b } -> std::same_as<T>;
};

// Use all three:
template <Container C, Printable E>
requires std::same_as<typename C::value_type, E>
void print_container(const C& c) {
    for (const E& e : c)
        std::cout << e << " ";
    std::cout << "\n";
}
```

0.6.19 Abbreviated Function Templates (C++20)

The cleanest concept syntax uses `auto` with concept constraints:

```
// Instead of:
template <std::integral T>
T double_it(T n) { return n * 2; }

// Write:
std::integral auto double_it(std::integral auto n) { return n * 2; }
//      ^-- abbreviated function template: auto = template parameter
//      std::integral auto means "auto, but constrained to integral types"
```

For simple cases this is much shorter. For complex cases with multiple interdependent parameters, the explicit template <typename T> form is clearer.

0.6.20 Concepts vs Runtime Type Checks

Concepts check types at **compile time**. The entire constraint system is resolved before any code runs. Failed constraints are compiler errors, not runtime exceptions.

Python (duck typing):

- Checked at runtime when the method is called
- TypeError raised if method missing
- Only the failing call path is caught

C++ concepts:

- Checked at compile time when the template is instantiated
- Compile error if constraint not satisfied
- All call sites checked, not just the ones tested
- Zero runtime overhead

This is what C++ means by “you don’t pay for what you don’t use” – concepts are checked once at compile time and then disappear from the runtime.

0.6.21 Common Mistakes in This Chapter

0.6.21.1 Mistake 1: Using requires in the Wrong Place

The bug:

```
template <typename T>
T foo(T a) requires std::integral<T> // requires clause at end of function signature
{ return a; }
// Fine syntactically, but easy to confuse with body. Prefer:
template <std::integral T> T foo(T a) { return a; }
```

Both are legal; the second is cleaner for simple cases.

0.6.21.2 Mistake 2: Concept Is Too Strict or Too Loose

The bug: Writing a concept that requires operator< but your algorithm actually needs operator<=:

```

template <typename T>
concept Ordered = requires(T a, T b) { a < b; }; // missing <=

template <Ordered T>
bool in_range(T val, T lo, T hi) {
    return lo <= val && val <= hi; // uses <= -- not checked by concept!
}

```

The fix: List every operation your template actually uses in the concept.

0.6.22 Exercises

Exercise 25.1 – Constrain my_max

Rewrite my_max from Chapter 23 with a concept that requires operator>. Test that it rejects Foo{} with a clear error.

Answer:

```

template <typename T>
concept HasGreaterThan = requires(T a, T b) {
    { a > b } -> std::convertible_to<bool>;
};

template <HasGreaterThan T>
T my_max(T a, T b) { return a > b ? a : b; }

my_max(3, 5); // OK
my_max(3.0, 5.0); // OK
// my_max(Foo{}, Foo{}); // Compile error with clear message

```

Exercise 25.2 – Numeric concept

Write a sum function that computes the sum of a std::vector<T> where T must satisfy std::integral or std::floating_point. Use a combined concept.

Answer:

```

template <typename T>
concept Arithmetic = std::integral<T> || std::floating_point<T>;

template <Arithmetic T>
T sum(const std::vector<T>& v) {
    T total{};
    for (const T& x : v) total += x;
    return total;
}

```

```
sum(std::vector<int>{1, 2, 3, 4, 5}); // 15
sum(std::vector<double>{1.1, 2.2, 3.3}); // 6.6
// sum(std::vector<std::string>{"a", "b"}); // ERROR: string not Arithmetic
```

Chapter 26: An Introduction to Template Metaprogramming

0.6.23 What Is Template Metaprogramming?

Template metaprogramming (TMP) is using the C++ template system to compute values and make decisions at **compile time**. The template system is Turing-complete – in principle you can compute anything at compile time.

In practice, TMP is used for: - Computing compile-time constants (sizes, masks, lookup tables) - Selecting code paths based on types without runtime overhead - Generating optimized code for specific type configurations - Implementing type traits and concepts

Modern C++ has made most raw TMP unnecessary. `constexpr` functions, `if constexpr`, and concepts replace many old TMP patterns more readably. But understanding TMP helps you read library code and write efficient generic code.

0.6.24 Compile-Time Recursion

TMP was originally done with recursive struct templates (before `constexpr`):

```
// Fibonacci at compile time (old style):
template <int N>
struct Fib {
    static constexpr int value = Fib<N-1>::value + Fib<N-2>::value;
};
template <> struct Fib<0> { static constexpr int value = 0; };
template <> struct Fib<1> { static constexpr int value = 1; };

constexpr int f10 = Fib<10>::value; // 55, computed at compile time
```

Modern equivalent with `constexpr` function (far more readable):

```
constexpr int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
constexpr int f10 = fib(10); // 55, computed at compile time
```

Always prefer `constexpr` functions over struct-recursion TMP for computations.

0.6.25 `std::enable_if` – Conditional Compilation (Old Style)

Before concepts, `std::enable_if` was used to enable or disable template instantiations based on type traits:

```
#include <type_traits>

// Only enabled for integral types (old style, pre-C++20):
template <typename T,
         typename = std::enable_if_t<std::is_integral_v<T>>>
T next(T n) { return n + 1; }

// With concepts (C++20, prefer this):
template <std::integral T>
T next(T n) { return n + 1; }
```

You will encounter `enable_if` in older code and library documentation. Understanding it: `std::enable_if_t<condition>` is void when condition is true (template is valid) and causes substitution failure when condition is false (template is removed from consideration). This is called **SFINAE** (Substitution Failure Is Not An Error).

0.6.26 Type Lists – Compile-Time Lists of Types

Variadic templates let you create lists of types that exist purely at compile time:

```
// A type list:
template <typename... Ts>
struct TypeList {};

using MyTypes = TypeList<int, double, std::string, bool>;

// Count the types in a list:
template <typename List>
struct Length;

template <typename... Ts>
struct Length<TypeList<Ts...>> {
    static constexpr int value = sizeof...(Ts);
};

constexpr int n = Length<MyTypes>::value; // 4
```

Type lists are the foundation of tuple implementations, plugin systems, and serialization frameworks that need to enumerate types at compile time.

0.6.27 Compile-Time Lookup Tables

A powerful pattern: compute a lookup table at compile time so it is a static constant array in the binary:

```
#include <array>

// Generate a table of squares at compile time:
constexpr std::array<int, 10> squares = []() {
    std::array<int, 10> arr{};
    for (int i = 0; i < 10; ++i) arr[i] = i * i;
    return arr;
}(); // immediately invoked lambda

// At runtime, squares is a pre-computed constant array in .rodata
// No runtime computation at all:
std::cout << squares[7] << "\n"; // 49 -- just a memory read
```

This pattern works because: 1. The lambda is `constexpr`-evaluable 2. `std::array` is a literal type 3. The entire computation happens at compile time

For game engines, look-up tables for sin/cos, byte-reversal, CRC, and Huffman trees are often pre-computed this way.

0.6.28 `std::tuple` – Compile-Time Heterogeneous Collections

`std::tuple` is a variadic class template that holds values of different types:

```
# Python tuple
t = (42, "hello", 3.14)
print(t[0]) # 42 (runtime indexing)
```

```
// C++ tuple
#include <tuple>
std::tuple<int, std::string, double> t{42, "hello", 3.14};

// Access by index -- index must be a compile-time constant:
std::get<0>(t); // 42
std::get<1>(t); // "hello"
std::get<2>(t); // 3.14

// Cannot do: std::get<i>(t) where i is a runtime variable
// The index is resolved at compile time

// C++17 structured bindings (much nicer):
auto [num, str, flt] = t;
std::cout << num << " " << str << " " << flt << "\n";
```

`std::get<N>` is resolved at compile time. Accessing `std::get<5>(t)` on a 3-element tuple is a compile error, not a runtime error.

0.6.29 `std::conditional` – Type Selection at Compile Time

Choose between two types based on a condition:

```
#include <type_traits>

// If condition is true, type is int; otherwise long long
template <bool IsSmall>
using StorageType = std::conditional_t<IsSmall, int, long long>;

StorageType<true> a = 5;           // int
StorageType<false> b = 5000000000; // long long

// Practical use: choose between fast and accurate computation
template <bool Fast>
using Float = std::conditional_t<Fast, float, double>;
```

0.6.30 The Difference Between TMP and `constexpr`

Template Metaprogramming:

- Computation done by the template instantiation engine
- Types are first-class citizens
- Syntax is complex (recursive struct templates, `enable_if`)
- Result is a type or compile-time constant
- Use when you need to manipulate TYPES at compile time

`constexpr`:

- Computation done by a regular function (or expression) evaluated at compile time
- Normal C++ syntax, easy to read
- Result is a VALUE (int, array, struct...)
- Use when you need to compute VALUES at compile time

C++20 guidance:

- For compile-time values: use `constexpr` functions
- For type selection: use `concepts`, `if constexpr`, `std::conditional`
- Raw TMP (recursive struct templates) is rarely needed in new code

0.6.31 Common Mistakes in This Chapter

0.6.31.1 Mistake 1: Trying to Use a Runtime Value as a Template Argument

The bug:


```
int n;
std::cin >> n;
std::array<int, n> arr; // ERROR: n is not a compile-time constant
```

The fix: Use `std::vector<int>(n)` for runtime sizes.

0.6.31.2 Mistake 2: Recursive TMP Instead of constexpr

The bug:

```
template <int N> struct Pow2 { static constexpr int v = 2 * Pow2<N-1>::v; };
template <> struct Pow2<0> { static constexpr int v = 1; };
```

Better: `constexpr int pow2(int n) { return 1 << n; }`

0.6.32 Exercises

Exercise 26.1 – Compile-time lookup table

Create a `constexpr std::array<int, 16>` of the first 16 powers of 2 (1, 2, 4, 8, ...). Access element 10 and verify it is 1024.

Answer:

```
constexpr std::array<int, 16> powers_of_2 = []() {
    std::array<int, 16> arr{};
    arr[0] = 1;
    for (int i = 1; i < 16; ++i) arr[i] = arr[i-1] * 2;
    return arr;
}();

static_assert(powers_of_2[10] == 1024); // compile-time check
std::cout << powers_of_2[10] << "\n"; // 1024
```

Exercise 26.2 – Type traits

Write a function `type_name<T>()` that returns a `std::string` naming the type as “integral”, “floating_point”, “string”, or “other”:

Answer:

```
#include <concepts>
#include <string>

template <typename T>
std::string type_name() {
    if constexpr (std::integral<T>) return "integral";
    else if constexpr (std::floating_point<T>) return "floating_point";
    else if constexpr (std::same_as<T, std::string>) return "string";
    else return "other";
}

type_name<int>();           // "integral"
type_name<double>();       // "floating_point"
type_name<std::string>();  // "string"
type_name<bool>();         // "integral" (bool is integral)
type_name<char*>();        // "other"
```

Exercise 26.3 – Tuple structured binding

Create a function `minmax(std::vector<T>)` that returns a `std::pair<T, T>` containing the minimum and maximum element. Use structured bindings to unpack the result.

Answer:

```
#include <vector>
#include <utility>
#include <algorithm>

template <typename T>
std::pair<T, T> minmax_pair(const std::vector<T>& v) {
    auto [mn, mx] = std::minmax_element(v.begin(), v.end());
    return {*mn, *mx};
}

std::vector<int> v = {5, 2, 8, 1, 9, 3};
auto [lo, hi] = minmax_pair(v);
std::cout << "min=" << lo << " max=" << hi << "\n"; // min=1 max=9
```

Part V is complete. You now understand C++ generic programming: function and class templates, specialization, variadic templates, concepts for type-safe constraints, and the foundations of template metaprogramming.

Part VI covers the C++ Standard Library in depth – containers, iterators, algorithms, lambdas, ranges, and utility types. These are the tools you will use in every real program. Ask to continue.

0.7 Part VI – The Standard Library

The C++ Standard Library is not a collection of helper functions bolted on after the fact. It is a carefully designed ecosystem of containers, iterators, algorithms, and utilities that work together through a common interface. Learning to use it well is the difference between writing C++ that feels like C with classes and C++ that is expressive, safe, and fast.

Chapter 27: Containers: vector, map, set, array, and Friends

0.7.1 The Container Taxonomy

All standard containers fall into three categories:

Sequence containers (ordered by position):

<code>std::array<T, N></code>	fixed-size array, stack-allocated
<code>std::vector<T></code>	dynamic array, heap-allocated
<code>std::deque<T></code>	double-ended queue
<code>std::list<T></code>	doubly-linked list
<code>std::forward_list<T></code>	singly-linked list

Associative containers (ordered by key):

<code>std::map<K, V></code>	sorted key-value pairs, unique keys
<code>std::multimap<K, V></code>	sorted key-value pairs, duplicate keys allowed
<code>std::set<K></code>	sorted unique keys
<code>std::multiset<K></code>	sorted keys, duplicates allowed

Unordered associative containers (hash-based):

<code>std::unordered_map<K, V></code>	hash map, unique keys
<code>std::unordered_set<K></code>	hash set, unique keys

Container adapters (built on other containers):

<code>std::stack<T></code>	LIFO adapter (uses deque by default)
<code>std::queue<T></code>	FIFO adapter (uses deque by default)
<code>std::priority_queue<T></code>	max-heap adapter (uses vector by default)

0.7.2 `std::vector<T>` – Your Default Container

Already covered in Chapter 10. Here are the parts you need for real programs:

0.7.2.1 Reservation and Capacity Management

```
std::vector<int> v;
v.reserve(1000);           // pre-allocate space for 1000 elements
                           // avoids repeated reallocations during push_back

std::cout << v.size()      << "\n"; // 0      (no elements yet)
std::cout << v.capacity() << "\n"; // 1000  (space reserved)

for (int i = 0; i < 1000; ++i)
    v.push_back(i);        // no reallocations happen (capacity was reserved)

v.shrink_to_fit();         // release excess capacity to OS
```

Without reserve (1000 push_backs):

Realloc at 1, 2, 4, 8, 16, ... 512 = ~10 reallocations
 Each realloc: allocate new array, copy all elements, free old array
 Total element copies: $1+2+4+\dots+512 \approx 1000$ extra copies

With reserve(1000):

Zero reallocations. Zero extra copies.

Reserve when you know an upper bound on the number of elements.

0.7.2.2 `emplace_back` vs `push_back`

```
struct Point { double x, y; Point(double x, double y) : x{x}, y{y} {} };

std::vector<Point> pts;

// push_back: constructs a Point, then copies/moves it into the vector
pts.push_back(Point{1.0, 2.0}); // construct temporary, then move-construct into vector

// emplace_back: constructs the Point DIRECTLY inside the vector memory
pts.emplace_back(1.0, 2.0);    // no temporary, arguments forwarded to constructor
```

`emplace_back` is generally preferred for class types – it constructs in-place, avoiding the temporary. For types where the move is cheap (most standard types), the difference is minimal. For non-movable types, `emplace_back` is the only option.

0.7.3 `std::deque<T>` – Fast at Both Ends

Like vector but also allows $O(1)$ insertion and removal at the front:

```
#include <deque>
std::deque<int> dq = {3, 4, 5};
dq.push_front(2);    // {2, 3, 4, 5}
dq.push_front(1);    // {1, 2, 3, 4, 5}
dq.push_back(6);     // {1, 2, 3, 4, 5, 6}
dq.pop_front();      // {2, 3, 4, 5, 6}
```

Use deque when you need fast insertion at both ends. Use vector otherwise – vector’s cache efficiency is better for sequential access.

0.7.4 std::map<K, V> – Sorted Key-Value Store

```
# Python dict -- hash map
scores = {"Alice": 95, "Bob": 87, "Carol": 92}
scores["Dave"] = 78
print(scores["Alice"]) # 95
```

```
// C++ map -- sorted by key (balanced BST internally)
#include <map>
std::map<std::string, int> scores;
scores["Alice"] = 95;
scores["Bob"]   = 87;
scores["Carol"] = 92;
scores["Dave"]  = 78;

std::cout << scores["Alice"] << "\n"; // 95
```

0.7.4.1 Key Difference: std::map vs Python dict

Feature	Python dict	std::map	std::unordered_map
Order	Insertion order (3.7+)	Sorted by key	No guaranteed order
Lookup	O(1) average	O(log n)	O(1) average
Key requirement	Hashable	Comparable (<)	Hashable
Memory	Compact	BST nodes (pointers)	Hash buckets

Use std::map when you need ordered iteration. Use std::unordered_map when you need fast lookups and don’t care about order.

0.7.4.2 Safe Lookup: at() vs operator[]

```
std::map<std::string, int> m = {"a", 1}, {"b", 2};;

m["c"];           // WARNING: inserts "c" with default value 0 if not present!
                  // m is now {"a":1, "b":2, "c":0}

m.at("d");        // throws std::out_of_range: "d" not in map (no insertion)
m.at("a");        // 1 -- safe read

// Best: check first
if (m.count("a")) { std::cout << m.at("a") << "\n"; }

// Or use find:
auto it = m.find("a");
if (it != m.end()) {
    std::cout << it->first << " -> " << it->second << "\n";
}
```

The operator `[]` creates missing keys with a default-constructed value. This is a very common source of bugs: you check `map[key]` to see if a key exists, and it silently inserts a zero.

0.7.4.3 Iterating Over a Map

```
std::map<std::string, int> m = {"Alice", 95}, {"Bob", 87}, {"Carol", 92};;

// Iteration is in sorted key order:
for (const auto& [key, value] : m) { // structured binding (C++17)
    std::cout << key << ": " << value << "\n";
}
// Alice: 95
// Bob: 87
// Carol: 92
```

0.7.4.4 Inserting and Checking Simultaneously

```
// insert_or_assign (C++17): always sets the value
m.insert_or_assign("Dave", 78); // inserts if new, overwrites if exists

// try_emplace (C++17): inserts only if key is absent, does nothing if present
m.try_emplace("Alice", 100);    // Alice already exists: nothing happens (still 95)
m.try_emplace("Eve", 88);       // Eve is new: inserted with 88

// erase:
m.erase("Bob");
```

0.7.5 `std::unordered_map<K, V>` – Hash Map

Same interface as `std::map` but uses hashing for $O(1)$ average lookup:

```
#include <unordered_map>
std::unordered_map<std::string, int> umap;
umap["Alice"] = 95;
umap["Bob"]   = 87;

// Same interface as map:
umap.at("Alice");    // 95
umap.count("Carol"); // 0 -- not present
auto it = umap.find("Bob");

// But NO guaranteed iteration order:
for (const auto& [k, v] : umap) { ... } // order is unpredictable
```

Use `unordered_map` for most cases where you need key-value lookup. Use `map` when sorted order matters (e.g., printing in alphabetical order, range queries).

0.7.6 `std::set<K>` – Sorted Unique Keys

```
# Python set
s = {3, 1, 4, 1, 5, 9, 2}
print(s) # {1, 2, 3, 4, 5, 9} (unordered)
```

```
#include <set>
std::set<int> s = {3, 1, 4, 1, 5, 9, 2}; // duplicates silently ignored
// s contains: {1, 2, 3, 4, 5, 9} (sorted, unique)

s.insert(7); // {1, 2, 3, 4, 5, 7, 9}
s.erase(4); // {1, 2, 3, 5, 7, 9}
s.count(5);  // 1 (present)
s.count(4);  // 0 (erased)

for (int n : s)
    std::cout << n << " "; // 1 2 3 5 7 9 (sorted order)
```

For hash-based set: `std::unordered_set<K>` – $O(1)$ lookup, no order.

0.7.7 Choosing the Right Container

I need to store N items and access by position:

- Size known at compile time: `std::array<T, N>`
- Size varies at runtime: `std::vector<T>`
- Need fast front insertion: `std::deque<T>`

I need key-value lookup:

- Order matters / sorted iteration: `std::map<K, V>`
- Maximum speed, order irrelevant: `std::unordered_map<K, V>`

I need to track unique items:

- Order matters: `std::set<K>`
- Maximum speed, no order: `std::unordered_set<K>`

I need LIFO (stack) behavior: `std::stack<T>`

I need FIFO (queue) behavior: `std::queue<T>`

I need the max element always: `std::priority_queue<T>`

0.7.8 Common Mistakes in This Chapter

0.7.8.1 Mistake 1: `map[key]` to Check Existence

The bug:

```
std::map<std::string, int> m = {{"a", 1}};
if (m["b"]) { ... } // inserts "b" with value 0! "b" is now in the map.
```

The fix: `if (m.count("b"))` or `if (m.find("b") != m.end())`

0.7.8.2 Mistake 2: Invalidating Iterators by Modifying the Container

The bug:

```
std::vector<int> v = {1, 2, 3, 4, 5};
for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it == 3) v.erase(it); // erase invalidates 'it' and all iterators after it!
}
```

The fix:

```
v.erase(std::remove(v.begin(), v.end(), 3), v.end()); // erase-remove idiom
```


0.7.8.3 Mistake 3: Linear Search on an Unsorted vector When a set Would Be $O(\log n)$

If you are repeatedly calling `std::find` on a vector, consider whether `std::set` or `std::unordered_set` would serve better.

0.7.9 Exercises

Exercise 27.1 – Word frequency count

Read a list of words and count how many times each word appears. Print words and counts in alphabetical order.

Answer:

```
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> freq;
    std::string word;
    while (std::cin >> word) {
        ++freq[word];    // OK to use [] here: we WANT insertion with 0 if new
    }
    for (const auto& [w, count] : freq) {
        std::cout << w << ": " << count << "\n";
    }
}
```

Exercise 27.2 – Reverse a deque

Push the integers 1 through 10 into a `std::deque`, then repeatedly pop from the front to print them in reverse order (10 down to 1). Do not use `std::reverse`.

Answer:

```
std::deque<int> dq;
for (int i = 1; i <= 10; ++i) dq.push_front(i);
// dq is now: 10 9 8 7 6 5 4 3 2 1

while (!dq.empty()) {
    std::cout << dq.front() << " ";
    dq.pop_front();
} // prints: 10 9 8 7 6 5 4 3 2 1
```

Exercise 27.3 – Unique elements

Given `std::vector<int> v = {5,3,1,4,2,3,5,1,4}`, produce a sorted vector of unique elements without writing a `sort+unique` manually.

Answer:

```
std::vector<int> v = {5,3,1,4,2,3,5,1,4};
std::set<int> s(v.begin(), v.end()); // set removes duplicates, sorts
std::vector<int> unique(s.begin(), s.end()); // back to vector: {1,2,3,4,5}
```

Chapter 28: Iterators**0.7.10 What Is an Iterator?**

An iterator is an object that points into a container and can be advanced. It generalizes the concept of a pointer – in fact, raw pointers are valid iterators for arrays.

```
# Python iteration: the iterator protocol (hidden from you)
it = iter([1, 2, 3])
next(it) # 1
next(it) # 2
next(it) # 3
```

```
// C++ iterators: explicit objects you manipulate
std::vector<int> v = {1, 2, 3};
auto it = v.begin(); // iterator to first element
std::cout << *it << "\n"; // 1 -- dereference like a pointer
++it;
std::cout << *it << "\n"; // 2
++it;
std::cout << *it << "\n"; // 3
++it;
// it == v.end() -- one past the last element (do not dereference!)
```

`v.begin()` returns an iterator to the first element. `v.end()` returns an iterator to one-past-the-last element.

The half-open range `[begin, end)` is a C++ convention. `end` itself is never valid to dereference – it is a sentinel.

0.7.11 Iterator Categories

Different containers support different iterator capabilities:

Input iterator: read once, advance once (e.g., reading from a file stream)

Output iterator: write once, advance once

Forward iterator: read/write, advance forward, multi-pass

Bidirectional: forward + go backward (std::list, std::map)

Random access: any position in O(1) (std::vector, std::array, raw array)

Contiguous: random access + elements are contiguous in memory (std::vector, std::array)

```
std::vector<int> v = {1, 2, 3, 4, 5};
auto it = v.begin();

it + 2;           // random access: jump forward by 2
it[2];           // random access: subscript
*(it + 2);        // 3
it += 3;          // advance by 3

std::list<int> l = {1, 2, 3};
auto lit = l.begin();
++lit;           // forward
--lit;           // backward (bidirectional)
// lit + 2;      // ERROR: list iterators are not random access
```

The algorithm functions in <algorithm> work on any appropriate iterator category. std::sort requires random-access iterators (works on vector, not list). std::find requires only forward iterators (works on both).

0.7.12 The Iterator-Pair Pattern

The standard library uses iterator pairs to describe ranges:

```
std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// Operate on a subrange:
std::sort(v.begin() + 2, v.begin() + 7); // sort only elements [2, 7)
// v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} (already sorted, but you could sort reversed subrange)

// Find in a subrange:
auto it = std::find(v.begin(), v.begin() + 5, 3); // search only first 5 elements
```

This is both powerful (you can describe any contiguous subrange) and error-prone (easy to get the boundaries wrong).

0.7.13 begin / end Free Functions

C++11 added free function versions that work on arrays too:

```
int arr[5] = {5, 3, 1, 4, 2};
std::sort(std::begin(arr), std::end(arr)); // sorts the C-array!

std::vector<int> v = {5, 3, 1};
std::sort(std::begin(v), std::end(v));    // same syntax
```

`std::begin(arr)` returns a pointer to `arr[0]`. `std::end(arr)` returns a pointer past the last element. Raw pointers are valid random-access iterators.

0.7.14 Reverse Iterators

```
std::vector<int> v = {1, 2, 3, 4, 5};

// Iterate backward:
for (auto it = v.rbegin(); it != v.rend(); ++it) {
    std::cout << *it << " "; // 5 4 3 2 1
}

// rbegin() = reverse iterator to last element
// rend()    = reverse iterator to one-before-first (sentinel)
// ++ on a reverse iterator goes backward
```

0.7.15 Insert Iterators

Output iterators that insert into a container instead of overwriting:

```
#include <iterator>
std::vector<int> src = {1, 2, 3};
std::vector<int> dst;

// back_inserter: calls push_back on each write
std::copy(src.begin(), src.end(), std::back_inserter(dst));
// dst = {1, 2, 3}

// front_inserter: calls push_front (only for deque, list)
std::deque<int> dq;
std::copy(src.begin(), src.end(), std::front_inserter(dq));
// dq = {3, 2, 1} (each is pushed to front)
```

0.7.16 Stream Iterators

Treat input/output streams as iterators:

```
#include <iterator>
#include <fstream>
#include <algorithm>

// Read all ints from stdin into a vector:
std::vector<int> v(
    std::istream_iterator<int>{std::cin},    // begin: reads ints from stdin
    std::istream_iterator<int>{}             // end: default-constructed = EOF
);

// Write a vector to stdout with spaces:
std::copy(v.begin(), v.end(),
    std::ostream_iterator<int>{std::cout, " "});
```

0.7.17 Common Mistakes in This Chapter

0.7.17.1 Mistake 1: Dereferencing end()

The bug:

```
auto it = v.end();
std::cout << *it;    // undefined behavior -- end() is one past the last element
```

The fix: Always check `it != v.end()` before dereferencing.

0.7.17.2 Mistake 2: Iterator Invalidation

When you modify a container, its iterators may become invalid:

```
std::vector<int> v = {1, 2, 3};
auto it = v.begin();
v.push_back(4);    // may reallocate! it is now dangling
std::cout << *it;    // undefined behavior
```

Rules: - `vector`: any operation that changes size may invalidate ALL iterators - `map/set`: erasing a node invalidates ONLY that node's iterator; other iterators remain valid - `list`: inserting never invalidates iterators; erasing only invalidates the erased node

0.7.18 Exercises

Exercise 28.1 – Manual iteration

Without using a range-based for loop, iterate over `std::vector<int> v = {10, 20, 30, 40, 50}` using begin/end iterators and print each element.

Answer:

```
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << "\n";
}
```

Exercise 28.2 – Count with iterators

Use `std::count` (takes begin, end, value) to count how many times 3 appears in `std::vector<int> v = {1, 3, 2, 3, 4, 3, 5}`.

Answer:

```
int c = std::count(v.begin(), v.end(), 3); // 3
```

Chapter 29: Algorithms: sort, find, transform, and the Rest

0.7.19 Why Algorithms

The standard library provides ~100 generic algorithms in `<algorithm>` and `<numeric>`. They work on any container via iterators, are highly optimized, and eliminate common hand-written loop bugs.

```
# Python: built-in functions and list methods
nums = [3, 1, 4, 1, 5, 9, 2]
nums.sort()
filtered = [x for x in nums if x > 3]
doubled = [x * 2 for x in nums]
total = sum(nums)
```

```
// C++: algorithms from <algorithm> and <numeric>
#include <algorithm>
#include <numeric>

std::vector<int> nums = {3, 1, 4, 1, 5, 9, 2};

std::sort(nums.begin(), nums.end());

std::vector<int> filtered;
std::copy_if(nums.begin(), nums.end(),
```

```

        std::back_inserter(filtered),
        [](int x) { return x > 3; }); // lambda as predicate

std::vector<int> doubled(nums.size());
std::transform(nums.begin(), nums.end(), doubled.begin(),
               [](int x) { return x * 2; });

int total = std::accumulate(nums.begin(), nums.end(), 0);

```

0.7.20 The Most Useful Algorithms

0.7.20.1 Searching

```

std::vector<int> v = {1, 5, 2, 8, 3};

// Find first element equal to 8:
auto it = std::find(v.begin(), v.end(), 8);
if (it != v.end()) std::cout << "Found at index " << (it - v.begin()) << "\n";

// Find first element satisfying a predicate:
auto it2 = std::find_if(v.begin(), v.end(), [](int x){ return x > 6; });
// *it2 == 8

// Check if any/all/none satisfy a predicate:
bool any = std::any_of(v.begin(), v.end(), [](int x){ return x > 7; }); // true
bool all = std::all_of(v.begin(), v.end(), [](int x){ return x > 0; }); // true
bool none = std::none_of(v.begin(), v.end(), [](int x){ return x > 10; }); // true

// Count elements satisfying predicate:
int count = std::count_if(v.begin(), v.end(), [](int x){ return x % 2 == 0; }); // 1 (just 2)

// Binary search (requires sorted range):
std::vector<int> s = {1, 2, 3, 4, 5};
bool found = std::binary_search(s.begin(), s.end(), 3); // true

```

0.7.20.2 Sorting and Ordering

```

std::vector<int> v = {3, 1, 4, 1, 5, 9};

// Sort ascending (default):
std::sort(v.begin(), v.end());

// Sort descending (custom comparator):
std::sort(v.begin(), v.end(), std::greater<int>{});

// Sort with lambda comparator:
std::vector<std::string> words = {"banana", "apple", "cherry"};
std::sort(words.begin(), words.end(),

```

```

    [](const std::string& a, const std::string& b) {
        return a.size() < b.size(); // sort by length
    });
// {"apple", "banana", "cherry"} (apple=5, banana=6, cherry=6)

// Stable sort: preserves relative order of equal elements
std::stable_sort(v.begin(), v.end());

// Partial sort: get smallest 3 elements in sorted order
std::partial_sort(v.begin(), v.begin() + 3, v.end());

// nth_element: guarantees v[n] is what would be there if sorted
// All elements before v[n] are ≤ v[n]; all after are ≥ v[n]
std::nth_element(v.begin(), v.begin() + 3, v.end());

```

0.7.20.3 Transforming

```

std::vector<int> v = {1, 2, 3, 4, 5};
std::vector<int> result(v.size());

// Apply function to each element, write to result:
std::transform(v.begin(), v.end(), result.begin(),
    [](int x){ return x * x; });
// result = {1, 4, 9, 16, 25}

// transform with two input ranges:
std::vector<int> a = {1, 2, 3};
std::vector<int> b = {10, 20, 30};
std::vector<int> c(3);
std::transform(a.begin(), a.end(), b.begin(), c.begin(),
    [](int x, int y){ return x + y; });
// c = {11, 22, 33}

// Replace elements:
std::replace(v.begin(), v.end(), 3, 99); // replace all 3s with 99
std::replace_if(v.begin(), v.end(),
    [](int x){ return x > 3; }, // predicate
    0); // replacement

// Fill:
std::fill(v.begin(), v.end(), 42); // fill all with 42
std::iota(v.begin(), v.end(), 1); // fill with 1,2,3,4,5 (in <numeric>)

```

0.7.20.4 The Erase-Remove Idiom

Standard containers do not have a built-in “remove matching elements” operation. The pattern is:

```

std::vector<int> v = {1, 2, 3, 2, 4, 2, 5};

// std::remove moves non-matching elements to the front, returns new end:
auto new_end = std::remove(v.begin(), v.end(), 2);
// v = {1, 3, 4, 5, ?, ?, ?} <- unspecified values after new_end

```



```
// new_end points to the first '?'

// Erase the "removed" elements:
v.erase(new_end, v.end());
// v = {1, 3, 4, 5}

// C++20: std::erase / std::erase_if (cleaner):
std::erase(v, 2); // remove all 2s
std::erase_if(v, [](int x){ return x % 2 == 0; }); // remove all evens
```

0.7.20.5 Numeric Algorithms (<numeric>)

```
#include <numeric>
std::vector<int> v = {1, 2, 3, 4, 5};

// Sum: accumulate with + (init=0)
int sum = std::accumulate(v.begin(), v.end(), 0); // 15

// Product: accumulate with *
int product = std::accumulate(v.begin(), v.end(), 1,
                             std::multiplies<int>{}); // 120

// Prefix sums: {1, 3, 6, 10, 15}
std::vector<int> prefix(5);
std::partial_sum(v.begin(), v.end(), prefix.begin());

// Inner product (dot product):
std::vector<int> w = {2, 3, 4, 5, 6};
int dot = std::inner_product(v.begin(), v.end(), w.begin(), 0);
// 1*2 + 2*3 + 3*4 + 4*5 + 5*6 = 2+6+12+20+30 = 70
```

0.7.21 Common Mistakes in This Chapter

0.7.21.1 Mistake 1: Forgetting That `std::remove` Does Not Actually Remove

The bug:

```
std::remove(v.begin(), v.end(), 3); // returns new_end, but v is unchanged in size
std::cout << v.size(); // still original size -- elements not actually removed
```

The fix: Always pair with `.erase()`: `v.erase(std::remove(...), v.end());`

0.7.21.2 Mistake 2: Sorting Before a Linear Search (Mismatch of Algorithm)

If you only search once, linear `std::find` ($O(n)$) is fine. If you search many times, sort first and use `std::binary_search` or a set. Sorting and then linear-searching every time is $O(n \log n + n)$ when you could do $O(\log n)$ per search after a one-time sort.

0.7.22 Exercises

Exercise 29.1 – Pipeline

Given `std::vector<int> v = {5, 3, 8, 1, 9, 2, 7, 4, 6}`: 1. Sort it 2. Remove all even numbers (use erase-remove or C++20 `std::erase_if`) 3. Print the result

Answer:

```
std::sort(v.begin(), v.end());
std::erase_if(v, [](int x){ return x % 2 == 0; });
for (int n : v) std::cout << n << " "; // 1 3 5 7 9
```

Exercise 29.2 – Transform and accumulate

Given a vector of prices (doubles), apply a 10% discount to each, then compute the total.

Answer:

```
std::vector<double> prices = {10.0, 25.0, 8.5, 42.0};

std::transform(prices.begin(), prices.end(), prices.begin(),
               [](double p){ return p * 0.9; });

double total = std::accumulate(prices.begin(), prices.end(), 0.0);
std::cout << "Total after discount: " << total << "\n"; // 76.95
```

Chapter 30: Lambdas and Function Objects

0.7.23 What Is a Lambda?

A lambda is an anonymous function defined inline at the point of use. In Python they are limited to single expressions; in C++ they are full functions.

```
# Python lambda (expression only)
square = lambda x: x * x
nums = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x*x, nums))
```

```
// C++ lambda (full function, multiple statements allowed)
auto square = [](int x) { return x * x; };
std::vector<int> nums = {1, 2, 3, 4, 5};
std::transform(nums.begin(), nums.end(), nums.begin(),
               [](int x) { return x * x; });
```

0.7.24 Lambda Syntax

[capture](parameters) -> return_type { body }

[capture] -- what variables from the enclosing scope are accessible
(parameters) -- function parameters (can be omitted if none)
-> return_type -- optional: compiler deduces it if omitted
{ body } -- function body (any C++ code)

Examples:

```
auto greet = []() { std::cout << "Hello!\n"; };        // no capture, no params
auto add    = [](int a, int b) { return a + b; };       // two params, deduced return
auto mul    = [](int a, int b) -> int { return a*b; }; // explicit return type

greet();        // "Hello!"
add(3, 4);       // 7
mul(3, 4);       // 12
```

0.7.25 Captures: Accessing the Enclosing Scope

The capture clause controls which variables from the surrounding scope are accessible inside the lambda:

```
int x = 10;
int y = 20;

// Capture by value (copy):
auto f1 = [x]() { return x + 1; };    // x is copied into the lambda
x = 99;
f1();    // 11 -- uses the COPY made at capture time, not current x

// Capture by reference:
```

```

auto f2 = [&x]() { return x + 1; };
x = 99;
f2();    // 100 -- uses the CURRENT x (reference)

// Capture all by value:
auto f3 = [=]() { return x + y; };    // copies all used local variables

// Capture all by reference:
auto f4 = [&]() { return x + y; };    // references all used local variables

// Mix: capture x by value, everything else by reference:
auto f5 = [x, &]() { return x + y; };

// Capture and modify (mutable lambda):
int count = 0;
auto counter = [count]() mutable { return ++count; };
// 'mutable' allows modifying the captured copy
counter();    // 1
counter();    // 2
// count is still 0 in the enclosing scope (captured by value)

```

0.7.26 Lambda Memory Model

A lambda is syntactic sugar for a **function object** (a struct with operator()):

```

int threshold = 5;
auto above = [threshold](int x) { return x > threshold; };

// The compiler generates something like:
struct Lambda {
    int threshold;    // captured variables become data members
    Lambda(int t) : threshold{t} {}
    bool operator()(int x) const { return x > threshold; }
};
Lambda above{threshold};    // lambda = instance of this struct
above(3);    // false
above(7);    // true

```

This means lambdas are not magic – they are just convenient syntax for creating small, local function objects.

0.7.27 Generic Lambdas (C++14)

```
// 'auto' in parameters makes the lambda a template:
auto square = [](auto x) { return x * x; };

square(3);           // int: 9
square(3.14);        // double: 9.8596
square(3.0f);         // float
```

The compiler generates a different instantiation for each argument type. This is equivalent to a template operator().

0.7.28 Storing Lambdas: `std::function`

`std::function<return_type(param_types)>` can store any callable – lambda, function pointer, functor:

```
#include <functional>

// Store different callables in the same variable:
std::function<int(int, int)> op;

op = [](int a, int b) { return a + b; };
op(3, 4); // 7

op = [](int a, int b) { return a * b; };
op(3, 4); // 12

// Store in a vector of callbacks:
std::vector<std::function<void()>> callbacks;
callbacks.push_back([]{ std::cout << "first\n"; });
callbacks.push_back([]{ std::cout << "second\n"; });
for (auto& cb : callbacks) cb();
```

Caveat: `std::function` has overhead (type erasure, heap allocation for large lambdas). For performance-critical code, prefer `auto` or template parameters when the callable type is known.

0.7.29 Immediately Invoked Lambdas

```
// Call the lambda right away (useful for complex initialization):
const int value = []() {
    int result = 0;
    for (int i = 1; i <= 100; ++i) result += i;
    return result;
}(); // <-- immediately invoked

// value == 5050, computed at runtime (or at compile time if constexpr)
```

0.7.30 Common Mistakes in This Chapter

0.7.30.1 Mistake 1: Capturing a Local Variable by Reference After It Goes Out of Scope

The bug:

```
std::function<int()> make_counter() {
    int count = 0;
    return [&count]() { return ++count; }; // DANGER: count is a local variable
} // count is destroyed here
auto counter = make_counter();
counter(); // undefined behavior: accesses destroyed 'count'
```

The fix: Capture by value [count] with mutable, or use a `shared_ptr<int>`:

```
return [count]() mutable { return ++count; }; // captures a copy -- safe
```

0.7.30.2 Mistake 2: [=] Capturing this by Value

The bug:

```
class Foo {
    int x = 10;
    auto make_lambda() {
        return [=]() { return x; }; // captures 'this' by pointer, not x by value!
        // 'x' inside a member function means 'this->x'
        // [=] captures 'this' by value (the pointer), not the object
    }
};
```

The fix: In C++17+, use `[*this]` to capture the entire object by value, or `[x = this->x]` to capture the specific member.

0.7.31 Exercises

Exercise 30.1 – Sort with lambda

Sort `std::vector<std::string>` words by length (shortest first). Break ties alphabetically.

Answer:

```
std::sort(words.begin(), words.end(),
    [](const std::string& a, const std::string& b) {
        if (a.size() != b.size()) return a.size() < b.size();
        return a < b; // alphabetical for same length
    });
```

Exercise 30.2 – Closure counter

Write a function `make_counter(int start)` that returns a lambda. Each call to the lambda returns the next integer, starting from `start`.

Answer:

```
auto make_counter(int start) {
    return [n = start]() mutable { return n++; };
}

auto c = make_counter(5);
c(); // 5
c(); // 6
c(); // 7
```

Chapter 31: Ranges and Views (C++20)

0.7.32 The Problem With Iterator Pairs

The traditional algorithm interface requires two iterators:

```
std::sort(v.begin(), v.end());
std::find(v.begin(), v.end(), 3);
```

This is verbose and error-prone (mismatched iterators from different containers). You also cannot compose operations cleanly:

```
// Sort, then filter, then transform -- with traditional algorithms:
std::sort(v.begin(), v.end());
std::vector<int> filtered;
std::copy_if(v.begin(), v.end(), std::back_inserter(filtered),
    [](int x){ return x % 2 == 0; });
std::transform(filtered.begin(), filtered.end(), filtered.begin(),
    [](int x){ return x * x; });
// Each step creates a new vector -- three allocations
```

0.7.33 Ranges: The C++20 Solution

C++20 introduces `std::ranges`, which lets algorithms take entire containers directly, and `std::views`, which provides lazy composable transformations:

```
#include <ranges>
#include <algorithm>

std::vector<int> v = {5, 3, 1, 4, 2};

// Algorithms take the whole range:
std::ranges::sort(v); // {1, 2, 3, 4, 5}
auto it = std::ranges::find(v, 3); // finds 3
bool any = std::ranges::any_of(v, [](int x){ return x > 4; }); // true
```

No more `v.begin()`, `v.end()`. The algorithm accepts the container directly.

0.7.34 Views: Lazy, Composable Transformations

A **view** is a lightweight, lazy adapter over a range. It does not copy data – it creates a “window” that transforms elements on demand:

```
# Python generators: lazy transformations
nums = range(1, 11)
evens = (x for x in nums if x % 2 == 0)
squared = (x*x for x in evens)
# Nothing computed yet -- all lazy
list(squared) # [4, 16, 36, 64, 100] -- computed here
```

```
// C++20 views: lazy transformations
#include <ranges>

std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// Compose views with | (pipe operator):
auto result = v
    | std::views::filter([](int x){ return x % 2 == 0; }) // keep evens
    | std::views::transform([](int x){ return x * x; }); // square them

for (int n : result) {
    std::cout << n << " "; // 4 16 36 64 100
}
// Nothing computed until the loop iterates! Zero intermediate vectors.
```

The `|` operator chains views. Each view wraps the previous one lazily. When you iterate, each element is pulled through the chain one at a time – no intermediate allocations.

0.7.35 Common Views

```
namespace sv = std::views; // shorthand

std::vector<int> v = {1, 2, 3, 4, 5};

// filter: keep elements satisfying predicate
auto evens = v | sv::filter([](int x){ return x%2 == 0; });

// transform: apply function to each element
auto doubled = v | sv::transform([](int x){ return x * 2; });

// take: first N elements
auto first3 = v | sv::take(3);           // {1, 2, 3}

// drop: skip first N elements
auto last3 = v | sv::drop(2);           // {3, 4, 5}

// reverse: iterate backward
auto rev = v | sv::reverse;             // {5, 4, 3, 2, 1}

// iota: generate integers
auto nums = sv::iota(1, 6);             // {1, 2, 3, 4, 5} (no vector needed)

// zip (C++23): iterate two ranges together
auto zipped = sv::zip(v, std::vector{10,20,30,40,50});
for (auto [a, b] : zipped)
    std::cout << a << "+" << b << " "; // 1+10 2+20 3+30 4+40 5+50
```

0.7.36 Materializing a View

Since views are lazy, they produce values on demand. To get a concrete container, iterate into one:

```
auto result_view = v
    | sv::filter([](int x){ return x > 2; })
    | sv::transform([](int x){ return x * 10; });

// Materialize into a vector:
std::vector<int> result(result_view.begin(), result_view.end());
// or in C++23:
std::vector<int> result2 = result_view | std::ranges::to<std::vector>();
```

0.7.37 Common Mistakes in This Chapter

0.7.37.1 Mistake 1: Storing a View to a Destroyed Container

The bug:

```

auto get_view() {
    std::vector<int> v = {1, 2, 3};
    return v | std::views::filter([](int x){ return x > 1; });
} // v is destroyed here!

auto view = get_view();
for (int n : view) { ... } // undefined behavior: iterating view over dead vector

```

The fix: Views are non-owning. The source must outlive the view. Materialize to a vector if you need to return data.

0.7.38 Exercises

Exercise 31.1 – Views pipeline

Using `std::views`, create a pipeline that: takes `{1..20}`, keeps multiples of 3, squares them, and takes the first 4 results. Print them.

Answer:

```

auto result = std::views::iota(1, 21)
    | std::views::filter([](int x){ return x % 3 == 0; })
    | std::views::transform([](int x){ return x * x; })
    | std::views::take(4);

for (int n : result) std::cout << n << " ";
// multiples of 3: 3,6,9,12,... -> squared: 9,36,81,144 -> first 4: 9 36 81 144

```

Chapter 32: Utility Types: `optional`, `variant`, `any`, `tuple`

0.7.39 `std::optional<T>` – A Value That May Not Exist

`optional<T>` holds either a `T` or nothing. It replaces the pattern of using a sentinel value (`-1`, `nullptr`, `""`) to mean “no value.”

```

# Python: None means "no value"
def find_user(id: int) -> dict | None:
    if id in db: return db[id]
    return None

user = find_user(42)
if user is not None:
    print(user["name"])

```

```
// C++: std::optional
#include <optional>

std::optional<std::string> find_user(int id) {
    if (id == 42) return std::string{"Alice"};
    return std::nullopt;    // no value
}

auto user = find_user(42);
if (user) {                // bool conversion: true if has value
    std::cout << *user << "\n";    // dereference like a pointer
    std::cout << user.value() << "\n"; // or .value() (throws if empty)
}

// With default:
std::string name = find_user(99).value_or("Unknown"); // "Unknown"
```

Use `optional` instead of: - Sentinel values (-1 for “not found”) - `bool` out-parameter pairs - Pointers just to express “maybe no value”

0.7.40 `std::variant<Types...>` – Type-Safe Union

`variant` holds exactly one value from a set of possible types. It is like a tagged union – you always know which type is currently stored.

```
# Python: dynamic typing handles this naturally
result = 42        # could be int
result = "error"    # or str
result = 3.14       # or float
```

```
// C++: variant -- exactly one of the listed types
#include <variant>

std::variant<int, std::string, double> v;

v = 42;
v = "hello";    // now holds string
v = 3.14;       // now holds double

// Check which type:
if (std::holds_alternative<double>(v)) {
    std::cout << "double: " << std::get<double>(v) << "\n";
}

// Visit: apply a callable to whatever is stored
std::visit([](auto& val) {
    std::cout << val << "\n"; // works for all types (generic lambda)
}, v);

// Pattern matching with overloaded visitor:
struct Visitor {
```

```

void operator()(int i)          { std::cout << "int: " << i << "\n"; }
void operator()(const std::string& s) { std::cout << "str: " << s << "\n"; }
void operator()(double d)        { std::cout << "dbl: " << d << "\n"; }
};

std::visit(Visitor{}, v);

```

0.7.40.1 Practical Use: Error Handling

```

// Return either a value or an error message:
std::variant<int, std::string> parse_int(const std::string& s) {
    try {
        return std::stoi(s);          // success: return int
    } catch (...) {
        return std::string{"parse error: "} + s; // failure: return error
    }
}

auto result = parse_int("42");
if (auto val = std::get_if<int>(&result)) {
    std::cout << "Parsed: " << *val << "\n";
} else {
    std::cout << "Error: " << std::get<std::string>(result) << "\n";
}

```

0.7.41 std::any – Type-Erased Storage

std::any can hold a value of any copyable type. Unlike variant, the type is not fixed at compile time:

```

#include <any>

std::any a = 42;
a = std::string{"hello"};
a = 3.14;

// Access requires knowing the type:
std::cout << std::any_cast<double>(a) << "\n";    // 3.14
std::any_cast<int>(a);    // throws std::bad_any_cast: stored type is double

// Safe version:
if (double* p = std::any_cast<double>(&a)) {
    std::cout << *p << "\n";    // 3.14
}

```

Use any when the type is genuinely unknown at compile time (plugin systems, scripting interfaces). Prefer variant when the set of possible types is known.

0.7.42 `std::tuple` Revisited: Structured Bindings and Helpers

```
// Creating tuples:
auto t = std::make_tuple(42, 3.14, std::string{"hi"});

// Access:
auto& [n, d, s] = t;    // structured binding (C++17)
std::get<0>(t);         // 42

// std::tie: bind tuple elements to existing variables
int x; double y; std::string z;
std::tie(x, y, z) = t;

// Return multiple values from a function:
std::tuple<int, int> div_rem(int a, int b) {
    return {a/b, a%b};
}
auto [quotient, remainder] = div_rem(17, 5);
std::cout << quotient << " r " << remainder << "\n"; // 3 r 2
```

0.7.43 `std::pair<A, B>` – Two Values

`pair` is `tuple` with exactly two elements, more readable names:

```
std::pair<int, std::string> p{42, "hello"};
std::cout << p.first << " " << p.second << "\n";

auto [num, str] = p;    // structured binding

// Common with maps:
for (const auto& [key, val] : my_map) {
    std::cout << key << " -> " << val << "\n";
}
```

0.7.44 Choosing the Right Utility Type

A value that might not exist:

`std::optional<T>` -- simple, clear, fast

A value that is one of several known types:

`std::variant<A,B,C>` -- type-safe, visit with pattern matching

A value of unknown type (runtime):

`std::any` -- flexible, slower, use sparingly

Multiple values of known types, fixed number:

`std::tuple<A,B,C>` -- heterogeneous, structured bindings make it clean

Two values:

`std::pair<A,B>` -- when two is the right number

0.7.45 Common Mistakes in This Chapter

0.7.45.1 Mistake 1: Accessing Empty optional

The bug:

```
std::optional<int> opt;
std::cout << *opt;           // undefined behavior -- opt has no value
std::cout << opt.value();    // throws std::bad_optional_access
```

The fix: Check if (opt) or use `opt.value_or(default)`.

0.7.45.2 Mistake 2: std::get with Wrong Type on variant

The bug:

```
std::variant<int, std::string> v = 42;
std::get<std::string>(v);    // throws std::bad_variant_access
```

The fix: Use `std::holds_alternative<T>(v)` to check, or `std::get_if<T>(&v)` which returns `nullptr` on mismatch.

0.7.46 Exercises

Exercise 32.1 – Safe division

Write `safe_divide(int a, int b)` returning `std::optional<double>`. Return `std::nullopt` if `b == 0`.

Answer:

```
std::optional<double> safe_divide(int a, int b) {
    if (b == 0) return std::nullopt;
    return static_cast<double>(a) / b;
}

auto r = safe_divide(10, 3);
std::cout << r.value_or(0.0) << "\n";    // 3.333...

auto r2 = safe_divide(10, 0);
std::cout << r2.value_or(0.0) << "\n";    // 0.0
```

Exercise 32.2 – Shape variant

Use `std::variant<Circle, Rectangle, Triangle>` (define these as simple structs with an `area()` method) and `std::visit` to compute and print the area of each shape in a vector.

Answer:

```
struct Circle { double r; double area() const { return 3.14159*r*r; } };
struct Rectangle { double w, h; double area() const { return w*h; } };
struct Triangle { double b, h; double area() const { return 0.5*b*h; } };

using Shape = std::variant<Circle, Rectangle, Triangle>;

std::vector<Shape> shapes = {
    Circle{5.0},
    Rectangle{3.0, 4.0},
    Triangle{6.0, 8.0}
};

for (const auto& shape : shapes) {
    double area = std::visit([](const auto& s){ return s.area(); }, shape);
    std::cout << "Area: " << area << "\n";
}
// Area: 78.5398
// Area: 12
// Area: 24
```

Exercise 32.3 – Multiple return values

Write `parse_date(std::string)` that parses “YYYY-MM-DD” and returns a `std::tuple<int, int, int>` (year, month, day). Use structured bindings to unpack it.

Answer:

```
#include <tuple>
#include <string>

std::tuple<int,int,int> parse_date(const std::string& s) {
    // Format: "YYYY-MM-DD"
    int y = std::stoi(s.substr(0, 4));
    int m = std::stoi(s.substr(5, 2));
    int d = std::stoi(s.substr(8, 2));
    return {y, m, d};
}

auto [year, month, day] = parse_date("2026-06-28");
std::cout << year << "/" << month << "/" << day << "\n"; // 2026/6/28
```

Part VI is complete. You now know the standard library well enough to write real programs: choosing the right container for each job, using iterators correctly, composing algorithms and lambdas, building lazy data pipelines with ranges and views, and using the utility types that make C++ code expressive without sacrificing performance.

Part VII covers modern C++ language features added since C++11 – auto, constexpr, std::format, coroutines, and modules. Ask to continue.